RNN & LSTM for Sequences: Variants, Evaluations and Applications

Table of Contents

Communicated by Ronald Williams

Long Short-Term Memory

Sepp Hochreiter
Fakultät für Informatik, Technische Universität München, 80290 München, Germany

Jürgen Schmidhuber
IDSIA, Corso Elvezia 36, 6900 Lugano, Switzerland

Learning to store information over extended time intervals by recurrent backpropagation takes a very long time, mostly because of insufficient, decaying error backflow. We briefly review Hochreiter's (1991) analysis of this problem, then address it by introducing a novel, efficient, gradient-based method called long short-term memory (LSTM). Truncating the gradient where this does not do harm, LSTM can learn to bridge minimal time lags in excess of 1000 discrete-time steps by enforcing constant error flow through constant error carousels within special units. Multiplicative gate units learn to open and close access to the constant error flow. LSTM is local in space and time; its computational complexity per time step and weight is $O(1)$. Our experiments with artificial data involve local, distributed, real-valued, and noisy pattern representations. In comparisons with real-time recurrent learning, back propagation through time, recurrent cascade correlation, Elman nets, and neural sequence chunking, LSTM leads to many more successful runs, and learns much faster. LSTM also solves complex, artificial long-time-lag tasks that have never been solved by previous recurrent network algorithms.

1 Introduction

In principle, recurrent networks can use their feedback connections to store representations of recent input events in the form of activations (short-term memory, as opposed to long-term memory embodied by slowly changing weights). This is potentially significant for many applications, including speech processing, non-Markovian control, and music composition (Mozer, 1992). The most widely used algorithms for learning what to put in short-term memory, however, take too much time or do not work well at all, especially when minimal time lags between inputs and corresponding teacher signals are long. Although theoretically fascinating, existing methods do not provide clear practical advantages over, say, backpropagation in feed-forward nets with limited time windows. This article reviews an analysis of the problem and suggests a remedy.

Neural Computation **9, 1735–1780** (1997)

The problem. With conventional backpropagation through time (BPTT; Williams & Zipser, 1992; Werbos, 1988) or real-time recurrent learning (RTRL; Robinson & Fallside, 1987), error signals flowing backward in time tend to (1) blow up or (2) vanish; the temporal evolution of the backpropagated error exponentially depends on the size of the weights (Hochreiter, 1991). Case 1 may lead to oscillating weights; in case 2, learning to bridge long time lags takes a prohibitive amount of time or does not work at all (see section 3).

This article presents long short-term memory (LSTM), a novel recurrent network architecture in conjunction with an appropriate gradient-based learning algorithm. LSTM is designed to overcome these error backflow problems. It can learn to bridge time intervals in excess of 1000 steps even in case of noisy, incompressible input sequences, without loss of short-time-lag capabilities. This is achieved by an efficient, gradient-based algorithm for an architecture enforcing constant (thus, neither exploding nor vanishing) error flow through internal states of special units (provided the gradient computation is truncated at certain architecture-specific points; this does not affect long-term error flow, though).

Section 2 briefly reviews previous work. Section 3 begins with an outline of the detailed analysis of vanishing errors due to Hochreiter (1991). It then introduces a naive approach to constant error backpropagation for didactic purposes and highlights its problems concerning information storage and retrieval. These problems lead to the LSTM architecture described in section 4. Section 5 presents numerous experiments and comparisons with competing methods. LSTM outperforms them and also learns to solve complex, artificial tasks no other recurrent net algorithm has solved. Section 6 discusses LSTM's limitations and advantages. The appendix contains a detailed description of the algorithm (A.1) and explicit error flow formulas (A.2).

2 Previous Work

This section focuses on recurrent nets with time-varying inputs (as opposed to nets with stationary inputs and fixed-point-based gradient calculations; e.g., Almeida, 1987; Pineda, 1987).

2.1 Gradient-Descent Variants. The approaches of Elman (1988), Fahlman (1991), Williams (1989), Schmidhuber (1992a), Pearlmutter (1989), and many of the related algorithms in Pearlmutter's comprehensive overview (1995) suffer from the same problems as BPTT and RTRL (see sections 1 and 3).

2.2 Time Delays. Other methods that seem practical for short time lags only are time-delay neural networks (Lang, Waibel, & Hinton, 1990) and Plate's method (Plate, 1993), which updates unit activations based on a

weighted sum of old activations (see also de Vries & Principe, 1991). Lin et al. (1996) propose variants of time-delay networks called NARX networks.

2.3 Time Constants. To deal with long time lags, Mozer (1992) uses time constants influencing changes of unit activations (deVries and Principe's 1991 approach may in fact be viewed as a mixture of time-delay neural networks and time constants). For long time lags, however, the time constants need external fine tuning (Mozer, 1992). Sun, Chen, and Lee's alternative approach (1993) updates the activation of a recurrent unit by adding the old activation and the (scaled) current net input. The net input, however, tends to perturb the stored information, which makes long-term storage impractical.

2.4 Ring's Approach. Ring (1993) also proposed a method for bridging long time lags. Whenever a unit in his network receives conflicting error signals, he adds a higher-order unit influencing appropriate connections. Although his approach can sometimes be extremely fast, to bridge a time lag involving 100 steps may require the addition of 100 units. Also, Ring's net does not generalize to unseen lag durations.

2.5 Bengio et al.'s Approach. Bengio, Simard, and Frasconi (1994) investigate methods such as simulated annealing, multigrid random search, time-weighted pseudo-Newton optimization, and discrete error propagation. Their "latch" and "two-sequence" problems are very similar to problem 3a in this article with minimal time lag 100 (see Experiment 3). Bengio and Frasconi (1994) also propose an expectation-maximazation approach for propagating targets. With n so-called state networks, at a given time, their system can be in one of only n different states. (See also the beginning of section 5.) But to solve continuous problems such as the adding problem (section 5.4), their system would require an unacceptable number of states (i.e., state networks).

2.6 Kalman Filters. Puskorius and Feldkamp (1994) use Kalman filter techniques to improve recurrent net performance. Since they use "a derivative discount factor imposed to decay exponentially the effects of past dynamic derivatives," there is no reason to believe that their Kalman filter-trained recurrent networks will be useful for very long minimal time lags.

2.7 Second Order Nets. We will see that LSTM uses multiplicative units (MUs) to protect error flow from unwanted perturbations. It is not the first recurrent net method using MUs, though. For instance, Watrous and Kuhn (1992) use MUs in second-order nets. There are some differences from LSTM: (1) Watrous and Kuhn's architecture does not enforce constant error flow and is not designed to solve long-time-lag problems; (2) it has fully connected second-order sigma-pi units, while the LSTM architecture's MUs

are used only to gate access to constant error flow; and (3) Watrous and Kuhn's algorithm costs $O(W^2)$ operations per time step, ours only $O(W)$, where W is the number of weights. See also Miller and Giles (1993) for additional work on MUs.

2.8 Simple Weight Guessing. To avoid long-time-lag problems of gradient-based approaches, we may simply randomly initialize all network weights until the resulting net happens to classify all training sequences correctly. In fact, recently we discovered (Schmidhuber & Hochreiter, 1996; Hochreiter & Schmidhuber, 1996, 1997) that simple weight guessing solves many of the problems in Bengio et al. (1994), Bengio and Frasconi (1994), Miller and Giles (1993), and Lin et al. (1996) faster than the algorithms these authors proposed. This does not mean that weight guessing is a good algorithm. It just means that the problems are very simple. More realistic tasks require either many free parameters (e.g., input weights) or high weight precision (e.g., for continuous-valued parameters), such that guessing becomes completely infeasible.

2.9 Adaptive Sequence Chunkers. Schmidhuber's hierarchical chunker systems (1992b, 1993) do have a capability to bridge arbitrary time lags, but only if there is local predictability across the subsequences causing the time lags (see also Mozer, 1992). For instance, in his postdoctoral thesis, Schmidhuber (1993) uses hierarchical recurrent nets to solve rapidly certain grammar learning tasks involving minimal time lags in excess of 1000 steps. The performance of chunker systems, however, deteriorates as the noise level increases and the input sequences become less compressible. LSTM does not suffer from this problem.

3 Constant Error Backpropagation

3.1 Exponentially Decaying Error

3.1.1 Conventional BPTT (e.g., Williams & Zipser, 1992). Output unit k's target at time t is denoted by $d_k(t)$. Using mean squared error, k's error signal is

$$\vartheta_k(t) = f'_k(net_k(t))(d_k(t) - y^k(t)),$$

where

$$y^i(t) = f_i(net_i(t))$$

is the activation of a noninput unit i with differentiable activation function f_i,

$$net_i(t) = \sum_j w_{ij} y^j(t-1)$$

is unit i's current net input, and w_{ij} is the weight on the connection from unit j to i. Some nonoutput unit j's backpropagated error signal is

$$\vartheta_j(t) = f_j'(net_j(t)) \sum_i w_{ij}\vartheta_i(t+1).$$

The corresponding contribution to w_{jl}'s total weight update is $\alpha\vartheta_j(t)y^l(t-1)$, where α is the learning rate and l stands for an arbitrary unit connected to unit j.

3.1.2 Outline of Hochreiter's Analysis (1991, pp. 19–21). Suppose we have a fully connected net whose noninput unit indices range from 1 to n. Let us focus on local error flow from unit u to unit v (later we will see that the analysis immediately extends to global error flow). The error occurring at an arbitrary unit u at time step t is propagated back into time for q time steps, to an arbitrary unit v. This will scale the error by the following factor:

$$\frac{\partial\vartheta_v(t-q)}{\partial\vartheta_u(t)} = \begin{cases} f_v'(net_v(t-1))w_{uv} & q=1 \\ f_v'(net_v(t-q))\sum_{l=1}^{n}\frac{\partial\vartheta_l(t-q+1)}{\partial\vartheta_u(t)}w_{lv} & q>1 \end{cases}. \tag{3.1}$$

With $l_q = v$ and $l_0 = u$, we obtain:

$$\frac{\partial\vartheta_v(t-q)}{\partial\vartheta_u(t)} = \sum_{l_1=1}^{n}\cdots\sum_{l_{q-1}=1}^{n}\prod_{m=1}^{q} f_{l_m}'(net_{l_m}(t-m))w_{l_m l_{m-1}} \tag{3.2}$$

(proof by induction). The sum of the n^{q-1} terms $\prod_{m=1}^{q} f_{l_m}'(net_{l_m}(t-m))w_{l_m l_{m-1}}$ determines the total error backflow (note that since the summation terms may have different signs, increasing the number of units n does not necessarily increase error flow).

3.1.3 Intuitive Explanation of Equation 3.2. If

$$|f_{l_m}'(net_{l_m}(t-m))w_{l_m l_{m-1}}| > 1.0$$

for all m (as can happen, e.g., with linear f_{l_m}), then the largest product increases exponentially with q. That is, the error blows up, and conflicting error signals arriving at unit v can lead to oscillating weights and unstable learning (for error blowups or bifurcations, see also Pineda, 1988; Baldi & Pineda, 1991; Doya, 1992). On the other hand, if

$$|f_{l_m}'(net_{l_m}(t-m))w_{l_m l_{m-1}}| < 1.0$$

for all m, then the largest product decreases exponentially with q. That is, the error vanishes, and nothing can be learned in acceptable time.

If f_{l_m} is the logistic sigmoid function, then the maximal value of f'_{l_m} is 0.25. If $y^{l_{m-1}}$ is constant and not equal to zero, then $|f'_{l_m}(net_{l_m})w_{l_m l_{m-1}}|$ takes on maximal values where

$$w_{l_m l_{m-1}} = \frac{1}{y^{l_{m-1}}} \coth\left(\frac{1}{2} net_{l_m}\right),$$

goes to zero for $|w_{l_m l_{m-1}}| \to \infty$, and is less than 1.0 for $|w_{l_m l_{m-1}}| < 4.0$ (e.g., if the absolute maximal weight value $w_{\max}$ is smaller than 4.0). Hence with conventional logistic sigmoid activation functions, the error flow tends to vanish as long as the weights have absolute values below 4.0, especially in the beginning of the training phase. In general, the use of larger initial weights will not help, though, as seen above, for $|w_{l_m l_{m-1}}| \to \infty$ the relevant derivative goes to zero "faster" than the absolute weight can grow (also, some weights will have to change their signs by crossing zero). Likewise, increasing the learning rate does not help either; it will not change the ratio of long-range error flow and short-range error flow. BPTT is too sensitive to recent distractions. (A very similar, more recent analysis was presented by Bengio et al., 1994.)

3.1.4 Global Error Flow. The local error flow analysis above immediately shows that global error flow vanishes too. To see this, compute

$$\sum_{u:\ u \text{ output unit}} \frac{\partial \vartheta_v(t-q)}{\partial \vartheta_u(t)}.$$

3.1.5 Weak Upper Bound for Scaling Factor. The following, slightly extended vanishing error analysis also takes n, the number of units, into account. For $q > 1$, equation 3.2 can be rewritten as

$$(W_{u^T})^T F'(t-1) \prod_{m=2}^{q-1} (WF'(t-m))\ W_v\ f'_v(net_v(t-q)),$$

where the weight matrix W is defined by $[W]_{ij} := w_{ij}$, v's outgoing weight vector W_v is defined by $[W_v]_i := [W]_{iv} = w_{iv}$, u's incoming weight vector W_{u^T} is defined by $[W_{u^T}]_i := [W]_{ui} = w_{ui}$, and for $m = 1, \ldots, q$, $F'(t-m)$ is the diagonal matrix of first-order derivatives defined as $[F'(t-m)]_{ij} := 0$ if $i \neq j$, and $[F'(t-m)]_{ij} := f'_i(net_i(t-m))$ otherwise. Here T is the transposition operator, $[A]_{ij}$ is the element in the ith column and jth row of matrix A, and $[x]_i$ is the ith component of vector x.

Using a matrix norm $\|\cdot\|_A$ compatible with vector norm $\|\cdot\|_x$, we define

$$f'_{\max} := \max_{m=1,\ldots,q} \{\|F'(t-m)\|_A\}.$$

For $\max_{i=1,\ldots,n}\{|x_i|\} \leq \|x\|_x$ we get $|x^T y| \leq n \|x\|_x \|y\|_x$. Since

$$|f'_v(net_v(t-q))| \leq \|F'(t-q)\|_A \leq f'_{\max},$$

we obtain the following inequality:

$$\left|\frac{\partial \vartheta_v(t-q)}{\partial \vartheta_u(t)}\right| \;\leq\; n\,(f'_{\max})^q\,\|W_v\|_x\,\|W_{u^T}\|_x\,\|W\|_A^{q-2} \;\leq\; n\left(f'_{\max}\,\|W\|_A\right)^q.$$

This inequality results from

$$\|W_v\|_x \;=\; \|We_v\|_x \;\leq\; \|W\|_A\,\|e_v\|_x \;\leq\; \|W\|_A$$

and

$$\|W_{u^T}\|_x \;=\; \|W^T e_u\|_x \;\leq\; \|W\|_A\,\|e_u\|_x \;\leq\; \|W\|_A,$$

where e_k is the unit vector whose components are 0 except for the kth component, which is 1. Note that this is a weak, extreme case upper bound; it will be reached only if all $\|F'(t-m)\|_A$ take on maximal values, and if the contributions of all paths across which error flows back from unit u to unit v have the same sign. Large $\|W\|_A$, however, typically result in small values of $\|F'(t-m)\|_A$, as confirmed by experiments (see, e.g., Hochreiter, 1991).

For example, with norms

$$\|W\|_A \;:=\; \max_r \sum_s |w_{rs}|$$

and

$$\|x\|_x := \max_r |x_r|,$$

we have $f'_{\max} = 0.25$ for the logistic sigmoid. We observe that if

$$|w_{ij}| \leq w_{\max} < \frac{4.0}{n} \;\; \forall i, j,$$

then $\|W\|_A \;\leq nw_{\max} < 4.0$ will result in exponential decay. By setting $\tau := \left(\frac{nw_{\max}}{4.0}\right) < 1.0$, we obtain

$$\left|\frac{\partial \vartheta_v(t-q)}{\partial \vartheta_u(t)}\right| \leq n\,(\tau)^q.$$

We refer to Hochreiter (1991) for additional results.

3.2 Constant Error Flow: Naive Approach.

3.2.1 A Single Unit. To avoid vanishing error signals, how can we achieve constant error flow through a single unit j with a single connection to itself? According to the rules above, at time t, j's local error backflow is $\vartheta_j(t) = f'_j(net_j(t))\vartheta_j(t+1)w_{jj}$. To enforce constant error flow through j, we require

$$f'_j(net_j(t))w_{jj} = 1.0.$$

Note the similarity to Mozer's fixed time constant system (1992)—a time constant of 1.0 is appropriate for potentially infinite time lags.[1]

3.2.2 The Constant Error Carousel. Integrating the differential equation above, we obtain

$$f_j(net_j(t)) = \frac{net_j(t)}{w_{jj}}$$

for arbitrary $net_j(t)$. This means f_j has to be linear, and unit j's activation has to remain constant:

$$y_j(t+1) = f_j(net_j(t+1)) = f_j(w_{jj}y^j(t)) = y^j(t).$$

In the experiments, this will be ensured by using the identity function f_j : $f_j(x) = x, \forall x$, and by setting $w_{jj} = 1.0$. We refer to this as the constant error carousel (CEC). CEC will be LSTM's central feature (see section 4).

Of course, unit j will not only be connected to itself but also to other units. This invokes two obvious, related problems (also inherent in all other gradient-based approaches):

1. Input weight conflict: For simplicity, let us focus on a single additional input weight w_{ji}. Assume that the total error can be reduced by switching on unit j in response to a certain input and keeping it active for a long time (until it helps to compute a desired output). Provided i is nonzero, since the same incoming weight has to be used for both storing certain inputs and ignoring others, w_{ji} will often receive conflicting weight update signals during this time (recall that j is linear). These signals will attempt to make w_{ji} participate in (1) storing the input (by switching on j) and (2) protecting the input (by preventing j from being switched off by irrelevant later inputs). This conflict makes learning difficult and calls for a more context-sensitive mechanism for controlling write operations through input weights.
2. Output weight conflict: Assume j is switched on and currently stores some previous input. For simplicity, let us focus on a single additional outgoing weight w_{kj}. The same w_{kj} has to be used for both retrieving j's content at certain times and preventing j from disturbing k at other times. As long as unit j is nonzero, w_{kj} will attract conflicting weight update signals generated during sequence processing. These signals will attempt to make w_{kj} participate in accessing the information stored in j and—at different times—protecting unit k from being perturbed by j. For instance, with many tasks there are certain short-time-lag errors that can be reduced in early training stages. However,

[1] We do not use the expression "time constant" in the differential sense, as Pearlmutter (1995) does.

at later training stages, j may suddenly start to cause avoidable errors in situations that already seemed under control by attempting to participate in reducing more difficult long-time-lag errors. Again, this conflict makes learning difficult and calls for a more context-sensitive mechanism for controlling read operations through output weights.

Of course, input and output weight conflicts are not specific for long time lags; they occur for short time lags as well. Their effects, however, become particularly pronounced in the long-time-lag case. As the time lag increases, stored information must be protected against perturbation for longer and longer periods, and, especially in advanced stages of learning, more and more already correct outputs also require protection against perturbation.

Due to the problems set out, the naive approach does not work well except in the case of certain simple problems involving local input-output representations and nonrepeating input patterns (see Hochreiter, 1991; Silva, Amarel, Langlois, & Almeida, 1996). The next section shows how to do it right.

4 The Concept of Long Short-Term Memory

4.1 Memory Cells and Gate Units. To construct an architecture that allows for constant error flow through special, self-connected units without the disadvantages of the naive approach, we extend the CEC embodied by the self-connected, linear unit j from section 3.2 by introducing additional features. A multiplicative input gate unit is introduced to protect the memory contents stored in j from perturbation by irrelevant inputs, and a multiplicative output gate unit is introduced to protect other units from perturbation by currently irrelevant memory contents stored in j.

The resulting, more complex unit is called a memory cell (see Figure 1). The jth memory cell is denoted c_j. Each memory cell is built around a central linear unit with a fixed self-connection (the CEC). In addition to net_{c_j}, c_j gets input from a multiplicative unit out_j (the output gate), and from another multiplicative unit in_j (the input gate). in_j's activation at time t is denoted by $y^{in_j}(t)$, out_j's by $y^{out_j}(t)$. We have

$$y^{out_j}(t) = f_{out_j}(net_{out_j}(t)); \; y^{in_j}(t) = f_{in_j}(net_{in_j}(t));$$

where

$$net_{out_j}(t) = \sum_u w_{out_j u} y^u(t-1),$$

and

$$net_{in_j}(t) = \sum_u w_{in_j u} y^u(t-1).$$

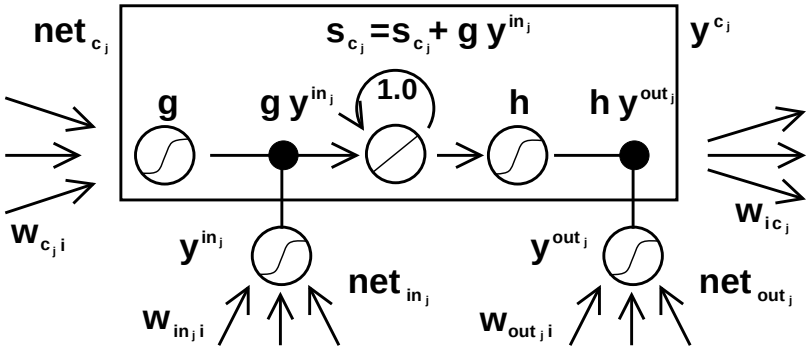


Figure 1: Architecture of memory cell c_j (the box) and its gate units in_j, out_j. The self-recurrent connection (with weight 1.0) indicates feedback with a delay of one time step. It builds the basis of the CEC. The gate units open and close access to CEC. See text and appendix A.1 for details.

We also have

$$net_{c_j}(t) = \sum_u w_{c_j u} y^u (t-1).$$

The summation indices u may stand for input units, gate units, memory cells, or even conventional hidden units if there are any (see section 4.3). All these different types of units may convey useful information about the current state of the net. For instance, an input gate (output gate) may use inputs from other memory cells to decide whether to store (access) certain information in its memory cell. There even may be recurrent self-connections like $w_{c_j c_j}$. It is up to the user to define the network topology. See Figure 2 for an example.

At time t, c_j's output $y^{c_j}(t)$ is computed as

$$y^{c_j}(t) = y^{out_j}(t) h(s_{c_j}(t)),$$

where the internal state $s_{c_j}(t)$ is

$$s_{c_j}(0) = 0, \; s_{c_j}(t) = s_{c_j}(t-1) + y^{in_j}(t) g\left(net_{c_j}(t)\right) \text{ for } t > 0.$$

The differentiable function g squashes net_{c_j}; the differentiable function h scales memory cell outputs computed from the internal state s_{c_j}.

4.2 Why Gate Units? To avoid input weight conflicts, in_j controls the error flow to memory cell c_j's input connections $w_{c_j i}$. To circumvent c_j's output weight conflicts, out_j controls the error flow from unit j's output

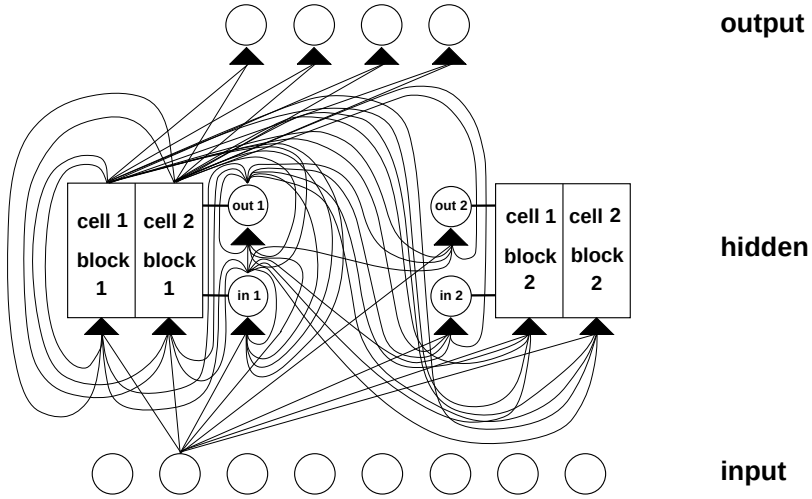


Figure 2: Example of a net with eight input units, four output units, and two memory cell blocks of size 2. *in*1 marks the input gate, *out*1 marks the output gate, and *cell*1/*block*1 marks the first memory cell of block 1. *cell*1/*block*1's architecture is identical to the one in Figure 1, with gate units *in*1 and *out*1 (note that by rotating Figure 1 by 90 degrees anticlockwise, it will match with the corresponding parts of Figure 2). The example assumes dense connectivity: each gate unit and each memory cell sees all non-output units. For simplicity, however, outgoing weights of only one type of unit are shown for each layer. With the efficient, truncated update rule, error flows only through connections to output unit, and through fixed self-connections within cell blocks (not shown here; see Figure 1). Error flow is truncated once it "wants" to leave memory cells or gate units. Therefore, no connection shown above serves to propagate error back to the unit from which the connection originates (except for connections to output units), although the connections themselves are modifiable. That is why the truncated LSTM algorithm is so efficient, despite its ability to bridge very long time lags. See the text and the appendix for details. Figure 2 shows the architecture used for experiment 6a; only the bias of the noninput units is omitted.

connections. In other words, the net can use in_j to decide when to keep or override information in memory cell c_j and out_j to decide when to access memory cell c_j and when to prevent other units from being perturbed by c_j (see Figure 1).

Error signals trapped within a memory cell's CEC cannot change, but different error signals flowing into the cell (at different times) via its output gate may get superimposed. The output gate will have to learn which

errors to trap in its CEC by appropriately scaling them. The input gate will have to learn when to release errors, again by appropriately scaling them. Essentially the multiplicative gate units open and close access to constant error flow through CEC.

Distributed output representations typically do require output gates. Both gate types are not always necessary, though; one may be sufficient. For instance, in experiments 2a and 2b in section 5, it will be possible to use input gates only. In fact, output gates are not required in case of local output encoding; preventing memory cells from perturbing already learned outputs can be done by simply setting the corresponding weights to zero. Even in this case, however, output gates can be beneficial: they prevent the net's attempts at storing long-time-lag memories (which are usually hard to learn) from perturbing activations representing easily learnable short-time-lag memories. (This will prove quite useful in experiment 1, for instance.)

4.3 Network Topology. We use networks with one input layer, one hidden layer, and one output layer. The (fully) self-connected hidden layer contains memory cells and corresponding gate units (for convenience, we refer to both memory cells and gate units as being located in the hidden layer). The hidden layer may also contain conventional hidden units providing inputs to gate units and memory cells. All units (except for gate units) in all layers have directed connections (serve as inputs) to all units in the layer above (or to all higher layers; see experiments 2a and 2b).

4.4 Memory Cell Blocks. S memory cells sharing the same input gate and the same output gate form a structure called a memory cell block of size S. Memory cell blocks facilitate information storage. As with conventional neural nets, it is not so easy to code a distributed input within a single cell. Since each memory cell block has as many gate units as a single memory cell (namely, two), the block architecture can be even slightly more efficient. A memory cell block of size 1 is just a simple memory cell. In the experiments in section 5, we will use memory cell blocks of various sizes.

4.5 Learning. We use a variant of RTRL (e.g., Robinson & Fallside, 1987) that takes into account the altered, multiplicative dynamics caused by input and output gates. To ensure nondecaying error backpropagation through internal states of memory cells, as with truncated BPTT (e.g., Williams & Peng, 1990), errors arriving at memory cell net inputs (for cell c_j, this includes net_{c_j}, net_{in_j}, net_{out_j}) do not get propagated back further in time (although they do serve to change the incoming weights). Only within memory cells, are errors propagated back through previous internal states s_{c_j}.[2] To visualize

[2] For intracellular backpropagation in a quite different context, see also Doya and Yoshizawa (1989).

this, once an error signal arrives at a memory cell output, it gets scaled by output gate activation and h'. Then it is within the memory cell's CEC, where it can flow back indefinitely without ever being scaled. When it leaves the memory cell through the input gate and g, it is scaled once more by input gate activation and g'. It then serves to change the incoming weights before it is truncated (see the appendix for formulas).

4.6 Computational Complexity. As with Mozer's focused recurrent backpropagation algorithm (Mozer, 1989), only the derivatives $\partial s_{c_j}/\partial w_{il}$ need to be stored and updated. Hence the LSTM algorithm is very efficient, with an excellent update complexity of $O(W)$, where W the number of weights (see details in the appendix). Hence, LSTM and BPTT for fully recurrent nets have the same update complexity per time step (while RTRL's is much worse). Unlike full BPTT, however, LSTM is local in space and time:[3] there is no need to store activation values observed during sequence processing in a stack with potentially unlimited size.

4.7 Abuse Problem and Solutions. In the beginning of the learning phase, error reduction may be possible without storing information over time. The network will thus tend to abuse memory cells, for example, as bias cells (it might make their activations constant and use the outgoing connections as adaptive thresholds for other units). The potential difficulty is that it may take a long time to release abused memory cells and make them available for further learning. A similar "abuse problem" appears if two memory cells store the same (redundant) information. There are at least two solutions to the abuse problem: (1) sequential network construction (e.g., Fahlman, 1991): a memory cell and the corresponding gate units are added to the network whenever the error stops decreasing (see experiment 2 in section 5), and (2) output gate bias: each output gate gets a negative initial bias, to push initial memory cell activations toward zero. Memory cells with more negative bias automatically get "allocated" later (see experiments 1, 3, 4, 5, and 6 in section 5).

4.8 Internal State Drift and Remedies. If memory cell c_j's inputs are mostly positive or mostly negative, then its internal state s_j will tend to drift away over time. This is potentially dangerous, for the $h'(s_j)$ will then adopt very small values, and the gradient will vanish. One way to circumvent this problem is to choose an appropriate function h. But $h(x) = x$, for instance, has the disadvantage of unrestricted memory cell output range. Our simple

[3] Following Schmidhuber (1989), we say that a recurrent net algorithm is *local in space* if the update complexity per time step and weight does not depend on network size. We say that a method is *local in time* if its storage requirements do not depend on input sequence length. For instance, RTRL is local in time but not in space. BPTT is local in space but not in time.

but effective way of solving drift problems at the beginning of learning is initially to bias the input gate in_j toward zero. Although there is a trade-off between the magnitudes of $h'(s_j)$ on the one hand and of y^{in_j} and f'_{in_j} on the other, the potential negative effect of input gate bias is negligible compared to the one of the drifting effect. With logistic sigmoid activation functions, there appears to be no need for fine-tuning the initial bias, as confirmed by experiments 4 and 5 in section 5.4.

5 Experiments

Which tasks are appropriate to demonstrate the quality of a novel long-time-lag algorithm? First, minimal time lags between relevant input signals and corresponding teacher signals must be long for all training sequences. In fact, many previous recurrent net algorithms sometimes manage to generalize from very short training sequences to very long test sequences (see, e.g., Pollack, 1991). But a real long-time-lag problem does not have any short-time-lag exemplars in the training set. For instance, Elman's training procedure, BPTT, offline RTRL, online RTRL, and others fail miserably on real long-time-lag problems. (See, e.g., Hochreiter, 1991; Mozer, 1992.) A second important requirement is that the tasks should be complex enough such that they cannot be solved quickly by simple-minded strategies such as random weight guessing.

Recently we discovered (Schmidhuber & Hochreiter, 1996; Hochreiter & Schmidhuber, 1996, 1997) that many long-time-lag tasks used in previous work can be solved more quickly by simple random weight guessing than by the proposed algorithms. For instance, guessing solved a variant of Bengio and Frasconi's parity problem (1994) much faster[4] than the seven methods tested by Bengio et al. (1994) and Bengio and Frasconi (1994). The same is true for some of Miller and Giles's problems (1993). Of course, this does not mean that guessing is a good algorithm. It just means that some previously used problems are not extremely appropriate to demonstrate the quality of previously proposed algorithms.

All our experiments (except experiment 1) involve long minimal time lags; there are no short-time-lag training exemplars facilitating learning. Solutions to most of our tasks are sparse in weight space. They require either many parameters and inputs or high weight precision, such that random weight guessing becomes infeasible.

We always use online learning (as opposed to batch learning) and logistic sigmoids as activation functions. For experiments 1 and 2, initial weights are chosen in the range $[-0.2, 0.2]$, for the other experiments in $[-0.1, 0.1]$. Training sequences are generated randomly according to the various task

[4] Different input representations and different types of noise may lead to worse guessing performance (Yoshua Bengio, personal communication, 1996).

descriptions. In slight deviation from the notation in appendix A.1, each discrete time step of each input sequence involves three processing steps: (1) use current input to set the input units, (2) compute activations of hidden units (including input gates, output gates, memory cells), and (3) compute output unit activations. Except for experiments 1, 2a, and 2b, sequence elements are randomly generated online, and error signals are generated only at sequence ends. Net activations are reset after each processed input sequence.

For comparisons with recurrent nets taught by gradient descent, we give results only for RTRL, except for comparison 2a, which also includes BPTT. Note, however, that untruncated BPTT (see, e.g., Williams & Peng, 1990) computes exactly the same gradient as offline RTRL. With long-time-lag problems, offline RTRL (or BPTT) and the online version of RTRL (no activation resets, online weight changes) lead to almost identical, negative results (as confirmed by additional simulations in Hochreiter, 1991; see also Mozer, 1992). This is because offline RTRL, online RTRL, and full BPTT all suffer badly from exponential error decay.

Our LSTM architectures are selected quite arbitrarily. If nothing is known about the complexity of a given problem, a more systematic approach would be to: start with a very small net consisting of one memory cell. If this does not work, try two cells, and so on. Alternatively, use sequential network construction (e.g., Fahlman, 1991).

Following is an outline of the experiments:

- Experiment 1 focuses on a standard benchmark test for recurrent nets: the embedded Reber grammar. Since it allows for training sequences with short time lags, it is not a long-time-lag problem. We include it because it provides a nice example where LSTM's output gates are truly beneficial, and it is a popular benchmark for recurrent nets that has been used by many authors. We want to include at least one experiment where conventional BPTT and RTRL do not fail completely (LSTM, however, clearly outperforms them). The embedded Reber grammar's minimal time lags represent a border case in the sense that it is still possible to learn to bridge them with conventional algorithms. Only slightly longer minimal time lags would make this almost impossible. The more interesting tasks in our article, however, are those that RTRL, BPTT, and others cannot solve at all.
- Experiment 2 focuses on noise-free and noisy sequences involving numerous input symbols distracting from the few important ones. The most difficult task (task 2c) involves hundreds of distractor symbols at random positions and minimal time lags of 1000 steps. LSTM solves it; BPTT and RTRL already fail in case of 10-step minimal time lags (see also Hochreiter, 1991; Mozer, 1992). For this reason RTRL and BPTT are omitted in the remaining, more complex experiments, all of which involve much longer time lags.

- Experiment 3 addresses long-time-lag problems with noise and signal on the same input line. Experiments 3a and 3b focus on Bengio et al.'s 1994 two-sequence problem. Because this problem can be solved quickly by random weight guessing, we also include a far more difficult two-sequence problem (experiment 3c), which requires learning real-valued, conditional expectations of noisy targets, given the inputs.
- Experiments 4 and 5 involve distributed, continuous-valued input representations and require learning to store precise, real values for very long time periods. Relevant input signals can occur at quite different positions in input sequences. Again minimal time lags involve hundreds of steps. Similar tasks never have been solved by other recurrent net algorithms.
- Experiment 6 involves tasks of a different complex type that also has not been solved by other recurrent net algorithms. Again, relevant input signals can occur at quite different positions in input sequences. The experiment shows that LSTM can extract information conveyed by the temporal order of widely separated inputs.

Section 5.7 provides a detailed summary of experimental conditions in two tables for reference.

5.1 Experiment 1: Embedded Reber Grammar.

5.1.1 Task. Our first task is to learn the embedded Reber grammar (Smith & Zipser, 1989; Cleeremans, Servan-Schreiber, & McClelland, 1989; Fahlman,1991). Since it allows for training sequences with short time lags (of as few as nine steps), it is not a long-time-lag problem. We include it for two reasons: (1) it is a popular recurrent net benchmark used by many authors, and we wanted to have at least one experiment where RTRL and BPTT do not fail completely, and (2) it shows nicely how output gates can be beneficial.

Starting at the left-most node of the directed graph in Figure 3, symbol strings are generated sequentially (beginning with the empty string) by following edges—and appending the associated symbols to the current string—until the right-most node is reached (the Reber grammar substrings are analogously generated from Figure 4). Edges are chosen randomly if there is a choice (probability: 0.5). The net's task is to read strings, one symbol at a time, and to predict the next symbol (error signals occur at every time step). To predict the symbol before last, the net has to remember the second symbol.

5.1.2 Comparison. We compare LSTM to Elman nets trained by Elman's training procedure (ELM) (results taken from Cleeremans et al., 1989), Fahlman's recurrent cascade-correlation (RCC) (results taken from Fahlman,

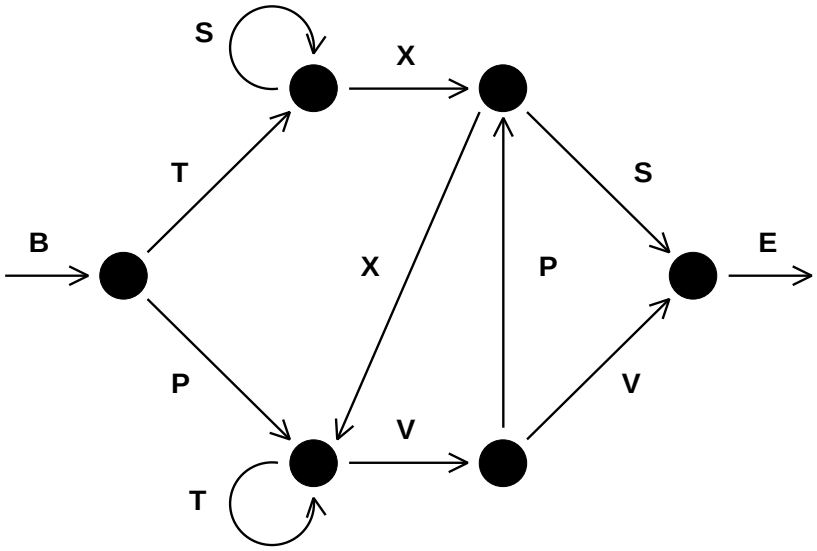


Figure 3: Transition diagram for the Reber grammar.

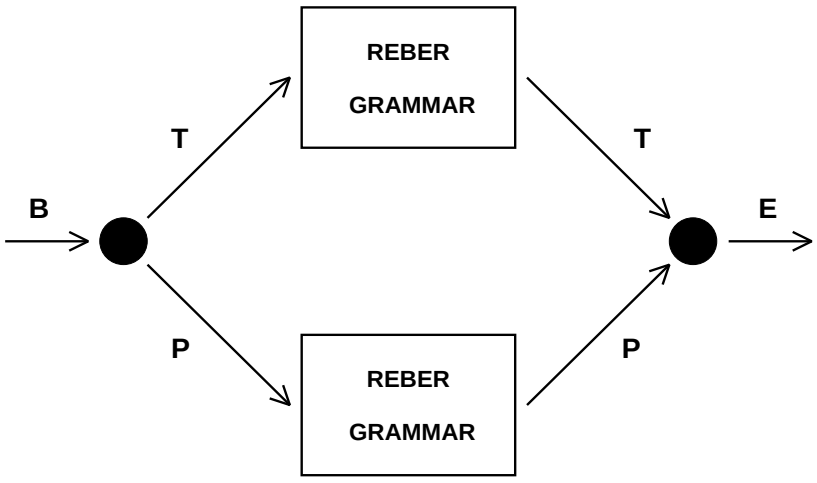


Figure 4: Transition diagram for the embedded Reber grammar. Each box represents a copy of the Reber grammar (see Figure 3).

1991), and RTRL (results taken from Smith & Zipser, 1989), where only the few successful trials are listed). Smith and Zipser actually make the task easier by increasing the probability of short-time-lag exemplars. We did not do this for LSTM.

5.1.3 Training/Testing. We use a local input-output representation (seven input units, seven output units). Following Fahlman, we use 256 training strings and 256 separate test strings. The training set is generated randomly; training exemplars are picked randomly from the training set. Test sequences are generated randomly, too, but sequences already used in the training set are not used for testing. After string presentation, all activations are reinitialized with zeros. A trial is considered successful if all string symbols of all sequences in both test set and training set are predicted correctly—that is, if the output unit(s) corresponding to the possible next symbol(s) is(are) always the most active ones.

5.1.4 Architectures. Architectures for RTRL, ELM, and RCC are reported in the references listed above. For LSTM, we use three (four) memory cell blocks. Each block has two (one) memory cells. The output layer's only incoming connections originate at memory cells. Each memory cell and each gate unit receives incoming connections from all memory cells and gate units (the hidden layer is fully connected; less connectivity may work as well). The input layer has forward connections to all units in the hidden layer. The gate units are biased. These architecture parameters make it easy to store at least three input signals (architectures 3-2 and 4-1 are employed to obtain comparable numbers of weights for both architectures: 264 for 4-1 and 276 for 3-2). Other parameters may be appropriate as well, however. All sigmoid functions are logistic with output range $[0, 1]$, except for h, whose range is $[-1, 1]$, and g, whose range is $[-2, 2]$. All weights are initialized in $[-0.2, 0.2]$, except for the output gate biases, which are initialized to -1, -2, and -3, respectively (see abuse problem, solution 2 of section 4). We tried learning rates of 0.1, 0.2, and 0.5.

5.1.5 Results. We use three different, randomly generated pairs of training and test sets. With each such pair we run 10 trials with different initial weights. See Table 1 for results (mean of 30 trials). Unlike the other methods, LSTM always learns to solve the task. Even when we ignore the unsuccessful trials of the other approaches, LSTM learns much faster.

5.1.6 Importance of Output Gates. The experiment provides a nice example where the output gate is truly beneficial. Learning to store the first T or P should not perturb activations representing the more easily learnable transitions of the original Reber grammar. This is the job of the output gates. Without output gates, we did not achieve fast learning.

Table 1: Experiment 1: Embedded Reber Grammar.

Method	Hidden Units	Number of Weights	Learning Rate	% of Success	After
RTRL	3	≈ 170	0.05	Some fraction	173,000
RTRL	12	≈ 494	0.1	Some fraction	25,000
ELM	15	≈ 435		0	>200,000
RCC	7–9	≈ 119–198		50	182,000
LSTM	4 blocks, size 1	264	0.1	100	39,740
LSTM	3 blocks, size 2	276	0.1	100	21,730
LSTM	3 blocks, size 2	276	0.2	97	14,060
LSTM	4 blocks, size 1	264	0.5	97	9500
LSTM	3 blocks, size 2	276	0.5	100	8440

Notes: Percentage of successful trials and number of sequence presentations until success for RTRL (results taken from Smith & Zipser, 1989), Elman net trained by Elman's procedure (results taken from Cleeremans et al., 1989), recurrent cascade-correlation (results taken from Fahlman, 1991), and our new approach (LSTM). Weight numbers in the first four rows are estimates, the corresponding papers do not provide all the technical details. Only LSTM almost always learns to solve the task (only 2 failures out of 150 trials). Even when we ignore the unsuccessful trials of the other approaches, LSTM learns much faster (the number of required training examples in the bottom row varies between 3800 and 24,100).

5.2 Experiment 2: Noise-Free and Noisy Sequences.

5.2.1 Task 2a: Noise-Free Sequences with Long Time Lags. There are $p + 1$ possible input symbols denoted $a_1, \ldots, a_{p-1}, a_p = x, a_{p+1} = y$. a_i is locally represented by the $p + 1$-dimensional vector whose ith component is 1 (all other components are 0). A net with $p + 1$ input units and $p + 1$ output units sequentially observes input symbol sequences, one at a time, permanently trying to predict the next symbol; error signals occur at every time step. To emphasize the long-time-lag problem, we use a training set consisting of only two very similar sequences: $(y, a_1, a_2, \ldots, a_{p-1}, y)$ and $(x, a_1, a_2, \ldots, a_{p-1}, x)$. Each is selected with probability 0.5. To predict the final element, the net has to learn to store a representation of the first element for p time steps.

We compare real-time recurrent learning for fully recurrent nets (RTRL), back-propagation through time (BPTT), the sometimes very successful two-net neural sequence chunker (CH; Schmidhuber, 1992b), and our new method (LSTM). In all cases, weights are initialized in $[-0.2, 0.2]$. Due to limited computation time, training is stopped after 5 million sequence presentations. A successful run is one that fulfills the following criterion: after training, during 10,000 successive, randomly chosen input sequences, the maximal absolute error of all output units is always below 0.25.

Table 2: Task 2a: Percentage of Successful Trials and Number of Training Sequences until Success.

Method	Delay p	Learning Rate	Number of Weights	% Successful Trials	Success After
RTRL	4	1.0	36	78	1,043,000
RTRL	4	4.0	36	56	892,000
RTRL	4	10.0	36	22	254,000
RTRL	10	1.0–10.0	144	0	$> 5{,}000{,}000$
RTRL	100	1.0–10.0	10404	0	$> 5{,}000{,}000$
BPTT	100	1.0–10.0	10404	0	$> 5{,}000{,}000$
CH	100	1.0	10506	33	32,400
LSTM	100	1.0	10504	100	5,040

Notes: Table entries refer to means of 18 trials. With 100 time-step delays, only CH and LSTM achieve successful trials. Even when we ignore the unsuccessful trials of the other approaches, LSTM learns much faster.

Architectures.

RTRL: One self-recurrent hidden unit, $p + 1$ nonrecurrent output units. Each layer has connections from all layers below. All units use the logistic activation function sigmoid in $[0, 1]$.

BPTT: Same architecture as the one trained by RTRL.

CH: Both net architectures like RTRL's, but one has an additional output for predicting the hidden unit of the other one (see Schmidhuber, 1992b, for details).

LSTM: As with RTRL, but the hidden unit is replaced by a memory cell and an input gate (no output gate required). g is the logistic sigmoid, and h is the identity function $h : h(x) = x, \forall x$. Memory cell and input gate are added once the error has stopped decreasing (see abuse problem: solution 1 in section 4).

Results. Using RTRL and a short four-time-step delay ($p = 4$), 7/9 of all trials were successful. No trial was successful with $p = 10$. With long time lags, only the neural sequence chunker and LSTM achieved successful trials; BPTT and RTRL failed. With $p = 100$, the two-net sequence chunker solved the task in only one-third of all trials. LSTM, however, always learned to solve the task. Comparing successful trials only, LSTM learned much faster. See Table 2 for details. It should be mentioned, however, that a hierarchical chunker can also always quickly solve this task (Schmidhuber, 1992c, 1993).

5.2.2 Task 2b: No Local Regularities. With task 2a, the chunker sometimes learns to predict the final element correctly, but only because of pre-

dictable local regularities in the input stream that allow for compressing the sequence. In a more difficult task, involving many more different possible sequences, we remove compressibility by replacing the deterministic subsequence $(a_1, a_2, \ldots, a_{p-1})$ by a random subsequence (of length $p-1$) over the alphabet $a_1, a_2, \ldots, a_{p-1}$. We obtain two classes (two sets of sequences) $\{(y, a_{i_1}, a_{i_2}, \ldots, a_{i_{p-1}}, y) \mid 1 \leq i_1, i_2, \ldots, i_{p-1} \leq p-1\}$ and $\{(x, a_{i_1}, a_{i_2}, \ldots, a_{i_{p-1}}, x) \mid 1 \leq i_1, i_2, \ldots, i_{p-1} \leq p-1\}$. Again, every next sequence element has to be predicted. The only totally predictable targets, however, are x and y, which occur at sequence ends. Training exemplars are chosen randomly from the two classes. Architectures and parameters are the same as in experiment 2a. A successful run is one that fulfills the following criterion: after training, during 10,000 successive, randomly chosen input sequences, the maximal absolute error of all output units is below 0.25 at sequence end.

Results. As expected, the chunker failed to solve this task (so did BPTT and RTRL, of course). LSTM, however, was always successful. On average (mean of 18 trials), success for $p = 100$ was achieved after 5680 sequence presentations. This demonstrates that LSTM does not require sequence regularities to work well.

5.2.3 Task 2c: Very Long Time Lags—No Local Regularities. This is the most difficult task in this subsection. To our knowledge, no other recurrent net algorithm can solve it. Now there are $p+4$ possible input symbols denoted $a_1, \ldots, a_{p-1}, a_p, a_{p+1} = e, a_{p+2} = b, a_{p+3} = x, a_{p+4} = y$. $a_1, \ldots, a_p$ are also called distractor symbols. Again, a_i is locally represented by the $p+4$-dimensional vector whose ith component is 1 (all other components are 0). A net with $p+4$ input units and 2 output units sequentially observes input symbol sequences, one at a time. Training sequences are randomly chosen from the union of two very similar subsets of sequences: $\{(b, y, a_{i_1}, a_{i_2}, \ldots, a_{i_{q+k}}, e, y) \mid 1 \leq i_1, i_2, \ldots, i_{q+k} \leq q\}$ and $\{(b, x, a_{i_1}, a_{i_2}, \ldots, a_{i_{q+k}}, e, x) \mid 1 \leq i_1, i_2, \ldots, i_{q+k} \leq q\}$. To produce a training sequence, we randomly generate a sequence prefix of length $q+2$, randomly generate a sequence suffix of additional elements ($\neq b, e, x, y$) with probability 9/10 or, alternatively, an e with probability 1/10. In the latter case, we conclude the sequence with x or y, depending on the second element. For a given k, this leads to a uniform distribution on the possible sequences with length $q+k+4$. The minimal sequence length is $q+4$; the expected length is

$$4 + \sum_{k=0}^{\infty} \frac{1}{10} \left(\frac{9}{10}\right)^k (q+k) = q + 14.$$

The expected number of occurrences of element a_i, $1 \leq i \leq p$, in a sequence is $(q+10)/p \approx \frac{q}{p}$. The goal is to predict the last symbol, which always occurs after the "trigger symbol" e. Error signals are generated only at sequence

Table 3: Task 2c: LSTM with Very Long Minimal Time Lags $q + 1$ and a Lot of Noise.

q (Time Lag -1)	p (Number of Random Inputs)	$\frac{q}{p}$	Number of Weights	Success After
50	50	1	364	30,000
100	100	1	664	31,000
200	200	1	1264	33,000
500	500	1	3064	38,000
1000	1,000	1	6064	49,000
1000	500	2	3064	49,000
1000	200	5	1264	75,000
1000	100	10	664	135,000
1000	50	20	364	203,000

Notes: p is the number of available distractor symbols ($p+4$ is the number of input units). q/p is the expected number of occurrences of a given distractor symbol in a sequence. The right-most column lists the number of training sequences required by LSTM (BPTT, RTRL, and the other competitors have no chance of solving this task). If we let the number of distractor symbols (and weights) increase in proportion to the time lag, learning time increases very slowly. The lower block illustrates the expected slowdown due to increased frequency of distractor symbols.

ends. To predict the final element, the net has to learn to store a representation of the second element for at least $q + 1$ time steps (until it sees the trigger symbol e). Success is defined as prediction error (for final sequence element) of both output units always below 0.2, for 10,000 successive, randomly chosen input sequences.

Architecture/Learning. The net has $p + 4$ input units and 2 output units. Weights are initialized in $[-0.2, 0.2]$. To avoid too much learning time variance due to different weight initializations, the hidden layer gets two memory cells (two cell blocks of size 1, although one would be sufficient). There are no other hidden units. The output layer receives connections only from memory cells. Memory cells and gate units receive connections from input units, memory cells, and gate units (the hidden layer is fully connected). No bias weights are used. h and g are logistic sigmoids with output ranges $[-1, 1]$ and $[-2, 2]$, respectively. The learning rate is 0.01. Note that the minimal time lag is $q+1$; the net never sees short training sequences facilitating the classification of long test sequences.

Results. Twenty trials were made for all tested pairs (p, q). Table 3 lists the mean of the number of training sequences required by LSTM to achieve success (BPTT and RTRL have no chance of solving nontrivial tasks with minimal time lags of 1000 steps).

Scaling. Table 3 shows that if we let the number of input symbols (and weights) increase in proportion to the time lag, learning time increases very slowly. This is another remarkable property of LSTM not shared by any other method we are aware of. Indeed, RTRL and BPTT are far from scaling reasonably; instead, they appear to scale exponentially and appear quite useless when the time lags exceed as few as 10 steps.

Distractor Influence. In Table 3, the column headed by q/p gives the expected frequency of distractor symbols. Increasing this frequency decreases learning speed, an effect due to weight oscillations caused by frequently observed input symbols.

5.3 Experiment 3: Noise and Signal on Same Channel. This experiment serves to illustrate that LSTM does not encounter fundamental problems if noise and signal are mixed on the same input line. We initially focus on Bengio et al.'s simple 1994 two-sequence problem. In experiment 3c we pose a more challenging two-sequence problem.

5.3.1 Task 3a (Two-Sequence Problem). The task is to observe and then classify input sequences. There are two classes, each occurring with probability 0.5. There is only one input line. Only the first N real-valued sequence elements convey relevant information about the class. Sequence elements at positions $t > N$ are generated by a gaussian with mean zero and variance 0.2. Case $N = 1$: the first sequence element is 1.0 for class 1, and -1.0 for class 2. Case $N = 3$: the first three elements are 1.0 for class 1 and -1.0 for class 2. The target at the sequence end is 1.0 for class 1 and 0.0 for class 2. Correct classification is defined as absolute output error at sequence end below 0.2. Given a constant T, the sequence length is randomly selected between T and $T + T/10$ (a difference to Bengio et al.'s problem is that they also permit shorter sequences of length $T/2$).

Guessing. Bengio et al. (1994) and Bengio and Frasconi (1994) tested seven different methods on the two-sequence problem. We discovered, however, that random weight guessing easily outperforms them all because the problem is so simple.[5] See Schmidhuber and Hochreiter (1996) and Hochreiter and Schmidhuber (1996, 1997) for additional results in this vein.

LSTM Architecture. We use a three-layer net with one input unit, one output unit, and three cell blocks of size 1. The output layer receives connections only from memory cells. Memory cells and gate units receive inputs from input units, memory cells, and gate units and have bias weights. Gate

[5] However, different input representations and different types of noise may lead to worse guessing performance (Yoshua Bengio, personal communication, 1996).

Table 4: Task 3a: Bengio et al.'s Two-Sequence Problem.

T	N	Stop: ST1	Stop: ST2	Number of Weights	ST2: Fraction Misclassified
100	3	27,380	39,850	102	0.000195
100	1	58,370	64,330	102	0.000117
1000	3	446,850	452,460	102	0.000078

Notes: *T* is minimal sequence length. *N* is the number of information-conveying elements at sequence begin. The column headed by ST1 (ST2) gives the number of sequence presentations required to achieve stopping criterion ST1 (ST2). The right-most column lists the fraction of misclassified posttraining sequences (with absolute error > 0.2) from a test set consisting of 2560 sequences (tested after ST2 was achieved). All values are means of 10 trials. We discovered, however, that this problem is so simple that random weight guessing solves it faster than LSTM and any other method for which there are published results.

units and output unit are logistic sigmoid in $[0, 1]$, h in $[-1, 1]$, and g in $[-2, 2]$.

Training/Testing. All weights (except the bias weights to gate units) are randomly initialized in the range $[-0.1, 0.1]$. The first input gate bias is initialized with -1.0, the second with -3.0, and the third with -5.0. The first output gate bias is initialized with -2.0, the second with -4.0, and the third with -6.0. The precise initialization values hardly matter though, as confirmed by additional experiments. The learning rate is 1.0. All activations are reset to zero at the beginning of a new sequence.

We stop training (and judge the task as being solved) according to the following criteria: ST1: none of 256 sequences from a randomly chosen test set is misclassified; ST2: ST1 is satisfied, and mean absolute test set error is below 0.01. In case of ST2, an additional test set consisting of 2560 randomly chosen sequences is used to determine the fraction of misclassified sequences.

Results. See Table 4. The results are means of 10 trials with different weight initializations in the range $[-0.1, 0.1]$. LSTM is able to solve this problem, though by far not as fast as random weight guessing (see "Guessing" above). Clearly this trivial problem does not provide a very good testbed to compare performance of various nontrivial algorithms. Still, it demonstrates that LSTM does not encounter fundamental problems when faced with signal and noise on the same channel.

5.3.2 Task 3b. The architecture, parameters, and other elements are as in task 3a, but now with gaussian noise (mean 0 and variance 0.2) added to the

Table 5: Task 3b: Modified Two-Sequence Problem.

T	N	Stop: ST1	Stop: ST2	Number of Weights	ST2: Fraction Misclassified
100	3	41,740	43,250	102	0.00828
100	1	74,950	78,430	102	0.01500
1000	1	481,060	485,080	102	0.01207

Note: Same as in Table 4, but now the information-conveying elements are also perturbed by noise.

information-conveying elements ($t <= N$). We stop training (and judge the task as being solved) according to the following, slightly redefined criteria: ST1: fewer than 6 out of 256 sequences from a randomly chosen test set are misclassified; ST2: ST1 is satisfied, and mean absolute test set error is below 0.04. In case of ST2, an additional test set consisting of 2560 randomly chosen sequences is used to determine the fraction of misclassified sequences.

Results. See Table 5. The results represent means of 10 trials with different weight initializations. LSTM easily solves the problem.

5.3.3 Task 3c. The architecture, parameters, and other elements are as in task 3a, but with a few essential changes that make the task nontrivial: the targets are 0.2 and 0.8 for class 1 and class 2, respectively, and there is gaussian noise on the targets (mean 0 and variance 0.1; S.D. 0.32). To minimize mean squared error, the system has to learn the conditional expectations of the targets given the inputs. Misclassification is defined as absolute difference between output and noise-free target (0.2 for class 1 and 0.8 for class 2) > 0.1. The network output is considered acceptable if the mean absolute difference between noise-free target and output is below 0.015. Since this requires high weight precision, task 3c (unlike tasks 3a and 3b) cannot be solved quickly by random guessing.

Training/Testing. The learning rate is 0.1. We stop training according to the following criterion: none of 256 sequences from a randomly chosen test set is misclassified, and mean absolute difference between the noise-free target and output is below 0.015. An additional test set consisting of 2560 randomly chosen sequences is used to determine the fraction of misclassified sequences.

Results. See Table 6. The results represent means of 10 trials with different weight initializations. Despite the noisy targets, LSTM still can solve the problem by learning the expected target values.

Table 6: Task 3c: Modified, More Challenging Two-Sequence Problem.

T	N	Stop	Number of Weights	Fraction Misclassified	Average Difference to Mean
100	3	269,650	102	0.00558	0.014
100	1	565,640	102	0.00441	0.012

Notes: Same as in Table 4, but with noisy real-valued targets. The system has to learn the conditional expectations of the targets given the inputs. The right-most column provides the average difference between network output and expected target. Unlike tasks 3a and 3b, this one cannot be solved quickly by random weight guessing.

5.4 Experiment 4: Adding Problem. The difficult task in this section is of a type that has never been solved by other recurrent net algorithms. It shows that LSTM can solve long-time-lag problems involving distributed, continuous-valued representations.

5.4.1 Task. Each element of each input sequence is a pair of components. The first component is a real value randomly chosen from the interval $[-1, 1]$; the second is 1.0, 0.0, or -1.0 and is used as a marker. At the end of each sequence, the task is to output the sum of the first components of those pairs that are marked by second components equal to 1.0. Sequences have random lengths between the minimal sequence length T and $T + T/10$. In a given sequence, exactly two pairs are marked, as follows: we first randomly select and mark one of the first 10 pairs (whose first component we call X_1). Then we randomly select and mark one of the first $T/2 - 1$ still unmarked pairs (whose first component we call X_2). The second components of all remaining pairs are zero except for the first and final pair, whose second components are -1. (In the rare case where the first pair of the sequence gets marked, we set X_1 to zero.) An error signal is generated only at the sequence end: the target is $0.5 + (X_1 + X_2)/4.0$ (the sum $X_1 + X_2$ scaled to the interval $[0, 1]$). A sequence is processed correctly if the absolute error at the sequence end is below 0.04.

5.4.2 Architecture. We use a three-layer net with two input units, one output unit, and two cell blocks of size 2. The output layer receives connections only from memory cells. Memory cells and gate units receive inputs from memory cells and gate units (the hidden layer is fully connected; less connectivity may work as well). The input layer has forward connections to all units in the hidden layer. All noninput units have bias weights. These architecture parameters make it easy to store at least two input signals (a cell block size of 1 works well, too). All activation functions are logistic with output range $[0, 1]$, except for h, whose range is $[-1, 1]$, and g, whose range is $[-2, 2]$.

Table 7: Experiment 4: Results for the Adding Problem.

T	Minimal Lag	Number of Weights	Number of Wrong Predictions	Success After
100	50	93	1 out of 2560	74,000
500	250	93	0 out of 2560	209,000
1000	500	93	1 out of 2560	853,000

Notes: T is the minimal sequence length, $T/2$ the minimal time lag. "Number of Wrong Predictions" is the number of incorrectly processed sequences (error > 0.04) from a test set containing 2560 sequences. The right-most column gives the number of training sequences required to achieve the stopping criterion. All values are means of 10 trials. For $T = 1000$ the number of required training examples varies between 370,000 and 2,020,000, exceeding 700,000 in only three cases.

5.4.3 State Drift Versus Initial Bias. Note that the task requires storing the precise values of real numbers for long durations; the system must learn to protect memory cell contents against even minor internal state drift (see section 4). To study the significance of the drift problem, we make the task even more difficult by biasing all noninput units, thus artificially inducing internal state drift. All weights (including the bias weights) are randomly initialized in the range $[-0.1, 0.1]$. Following section 4's remedy for state drifts, the first input gate bias is initialized with -3.0 and the second with -6.0 (though the precise values hardly matter, as confirmed by additional experiments).

5.4.4 Training/Testing. The learning rate is 0.5. Training is stopped once the average training error is below 0.01, and the 2000 most recent sequences were processed correctly.

5.4.5 Results. With a test set consisting of 2560 randomly chosen sequences, the average test set error was always below 0.01, and there were never more than three incorrectly processed sequences. Table 7 shows details.

The experiment demonstrates that LSTM is able to work well with distributed representations, LSTM is able to learn to perform calculations involving continuous values, and since the system manages to store continuous values without deterioration for minimal delays of $T/2$ time steps, there is no significant, harmful internal state drift.

5.5 Experiment 5: Multiplication Problem. One may argue that LSTM is a bit biased toward tasks such as the adding problem from the previous subsection. Solutions to the adding problem may exploit the CEC's built-in integration capabilities. Although this CEC property may be viewed as a

Table 8: Experiment 5: Results for the Multiplication Problem.

T	Minimal Lag	Number of Weights	n_{seq}	Number of Wrong Predictions	MSE	Success After
100	50	93	140	139 out of 2560	0.0223	482,000
100	50	93	13	14 out of 2560	0.0139	1,273,000

Notes: T is the minimal sequence length and $T/2$ the minimal time lag. We test on a test set containing 2560 sequences as soon as less than n_{seq} of the 2000 most recent training sequences lead to error > 0.04. "Number of Wrong Predictions" is the number of test sequences with error > 0.04. MSE is the mean squared error on the test set. The right-most column lists numbers of training sequences required to achieve the stopping criterion. All values are means of 10 trials.

feature rather than a disadvantage (integration seems to be a natural subtask of many tasks occurring in the real world), the question arises whether LSTM can also solve tasks with inherently nonintegrative solutions. To test this, we change the problem by requiring the final target to equal the product (instead of the sum) of earlier marked inputs.

5.5.1 Task. This is like the task in section 5.4, except that the first component of each pair is a real value randomly chosen from the interval $[0, 1]$. In the rare case where the first pair of the input sequence gets marked, we set X_1 to 1.0. The target at sequence end is the product $X_1 \times X_2$.

5.5.2 Architecture. This is as in section 5.4. All weights (including the bias weights) are randomly initialized in the range $[-0.1, 0.1]$.

5.5.3 Training/Testing. The learning rate is 0.1. We test performance twice: as soon as less than n_{seq} of the 2000 most recent training sequences lead to absolute errors exceeding 0.04, where $n_{seq} = 140$ and $n_{seq} = 13$. Why these values? $n_{seq} = 140$ is sufficient to learn storage of the relevant inputs. It is not enough though to fine-tune the precise final outputs. $n_{seq} = 13$, however, leads to quite satisfactory results.

5.5.4 Results. For $n_{seq} = 140$ ($n_{seq} = 13$) with a test set consisting of 2560 randomly chosen sequences, the average test set error was always below 0.026 (0.013), and there were never more than 170 (15) incorrectly processed sequences. Table 8 shows details. (A net with additional standard hidden units or with a hidden layer above the memory cells may learn the fine-tuning part more quickly.)

The experiment demonstrates that LSTM can solve tasks involving both continuous-valued representations and nonintegrative information processing.

5.6 Experiment 6: Temporal Order. In this subsection, LSTM solves other difficult (but artificial) tasks that have never been solved by previous recurrent net algorithms. The experiment shows that LSTM is able to extract information conveyed by the temporal order of widely separated inputs.

5.6.1 Task 6a: Two Relevant, Widely Separated Symbols. The goal is to classify sequences. Elements and targets are represented locally (input vectors with only one nonzero bit). The sequence starts with an E, ends with a B (the "trigger symbol"), and otherwise consists of randomly chosen symbols from the set $\{a, b, c, d\}$ except for two elements at positions t_1 and t_2 that are either X or Y. The sequence length is randomly chosen between 100 and 110, t_1 is randomly chosen between 10 and 20, and t_2 is randomly chosen between 50 and 60. There are four sequence classes Q, R, S, U, which depend on the temporal order of X and Y. The rules are: $X, X \rightarrow Q$; $X, Y \rightarrow R$; $Y, X \rightarrow S$; $Y, Y \rightarrow U$.

5.6.2 Task 6b: Three Relevant, Widely Separated Symbols. Again, the goal is to classify sequences. Elements and targets are represented locally. The sequence starts with an E, ends with a B (the trigger symbol), and otherwise consists of randomly chosen symbols from the set $\{a, b, c, d\}$ except for three elements at positions t_1, t_2, and t_3 that are either X or Y. The sequence length is randomly chosen between 100 and 110, t_1 is randomly chosen between 10 and 20, t_2 is randomly chosen between 33 and 43, and t_3 is randomly chosen between 66 and 76. There are eight sequence classes—Q, R, S, U, V, A, B, C—which depend on the temporal order of the Xs and Ys. The rules are: $X, X, X \rightarrow Q$; $X, X, Y \rightarrow R$; $X, Y, X \rightarrow S$; $X, Y, Y \rightarrow U$; $Y, X, X \rightarrow V$; $Y, X, Y \rightarrow A$; $Y, Y, X \rightarrow B$; $Y, Y, Y \rightarrow C$.

There are as many output units as there are classes. Each class is locally represented by a binary target vector with one nonzero component. With both tasks, error signals occur only at the end of a sequence. The sequence is classified correctly if the final absolute error of all output units is below 0.3.

Architecture. We use a three-layer net with eight input units, two (three) cell blocks of size 2, and four (eight) output units for task 6a (6b). Again all noninput units have bias weights, and the output layer receives connections from memory cells, only. Memory cells and gate units receive inputs from input units, memory cells, and gate units (the hidden layer is fully connected; less connectivity may work as well). The architecture parameters for task 6a (6b) make it easy to store at least two (three) input signals. All activation functions are logistic with output range $[0, 1]$, except for h, whose range is $[-1, 1]$, and g, whose range is $[-2, 2]$.

Table 9: Experiment 6: Results for the Temporal Order Problem.

Task	Number of Weights	Number of Wrong Predictions	Success After
Task 6a	156	1 out of 2560	31,390
Task 6b	308	2 out of 2560	571,100

Notes: "Number of Wrong Predictions" is the number of incorrectly classified sequences (error > 0.3 for at least one output unit) from a test set containing 2560 sequences. The right-most column gives the number of training sequences required to achieve the stopping criterion. The results for task 6a are means of 20 trials; those for task 6b of 10 trials.

Training/Testing. The learning rate is 0.5 (0.1) for experiment 6a (6b). Training is stopped once the average training error falls below 0.1 and the 2000 most recent sequences were classified correctly. All weights are initialized in the range $[-0.1, 0.1]$. The first input gate bias is initialized with -2.0, the second with -4.0, and (for experiment 6b) the third with -6.0 (again, we confirmed by additional experiments that the precise values hardly matter).

Results. With a test set consisting of 2560 randomly chosen sequences, the average test set error was always below 0.1, and there were never more than three incorrectly classified sequences. Table 9 shows details.

The experiment shows that LSTM is able to extract information conveyed by the temporal order of widely separated inputs. In task 6a, for instance, the delays between the first and second relevant input and between the second relevant input and sequence end are at least 30 time steps.

Typical Solutions. In experiment 6a, how does LSTM distinguish between temporal orders (X, Y) and (Y, X)? One of many possible solutions is to store the first X or Y in cell block 1 and the second X/Y in cell block 2. Before the first X/Y occurs, block 1 can see that it is still empty by means of its recurrent connections. After the first X/Y, block 1 can close its input gate. Once block 1 is filled and closed, this fact will become visible to block 2 (recall that all gate units and all memory cells receive connections from all nonoutput units).

Typical solutions, however, require only one memory cell block. The block stores the first X or Y; once the second X/Y occurs, it changes its state depending on the first stored symbol. Solution type 1 exploits the connection between memory cell output and input gate unit. The following events cause different input gate activations: X occurs in conjunction with a filled block; X occurs in conjunction with an empty block. Solution type 2 is based on a strong, positive connection between memory cell output and memory cell input. The previous occurrence of X (Y) is represented by a

Table 10: Summary of Experimental Conditions for LSTM, Part I.

(1) Task	(2) p	(3) lag	(4) b	(5) s	(6) in	(7) out	(8) w	(9) c	(10) ogb	(11) igb	(12) bias	(13) h	(14) g	(15) α
1-1	9	9	4	1	7	7	264	F	$-1, -2, -3, -4$	r	ga	h1	g2	0.1
1-2	9	9	3	2	7	7	276	F	$-1, -2, -3$	r	ga	h1	g2	0.1
1-3	9	9	3	2	7	7	276	F	$-1, -2, -3$	r	ga	h1	g2	0.2
1-4	9	9	4	1	7	7	264	F	$-1, -2, -3, -4$	r	ga	h1	g2	0.5
1-5	9	9	3	2	7	7	276	F	$-1, -2, -3$	r	ga	h1	g2	0.5
2a	100	100	1	1	101	101	10,504	B	No og	None	None	id	g1	1.0
2b	100	100	1	1	101	101	10,504	B	No og	None	None	id	g1	1.0
2c-1	50	50	2	1	54	2	364	F	None	None	None	h1	g2	0.01
2c-2	100	100	2	1	104	2	664	F	None	None	None	h1	g2	0.01
2c-3	200	200	2	1	204	2	1264	F	None	None	None	h1	g2	0.01
2c-4	500	500	2	1	504	2	3064	F	None	None	None	h1	g2	0.01
2c-5	1000	1000	2	1	1004	2	6064	F	None	None	None	h1	g2	0.01
2c-6	1000	1000	2	1	504	2	3064	F	None	None	None	h1	g2	0.01
2c-7	1000	1000	2	1	204	2	1264	F	None	None	None	h1	g2	0.01
2c-8	1000	1000	2	1	104	2	664	F	None	None	None	h1	g2	0.01
2c-9	1000	1000	2	1	54	2	364	F	None	None	None	h1	g2	0.01
3a	100	100	3	1	1	1	102	F	$-2, -4, -6$	$-1, -3, -5$	b1	h1	g2	1.0
3b	100	100	3	1	1	1	102	F	$-2, -4, -6$	$-1, -3, -5$	b1	h1	g2	1.0
3c	100	100	3	1	1	1	102	F	$-2, -4, -6$	$-1, -3, -5$	b1	h1	g2	0.1
4-1	100	50	2	2	2	1	93	F	r	$-3, -6$	All	h1	g2	0.5
4-2	500	250	2	2	2	1	93	F	r	$-3, -6$	All	h1	g2	0.5
4-3	1000	500	2	2	2	1	93	F	r	$-3, -6$	All	h1	g2	0.5
5	100	50	2	2	2	1	93	F	r	r	All	h1	g2	0.1
6a	100	40	2	2	8	4	156	F	r	$-2, -4$	All	h1	g2	0.5
6b	100	24	3	2	8	8	308	F	r	$-2, -4, -6$	All	h1	g2	0.1

Notes: Col. 1: task number. Col. 2: minimal sequence length *p*. Col. 3: minimal number of steps between most recent relevant input information and teacher signal. Col. 4: number of cell blocks *b*. Col. 5: block size *s*. Col. 6: Number of input units *in*. Col. 7: Number of output units *out*. Col. 8: number of weights *w*. Col. 9: *c* describes connectivity: *F* means "output layer receives connections from memory cells; memory cells and gate units receive connections from input units, memory cells and gate units"; *B* means "each layer receives connections from all layers below." Col. 10: Initial output gate bias *ogb*, where *r* stands for "randomly chosen from the interval $[-0.1, 0.1]$" and *no og* means "no output gate used." Col. 11: initial input gate bias *igb* (see Col. 10). Col. 12: which units have bias weights? *b1* stands for "all hidden units", *ga* for "only gate units," and *all* for "all noninput units." Col. 13: the function *h*, where *id* is identity function, *h1* is logistic sigmoid in $[-2, 2]$. Col. 14: the logistic function *g*, where *g*1 is sigmoid in $[0, 1]$, *g*2 in $[-1, 1]$. Col. 15: learning rate α.

positive (negative) internal state. Once the input gate opens for the second time, so does the output gate, and the memory cell output is fed back to its own input. This causes (X, Y) to be represented by a positive internal state, because X contributes to the new internal state twice (via current internal state and cell output feedback). Similarly, (Y, X) gets represented by a negative internal state.

5.7 Summary of Experimental Conditions. Tables 10 and 11 provide an overview of the most important LSTM parameters and architectural details for experiments 1 through 6. The conditions of the simple experiments 2a

Table 11: Summary of Experimental Conditions for LSTM, Part II.

(1) Task	(2) Select	(3) Interval	(4) Test Set Size	(5) Stopping Criterion	(6) Success
1	t1	$[-0.2, 0.2]$	256	Training and test correctly pred.	See text
2a	t1	$[-0.2, 0.2]$	no test set	After 5 million exemplars	ABS(0.25)
2b	t2	$[-0.2, 0.2]$	10,000	After 5 million exemplars	ABS(0.25)
2c	t2	$[-0.2, 0.2]$	10,000	After 5 million exemplars	ABS(0.2)
3a	t3	$[-0.1, 0.1]$	2560	ST1 and ST2 (see text)	ABS(0.2)
3b	t3	$[-0.1, 0.1]$	2560	ST1 and ST2 (see text)	ABS(0.2)
3c	t3	$[-0.1, 0.1]$	2560	ST1 and ST2 (see text)	See text
4	t3	$[-0.1, 0.1]$	2560	ST3(0.01)	ABS(0.04)
5	t3	$[-0.1, 0.1]$	2560	see text	ABS(0.04)
6a	t3	$[-0.1, 0.1]$	2560	ST3(0.1)	ABS(0.3)
6b	t3	$[-0.1, 0.1]$	2560	ST3(0.1)	ABS(0.3)

Notes: Col. 1: task number. Col. 2: training exemplar selelction, where $t1$ stands for "randomly chosen form training set," $t2$ for "randomly chosen from two classes," and $t3$ for "randomly generated on line." Col. 3: weight initialization interval. Col. 4: test set size. Col. 5: Stopping criterion for training, where $ST3(\beta)$ stands for "average training error below β and the 2000 most recent sequences were processed correctly." Col. 6: success (correct classification) criterion, where $ABS(\beta)$ stands for "absolute error of all output units at sequence end is below β."

and 2b differ slightly from those of the other, more systematic experiments, due to historical reasons.

6 Discussion

6.1 Limitations of LSTM.

- The particularly efficient truncated backpropagation version of the LSTM algorithm will not easily solve problems similar to strongly delayed XOR problems, where the goal is to compute the XOR of two widely separated inputs that previously occurred somewhere in a noisy sequence. The reason is that storing only one of the inputs will not help to reduce the expected error; the task is nondecomposable in the sense that it is impossible to reduce the error incrementally by first solving an easier subgoal.

  In theory, this limitation can be circumvented by using the full gradient (perhaps with additional conventional hidden units receiving input from the memory cells). But we do not recommend computing the full gradient for the following reasons: (1) It increases computational complexity, (2) constant error flow through CECs can be shown only for truncated LSTM, and (3) we actually did conduct a few experiments with nontruncated LSTM. There was no significant difference to truncated LSTM, exactly because outside the CECs, error flow tends

to vanish quickly. For the same reason, full BPTT does not outperform truncated BPTT.

- Each memory cell block needs two additional units (input and output gate). In comparison to standard recurrent nets, however, this does not increase the number of weights by more than a factor of 9: each conventional hidden unit is replaced by at most three units in the LSTM architecture, increasing the number of weights by a factor of 3^2 in the fully connected case. Note, however, that our experiments use quite comparable weight numbers for the architectures of LSTM and competing approaches.
- Due to its constant error flow through CECs within memory cells, LSTM generally runs into problems similar to those of feedforward nets' seeing the entire input string at once. For instance, there are tasks that can be quickly solved by random weight guessing but not by the truncated LSTM algorithm with small weight initializations, such as the 500-step parity problem (see the introduction to section 5). Here, LSTM's problems are similar to the ones of a feedforward net with 500 inputs, trying to solve 500-bit parity. Indeed LSTM typically behaves much like a feedforward net trained by backpropagation that sees the entire input. But that is also precisely why it so clearly outperforms previous approaches on many nontrivial tasks with significant search spaces.
- LSTM does not have any problems with the notion of recency that go beyond those of other approaches. All gradient-based approaches, however, suffer from a practical inability to count discrete time steps precisely. If it makes a difference whether a certain signal occurred 99 or 100 steps ago, then an additional counting mechanism seems necessary. Easier tasks, however, such as one that requires making a difference only between, say, 3 and 11 steps, do not pose any problems to LSTM. For instance, by generating an appropriate negative connection between memory cell output and input, LSTM can give more weight to recent inputs and learn decays where necessary.

6.2 Advantages of LSTM.

- The constant error backpropagation within memory cells results in LSTM's ability to bridge very long time lags in case of problems similar to those discussed above.
- For long-time-lag problems such as those discussed in this article, LSTM can handle noise, distributed representations, and continuous values. In contrast to finite state automata or hidden Markov models, LSTM does not require an a priori choice of a finite number of states. In principle, it can deal with unlimited state numbers.

- For problems discussed in this article, LSTM generalizes well, even if the positions of widely separated, relevant inputs in the input sequence do not matter. Unlike previous approaches, ours quickly learns to distinguish between two or more widely separated occurrences of a particular element in an input sequence, without depending on appropriate short-time-lag training exemplars.
- There appears to be no need for parameter fine tuning. LSTM works well over a broad range of parameters such as learning rate, input gate bias, and output gate bias. For instance, to some readers the learning rates used in our experiments may seem large. However, a large learning rate pushes the output gates toward zero, thus automatically countermanding its own negative effects.
- The LSTM algorithm's update complexity per weight and time step is essentially that of BPTT, namely, $O(1)$. This is excellent in comparison to other approaches such as RTRL. Unlike full BPTT, however, LSTM is local in both space and time.

7 Conclusion

Each memory cell's internal architecture guarantees constant error flow within its CEC, provided that truncated backpropagation cuts off error flow trying to leak out of memory cells. This represents the basis for bridging very long time lags. Two gate units learn to open and close access to error flow within each memory cell's CEC. The multiplicative input gate affords protection of the CEC from perturbation by irrelevant inputs. Similarly, the multiplicative output gate protects other units from perturbation by currently irrelevant memory contents.

To find out about LSTM's practical limitations we intend to apply it to real-world data. Application areas will include time-series prediction, music composition, and speech processing. It will also be interesting to augment sequence chunkers (Schmidhuber, 1992b, 1993) by LSTM to combine the advantages of both.

Appendix

A.1 Algorithm Details. In what follows, the index k ranges over output units, i ranges over hidden units, c_j stands for the jth memory cell block, c_j^v denotes the vth unit of memory cell block c_j, u, l, m stand for arbitrary units, and t ranges over all time steps of a given input sequence.

The gate unit logistic sigmoid (with range [0, 1]) used in the experiments is

$$f(x) = \frac{1}{1 + \exp(-x)} . \tag{A.1}$$

The function h (with range $[-1, 1]$) used in the experiments is

$$h(x) = \frac{2}{1 + \exp(-x)} - 1 . \tag{A.2}$$

The function g (with range $[-2, 2]$) used in the experiments is

$$g(x) = \frac{4}{1 + \exp(-x)} - 2 . \tag{A.3}$$

A.1.1 Forward Pass. The net input and the activation of hidden unit i are

$$\begin{aligned} net_i(t) &= \sum_u w_{iu} y^u(t-1) \\ y^i(t) &= f_i(net_i(t)) . \end{aligned} \tag{A.4}$$

The net input and the activation of in_j are

$$\begin{aligned} net_{in_j}(t) &= \sum_u w_{in_j u} y^u(t-1) \\ y^{in_j}(t) &= f_{in_j}(net_{in_j}(t)) . \end{aligned} \tag{A.5}$$

The net input and the activation of out_j are

$$\begin{aligned} net_{out_j}(t) &= \sum_u w_{out_j u} y^u(t-1) \\ y^{out_j}(t) &= f_{out_j}(net_{out_j}(t)) . \end{aligned} \tag{A.6}$$

The net input $net_{c_j^v}$, the internal state $s_{c_j^v}$, and the output activation $y^{c_j^v}$ of the vth memory cell of memory cell block c_j are:

$$\begin{aligned} net_{c_j^v}(t) &= \sum_u w_{c_j^v u} y^u(t-1) \\ s_{c_j^v}(t) &= s_{c_j^v}(t-1) + y^{in_j}(t) g\left(net_{c_j^v}(t)\right) \\ y^{c_j^v}(t) &= y^{out_j}(t) h(s_{c_j^v}(t)) . \end{aligned} \tag{A.7}$$

The net input and the activation of output unit k are

$$\begin{aligned} net_k(t) &= \sum_{u:\ u \text{ not a gate}} w_{ku} y^u(t-1) \\ y^k(t) &= f_k(net_k(t)) . \end{aligned}$$

The backward pass to be described later is based on the following truncated backpropagation formulas.

A.1.2 Approximate Derivatives for Truncated Backpropagation. The truncated version (see section 4) only approximates the partial derivatives, which is reflected by the $\approx_{tr}$ signs in the notation below. It truncates error flow once it leaves memory cells or gate units. Truncation ensures that there are no loops across which an error that left some memory cell through its input or input gate can reenter the cell through its output or output gate. This in turn ensures constant error flow through the memory cell's CEC.

In the truncated backpropagation version, the following derivatives are replaced by zero:

$$\frac{\partial net_{in_j}(t)}{\partial y^u(t-1)} \approx_{tr} 0 \ \ \forall u,$$

$$\frac{\partial net_{out_j}(t)}{\partial y^u(t-1)} \approx_{tr} 0 \ \ \forall u,$$

and

$$\frac{\partial net_{c_j}(t)}{\partial y^u(t-1)} \approx_{tr} 0 \ \ \forall u.$$

Therefore we get

$$\frac{\partial y^{in_j}(t)}{\partial y^u(t-1)} = f'_{in_j}(net_{in_j}(t)) \frac{\partial net_{in_j}(t)}{\partial y^u(t-1)} \approx_{tr} 0 \ \ \forall u,$$

$$\frac{\partial y^{out_j}(t)}{\partial y^u(t-1)} = f'_{out_j}(net_{out_j}(t)) \frac{\partial net_{out_j}(t)}{\partial y^u(t-1)} \approx_{tr} 0 \ \ \forall u,$$

and

$$\begin{aligned}\frac{\partial y^{c_j}(t)}{\partial y^u(t-1)} = & \frac{\partial y^{c_j}(t)}{\partial net_{out_j}(t)} \frac{\partial net_{out_j}(t)}{\partial y^u(t-1)} + \frac{\partial y^{c_j}(t)}{\partial net_{in_j}(t)} \frac{\partial net_{in_j}(t)}{\partial y^u(t-1)} \\ & + \frac{\partial y^{c_j}(t)}{\partial net_{c_j}(t)} \frac{\partial net_{c_j}(t)}{\partial y^u(t-1)} \approx_{tr} 0 \ \ \forall u.\end{aligned}$$

This implies for all w_{lm} not on connections to c_j^v, in_j, out_j (that is, $l \notin \{c_j^v, in_j, out_j\}$):

$$\frac{\partial y^{c_j^v}(t)}{\partial w_{lm}} = \sum_u \frac{\partial y^{c_j^v}(t)}{\partial y^u(t-1)} \frac{\partial y^u(t-1)}{\partial w_{lm}} \approx_{tr} 0.$$

The truncated derivatives of output unit k are:

$$\frac{\partial y^k(t)}{\partial w_{lm}} = f'_k(net_k(t)) \left(\sum_{u:\ u \text{ not a gate}} w_{ku} \frac{\partial y^u(t-1)}{\partial w_{lm}} + \delta_{kl} y^m(t-1) \right)$$

$$
\begin{aligned}
&\approx_{tr} f_k'(net_k(t)) \left(\sum_j \sum_{v=1}^{S_j} \delta_{c_j^v l} w_{kc_j^v} \frac{\partial y^{c_j^v}(t-1)}{\partial w_{lm}} \right. \\
&\quad + \sum_j \left(\delta_{in_j l} + \delta_{out_j l} \right) \sum_{v=1}^{S_j} w_{kc_j^v} \frac{\partial y^{c_j^v}(t-1)}{\partial w_{lm}} \\
&\quad \left. + \sum_{i:\ i \text{ hidden unit}} w_{ki} \frac{\partial y^i(t-1)}{\partial w_{lm}} + \delta_{kl} y^m(t-1) \right) \\
&= f_k'(net_k(t)) \begin{cases} y^m(t-1) & l = k \\ w_{kc_j^v} \frac{\partial y^{c_j^v}(t-1)}{\partial w_{lm}} & l = c_j^v \\ \sum_{v=1}^{S_j} w_{kc_j^v} \frac{\partial y^{c_j^v}(t-1)}{\partial w_{lm}} & l = in_j \text{ OR } l = out_j \\ \sum_{i:\ i \text{ hidden unit}} w_{ki} \frac{\partial y^i(t-1)}{\partial w_{lm}} & l \text{ otherwise} \end{cases}
\end{aligned} \tag{A.8}
$$

where δ is the Kronecker delta ($\delta_{ab} = 1$ if $a = b$ and 0 otherwise), and S_j is the size of memory cell block c_j. The truncated derivatives of a hidden unit i that is not part of a memory cell are:

$$
\frac{\partial y^i(t)}{\partial w_{lm}} = f_i'(net_i(t)) \frac{\partial net_i(t)}{\partial w_{lm}} \approx_{tr} \delta_{li} f_i'(net_i(t)) y^m(t-1) . \tag{A.9}
$$

(Here it would be possible to use the full gradient without affecting constant error flow through internal states of memory cells.)

Cell block c_j's truncated derivatives are:

$$
\begin{aligned}
\frac{\partial y^{in_j}(t)}{\partial w_{lm}} &= f_{in_j}'(net_{in_j}(t)) \frac{\partial net_{in_j}(t)}{\partial w_{lm}} \\
&\approx_{tr} \delta_{in_j l} f_{in_j}'(net_{in_j}(t)) y^m(t-1) . \qquad \text{(A.10)} \\
\frac{\partial y^{out_j}(t)}{\partial w_{lm}} &= f_{out_j}'(net_{out_j}(t)) \frac{\partial net_{out_j}(t)}{\partial w_{lm}} \\
&\approx_{tr} \delta_{out_j l} f_{out_j}'(net_{out_j}(t)) y^m(t-1) . \qquad \text{(A.11)} \\
\frac{\partial s_{c_j^v}(t)}{\partial w_{lm}} &= \frac{\partial s_{c_j^v}(t-1)}{\partial w_{lm}} \\
&\quad + \frac{\partial y^{in_j}(t)}{\partial w_{lm}} g\left(net_{c_j^v}(t) \right) + y^{in_j}(t) g'\left(net_{c_j^v}(t) \right) \frac{\partial net_{c_j^v}(t)}{\partial w_{lm}} \\
&\approx_{tr} \left(\delta_{in_j l} + \delta_{c_j^v l} \right) \frac{\partial s_{c_j^v}(t-1)}{\partial w_{lm}} + \delta_{in_j l} \frac{\partial y^{in_j}(t)}{\partial w_{lm}} g\left(net_{c_j^v}(t) \right) \\
&\quad + \delta_{c_j^v l} y^{in_j}(t) g'\left(net_{c_j^v}(t) \right) \frac{\partial net_{c_j^v}(t)}{\partial w_{lm}}
\end{aligned}
$$

$$= \left(\delta_{in_j l} + \delta_{c_j^v l}\right) \frac{\partial s_{c_j^v}(t-1)}{\partial w_{lm}} + \delta_{in_j l} f'_{in_j}(net_{in_j}(t))\, g\left(net_{c_j^v}(t)\right) y^m(t-1) + \delta_{c_j^v l}\, y^{in_j}(t)\, g'\left(net_{c_j^v}(t)\right) y^m(t-1)\,. \tag{A.12}$$

$$\frac{\partial y^{c_j^v}(t)}{\partial w_{lm}} = \frac{\partial y^{out_j}(t)}{\partial w_{lm}} h(s_{c_j^v}(t)) + h'(s_{c_j^v}(t)) \frac{\partial s_{c_j^v}(t)}{\partial w_{lm}} y^{out_j}(t)$$

$$\approx_{tr} \delta_{out_j l} \frac{\partial y^{out_j}(t)}{\partial w_{lm}} h(s_{c_j^v}(t)) + \left(\delta_{in_j l} + \delta_{c_j^v l}\right) h'(s_{c_j^v}(t)) \frac{\partial s_{c_j^v}(t)}{\partial w_{lm}} y^{out_j}(t)\,. \tag{A.13}$$

To update the system efficiently at time t, the only (truncated) derivatives that need to be stored at time $t-1$ are

$$\frac{\partial s_{c_j^v}(t-1)}{\partial w_{lm}},$$

where $l = c_j^v$ or $l = in_j$.

A.1.3 Backward Pass. We will describe the backward pass only for the particularly efficient truncated gradient version of the LSTM algorithm. For simplicity we will use equal signs even where approximations are made according to the truncated backpropagation equations above.

The squared error at time t is given by

$$E(t) = \sum_{k:\, k \text{ output unit}} \left(t^k(t) - y^k(t)\right)^2, \tag{A.14}$$

where $t^k(t)$ is output unit k's target at time t.

Time t's contribution to w_{lm}'s gradient-based update with learning rate α is

$$\Delta w_{lm}(t) = -\alpha \frac{\partial E(t)}{\partial w_{lm}}\,. \tag{A.15}$$

We define some unit l's error at time step t by

$$e_l(t) := -\frac{\partial E(t)}{\partial net_l(t)}\,. \tag{A.16}$$

Using (almost) standard backpropagation, we first compute updates for weights to output units ($l = k$), weights to hidden units ($l = i$) and weights

to output gates ($l = out_j$). We obtain (compare formulas A.8, A.9, and A.11):

$$l = k \text{ (output)}: \quad e_k(t) = f_k'(net_k(t)) \left(t^k(t) - y^k(t)\right), \tag{A.17}$$

$$l = i \text{ (hidden)}: \quad e_i(t) = f_i'(net_i(t)) \sum_{k:\, k \text{ output unit}} w_{ki} e_k(t), \tag{A.18}$$

$l = out_j$ (output gates):

$$e_{out_j}(t) = f_{out_j}'(net_{out_j}(t)) \left(\sum_{v=1}^{S_j} h(s_{c_j^v}(t)) \sum_{k:\, k \text{ output unit}} w_{kc_j^v} e_k(t) \right). \tag{A.19}$$

For all possible l time t's contribution to w_{lm}'s update is

$$\Delta w_{lm}(t) = \alpha\, e_l(t)\, y^m(t-1). \tag{A.20}$$

The remaining updates for weights to input gates ($l = in_j$) and to cell units ($l = c_j^v$) are less conventional. We define some internal state $s_{c_j^v}$'s error:

$$\begin{aligned} e_{s_{c_j^v}} &:= -\frac{\partial E(t)}{\partial s_{c_j^v}(t)} \\ &= f_{out_j}(net_{out_j}(t))\; h'(s_{c_j^v}(t)) \sum_{k:\, k \text{ output unit}} w_{kc_j^v} e_k(t). \end{aligned} \tag{A.21}$$

We obtain for $l = in_j$ or $l = c_j^v,\ v = 1, \ldots, S_j$

$$-\frac{\partial E(t)}{\partial w_{lm}} = \sum_{v=1}^{S_j} e_{s_{c_j^v}}(t) \frac{\partial s_{c_j^v}(t)}{\partial w_{lm}}. \tag{A.22}$$

The derivatives of the internal states with respect to weights and the corresponding weight updates are as follows (compare expression A.12):

$l = in_j$ (input gates):

$$\frac{\partial s_{c_j^v}(t)}{\partial w_{in_j m}} = \frac{\partial s_{c_j^v}(t-1)}{\partial w_{in_j m}} + g(net_{c_j^v}(t))\, f_{in_j}'(net_{in_j}(t)) y^m(t-1); \tag{A.23}$$

therefore, time t's contribution to $w_{in_j m}$'s update is (compare expression A.8):

$$\Delta w_{in_j m}(t) = \alpha \sum_{v=1}^{S_j} e_{s_{c_j^v}}(t) \frac{\partial s_{c_j^v}(t)}{\partial w_{in_j m}}. \tag{A.24}$$

Similarly we get (compare expression A.12):

$$l = c_j^v \text{ (memory cells) :}$$
$$\frac{\partial s_{c_j^v}(t)}{\partial w_{c_j^v m}} = \frac{\partial s_{c_j^v}(t-1)}{\partial w_{c_j^v m}} + g'(net_{c_j^v}(t))\, f_{in_j}(net_{in_j}(t)) y^m(t-1)\,; \tag{A.25}$$

therefore time t's contribution to $w_{c_j^v m}$'s update is (compare expression A.8):

$$\Delta w_{c_j^v m}(t) = \alpha e_{s_{c_j^v}}(t) \frac{\partial s_{c_j^v}(t)}{\partial w_{c_j^v m}}\,. \tag{A.26}$$

All we need to implement for the backward pass are equations A.17 through A.21 and A.23 through A.26. Each weight's total update is the sum of the contributions of all time steps.

A.1.4 Computational Complexity. LSTM's update complexity per time step is

$$O(KH + KCS + HI + CSI) = O(W), \tag{A.27}$$

where K is the number of output units, C is the number of memory cell blocks, $S > 0$ is the size of the memory cell blocks, H is the number of hidden units, I is the (maximal) number of units forward connected to memory cells, gate units and hidden units, and

$$W = KH + KCS + CSI + 2CI + HI = O(KH + KCS + CSI + HI)$$

is the number of weights. Expression A.27 is obtained by considering all computations of the backward pass: equation A.17 needs K steps; A.18 needs KH steps; A.19 needs KSC steps; A.20 needs $K(H+C)$ steps for output units, HI steps for hidden units, CI steps for output gates; A.21 needs KCS steps; A.23 needs CSI steps; A.24 needs CSI steps; A.25 needs CSI steps; A.26 needs CSI steps. The total is $K + 2KH + KC + 2KSC + HI + CI + 4CSI$ steps, or $O(KH + KSC + HI + CSI)$ steps. We conclude that LSTM algorithm's update complexity per time step is just like BPTT's for a fully recurrent net.

At a given time step, only the $2CSI$ most recent $\partial s_{c_j^v}/\partial w_{lm}$ values from equations A.23 and A.25 need to be stored. Hence LSTM's storage complexity also is $O(W)$; it does not depend on the input sequence length.

A.2 Error Flow. We compute how much an error signal is scaled while flowing back through a memory cell for q time steps. As a by-product, this analysis reconfirms that the error flow within a memory cell's CEC is indeed constant, provided that truncated backpropagation cuts off error flow trying to leave memory cells (see also section 3.2). The analysis also highlights a

potential for undesirable long-term drifts of s_{c_j}, as well as the beneficial, countermanding influence of negatively biased input gates.

Using the truncated backpropagation learning rule, we obtain

$$\begin{aligned}
\frac{\partial s_{c_j}(t-k)}{\partial s_{c_j}(t-k-1)} &= 1 + \frac{\partial y^{in_j}(t-k)}{\partial s_{c_j}(t-k-1)} g\left(net_{c_j}(t-k)\right) \\
&\quad + y^{in_j}(t-k) g'\left(net_{c_j}(t-k)\right) \frac{\partial net_{c_j}(t-k)}{\partial s_{c_j}(t-k-1)} \\
&= 1 + \sum_u \left[\frac{\partial y^{in_j}(t-k)}{\partial y^u(t-k-1)} \frac{\partial y^u(t-k-1)}{\partial s_{c_j}(t-k-1)} \right] \\
&\quad \times g\left(net_{c_j}(t-k)\right) \\
&\quad + y^{in_j}(t-k) g'\left(net_{c_j}(t-k)\right) \\
&\quad \times \sum_u \left[\frac{\partial net_{c_j}(t-k)}{\partial y^u(t-k-1)} \frac{\partial y^u(t-k-1)}{\partial s_{c_j}(t-k-1)} \right] \\
&\approx_{tr} 1. \qquad \text{(A.28)}
\end{aligned}$$

The $\approx_{tr}$ sign indicates equality due to the fact that truncated backpropagation replaces by zero the following derivatives:

$$\frac{\partial y^{in_j}(t-k)}{\partial y^u(t-k-1)} \ \forall u \quad \text{and} \quad \frac{\partial net_{c_j}(t-k)}{\partial y^u(t-k-1)} \ \forall u.$$

In what follows, an error $\vartheta_j(t)$ starts flowing back at c_j's output. We redefine

$$\vartheta_j(t) := \sum_i w_{ic_j} \vartheta_i(t+1) \, . \qquad \text{(A.29)}$$

Following the definitions and conventions of section 3.1, we compute error flow for the truncated backpropagation learning rule. The error occurring at the output gate is

$$\vartheta_{out_j}(t) \approx_{tr} \frac{\partial y^{out_j}(t)}{\partial net_{out_j}(t)} \frac{\partial y^{c_j}(t)}{\partial y^{out_j}(t)} \vartheta_j(t) \, . \qquad \text{(A.30)}$$

The error occurring at the internal state is

$$\vartheta_{s_{c_j}}(t) = \frac{\partial s_{c_j}(t+1)}{\partial s_{c_j}(t)} \vartheta_{s_{c_j}}(t+1) + \frac{\partial y^{c_j}(t)}{\partial s_{c_j}(t)} \vartheta_j(t) \, . \qquad \text{(A.31)}$$

Since we use truncated backpropagation we have

$$\vartheta_j(t) = \sum_{i: i \text{ no gate and no memory cell}} w_{ic_j} \vartheta_i(t+1);$$

therefore we get

$$\frac{\partial \vartheta_j(t)}{\partial \vartheta_{s_{c_j}}(t+1)} = \sum_i w_{ic_j} \frac{\partial \vartheta_i(t+1)}{\partial \vartheta_{s_{c_j}}(t+1)} \approx_{tr} 0 . \tag{A.32}$$

Equations A.31 and A.32 imply constant error flow through internal states of memory cells:

$$\frac{\partial \vartheta_{s_{c_j}}(t)}{\partial \vartheta_{s_{c_j}}(t+1)} = \frac{\partial s_{c_j}(t+1)}{\partial s_{c_j}(t)} \approx_{tr} 1 . \tag{A.33}$$

The error occurring at the memory cell input is

$$\vartheta_{c_j}(t) = \frac{\partial g(net_{c_j}(t))}{\partial net_{c_j}(t)} \frac{\partial s_{c_j}(t)}{\partial g(net_{c_j}(t))} \vartheta_{s_{c_j}}(t) . \tag{A.34}$$

The error occurring at the input gate is

$$\vartheta_{in_j}(t) \approx_{tr} \frac{\partial y^{in_j}(t)}{\partial net_{in_j}(t)} \frac{\partial s_{c_j}(t)}{\partial y^{in_j}(t))} \vartheta_{s_{c_j}}(t) . \tag{A.35}$$

A.2.1 No External Error Flow. Errors are propagated back from units l to unit v along outgoing connections with weights w_{lv}. This "external error" (note that for conventional units there is nothing but external error) at time t is

$$\vartheta_v^e(t) = \frac{\partial y^v(t)}{\partial net_v(t)} \sum_l \frac{\partial net_l(t+1)}{\partial y^v(t)} \vartheta_l(t+1) . \tag{A.36}$$

We obtain

$$\begin{aligned} \frac{\partial \vartheta_v^e(t-1)}{\partial \vartheta_j(t)} &= \frac{\partial y^v(t-1)}{\partial net_v(t-1)} \left(\frac{\partial \vartheta_{out_j}(t)}{\partial \vartheta_j(t)} \frac{\partial net_{out_j}(t)}{\partial y^v(t-1)} \right. \\ &\quad \left. + \frac{\partial \vartheta_{in_j}(t)}{\partial \vartheta_j(t)} \frac{\partial net_{in_j}(t)}{\partial y^v(t-1)} + \frac{\partial \vartheta_{c_j}(t)}{\partial \vartheta_j(t)} \frac{\partial net_{c_j}(t)}{\partial y^v(t-1)} \right) \\ &\approx_{tr} 0 . \end{aligned} \tag{A.37}$$

We observe that the error ϑ_j arriving at the memory cell output is not back-propagated to units v by external connections to in_j, out_j, c_j.

A.2.2 Error Flow Within Memory Cells. We now focus on the error back-flow within a memory cell's CEC. This is actually the only type of error flow that can bridge several time steps. Suppose error $\vartheta_j(t)$ arrives at c_j's output

at time t and is propagated back for q steps until it reaches in_j or the memory cell input $g(net_{c_j})$. It is scaled by a factor of

$$\frac{\partial \vartheta_v(t-q)}{\partial \vartheta_j(t)},$$

where $v = in_j, c_j$. We first compute

$$\frac{\partial \vartheta_{s_{c_j}}(t-q)}{\partial \vartheta_j(t)} \approx_{tr} \begin{cases} \frac{\partial y^{c_j}(t)}{\partial s_{c_j}(t)} & q = 0 \\ \frac{\partial s_{c_j}(t-q+1)}{\partial s_{c_j}(t-q)} \frac{\partial \vartheta_{s_{c_j}}(t-q+1)}{\partial \vartheta_j(t)} & q > 0 \end{cases} . \tag{A.38}$$

Expanding equation A.38, we obtain

$$\begin{aligned} \frac{\partial \vartheta_v(t-q)}{\partial \vartheta_j(t)} &\approx_{tr} \frac{\partial \vartheta_v(t-q)}{\partial \vartheta_{s_{c_j}}(t-q)} \frac{\partial \vartheta_{s_{c_j}}(t-q)}{\partial \vartheta_j(t)} \\ &\approx_{tr} \frac{\partial \vartheta_v(t-q)}{\partial \vartheta_{s_{c_j}}(t-q)} \left(\prod_{m=q}^{1} \frac{\partial s_{c_j}(t-m+1)}{\partial s_{c_j}(t-m)} \right) \frac{\partial y^{c_j}(t)}{\partial s_{c_j}(t)} \\ &\approx_{tr} y^{out_j}(t) h'(s_{c_j}(t)) \begin{cases} g'(net_{c_j}(t-q) y^{in_j}(t-q) & v = c_j \\ g(net_{c_j}(t-q) f'_{in_j}(net_{in_j}(t-q)) & v = in_j \end{cases} \end{aligned} \tag{A.39}$$

Consider the factors in the previous equation's last expression. Obviously, error flow is scaled only at times t (when it enters the cell) and $t-q$ (when it leaves the cell), but not in between (constant error flow through the CEC). We observe:

1. The output gate's effect is $y^{out_j}(t)$ scales down those errors that can be reduced early during training without using the memory cell. It also scales down those errors resulting from using (activating / deactivating) the memory cell at later training stages. Without the output gate, the memory cell might, for instance, suddenly start causing avoidable errors in situations that already seemed under control (because it was easy to reduce the corresponding errors without memory cells). See "Output Weight Conflict" in section 3 and "Abuse Problem and Solution" (section 4.7).
2. If there are large positive or negative $s_{c_j}(t)$ values (because s_{c_j} has drifted since time step $t-q$), then $h'(s_{c_j}(t))$ may be small (assuming that h is a logistic sigmoid). See section 4. Drifts of the memory cell's internal state s_{c_j} can be countermanded by negatively biasing the input gate in_j (see section 4 and the next point). Recall from section 4 that the precise bias value does not matter much.
3. $y^{in_j}(t-q)$ and $f'_{in_j}(net_{in_j}(t-q))$ are small if the input gate is negatively biased (assume f_{in_j} is a logistic sigmoid). However, the potential sig-

nificance of this is negligible compared to the potential significance of drifts of the internal state s_{c_j}.

Some of the factors above may scale down LSTM's overall error flow, but not in a manner that depends on the length of the time lag. The flow will still be much more effective than an exponentially (of order q) decaying flow without memory cells.

Acknowledgments

Thanks to Mike Mozer, Wilfried Brauer, Nic Schraudolph, and several anonymous referees for valuable comments and suggestions that helped to improve a previous version of this article (Hochreiter and Schmidhuber, 1995). This work was supported by DFG grant SCHM 942/3-1 from Deutsche Forschungsgemeinschaft.

References

Almeida, L. B. (1987). A learning rule for asynchronous perceptrons with feedback in a combinatorial environment. In *IEEE 1st International Conference on Neural Networks, San Diego* (Vol. 2, pp. 609–618).

Baldi, P., & Pineda, F. (1991). Contrastive learning and neural oscillator. *Neural Computation, 3*, 526–545.

Bengio, Y., & Frasconi, P. (1994). Credit assignment through time: Alternatives to backpropagation. In J. D. Cowan, G. Tesauro, & J. Alspector (Eds.), *Advances in neural information processing systems 6* (pp. 75–82). San Mateo, CA: Morgan Kaufmann.

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks, 5*(2), 157–166.

Cleeremans, A., Servan-Schreiber, D., & McClelland, J. L. (1989). Finite-state automata and simple recurrent networks. *Neural Computation, 1*, 372–381.

de Vries, B., & Principe, J. C. (1991). A theory for neural networks with time delays. In R. P. Lippmann, J. E. Moody, & D. S. Touretzky (Eds.), *Advances in neural information processing systems 3*, (pp. 162–168). San Mateo, CA: Morgan Kaufmann.

Doya, K. (1992). Bifurcations in the learning of recurrent neural networks. In *Proceedings of 1992 IEEE International Symposium on Circuits and Systems* (pp. 2777–2780).

Doya, K., & Yoshizawa, S. (1989). Adaptive neural oscillator using continuous-time backpropagation learning. *Neural Networks, 2*, 375–385.

Elman, J. L. (1988). *Finding structure in time* (Tech. Rep. No. CRL 8801). San Diego: Center for Research in Language, University of California, San Diego.

Fahlman, S. E. (1991). The recurrent cascade-correlation learning algorithm. In R. P. Lippmann, J. E. Moody, & D. S. Touretzky (Eds.), *Advances in neural information processing systems 3* (pp. 190–196). San Mateo, CA: Morgan Kaufmann.

LONG SHORT-TERM MEMORY BASED RECURRENT NEURAL NETWORK ARCHITECTURES FOR LARGE VOCABULARY SPEECH RECOGNITION

Haşim Sak, Andrew Senior, Françoise Beaufays

Google

{hasim,andrewsenior,fsb@google.com}

ABSTRACT

Long Short-Term Memory (LSTM) is a recurrent neural network (RNN) architecture that has been designed to address the vanishing and exploding gradient problems of conventional RNNs. Unlike feedforward neural networks, RNNs have cyclic connections making them powerful for modeling sequences. They have been successfully used for sequence labeling and sequence prediction tasks, such as handwriting recognition, language modeling, phonetic labeling of acoustic frames. However, in contrast to the deep neural networks, the use of RNNs in speech recognition has been limited to phone recognition in small scale tasks. In this paper, we present novel LSTM based RNN architectures which make more effective use of model parameters to train acoustic models for large vocabulary speech recognition. We train and compare LSTM, RNN and DNN models at various numbers of parameters and configurations. We show that LSTM models converge quickly and give state of the art speech recognition performance for relatively small sized models. [1]

Index Terms— Long Short-Term Memory, LSTM, recurrent neural network, RNN, speech recognition.

1. INTRODUCTION

Unlike feedforward neural networks (FFNN) such as deep neural networks (DNNs), the architecture of recurrent neural networks (RNNs) have cycles feeding the activations from previous time steps as input to the network to make a decision for the current input. The activations from the previous time step are stored in the internal state of the network and they provide indefinite temporal contextual information in contrast to the fixed contextual windows used as inputs in FFNNs. Therefore, RNNs use a dynamically changing contextual window of all sequence history rather than a static fixed size window over the sequence. This capability makes RNNs better suited for sequence modeling tasks such as sequence prediction and sequence labeling tasks.

However, training conventional RNNs with the gradient-based back-propagation through time (BPTT) technique is difficult due to the vanishing gradient and exploding gradient problems [1]. In addition, these problems limit the capability of RNNs to model the long range context dependencies to 5-10 discrete time steps between relevant input signals and output.

To address these problems, an elegant RNN architecture – *Long Short-Term Memory* (LSTM) – has been designed [2]. The original architecture of LSTMs contained special units called *memory blocks* in the recurrent hidden layer. The memory blocks contain memory cells with self-connections storing (remembering) the temporal state of the network in addition to special multiplicative units called gates to control the flow of information. Each memory block contains an *input gate* which controls the flow of input activations into the memory cell and an *output gate* which controls the output flow of cell activations into the rest of the network. Later, to address a weakness of LSTM models preventing them from processing continuous input streams that are not segmented into subsequences – which would allow resetting the cell states at the begining of subsequences – a *forget gate* was added to the memory block [3]. A forget gate scales the internal state of the cell before adding it as input to the cell through self recurrent connection of the cell, therefore adaptively forgetting or resetting cell's memory. Besides, the modern LSTM architecture contains *peephole connections* from its internal cells to the gates in the same cell to learn precise timing of the outputs [4].

LSTMs and conventional RNNs have been successfully applied to sequence prediction and sequence labeling tasks. LSTM models have been shown to perform better than RNNs on learning context-free and context-sensitive languages [5]. Bidirectional LSTM networks similar to bidirectional RNNs [6] operating on the input sequence in both direction to make a decision for the current input has been proposed for phonetic labeling of acoustic frames on the TIMIT speech database [7]. For online and offline handwriting recognition, bidirectional LSTM networks with a connectionist temporal classification (CTC) output layer using a forward backward type of algorithm which allows the network to be trained on unsegmented sequence data, have been shown to outperform a state of the art HMM-based system [8]. Recently, following the success of DNNs for acoustic modeling [9, 10, 11], a deep LSTM RNN – a stack of multiple LSTM layers – combined with a CTC output layer and an RNN transducer predicting phone sequences – has been shown to get the state of the art results in phone recognition on the TIMIT database [12]. In language modeling, a conventional RNN has obtained very significant reduction of perplexity over standard n-gram models [13].

While DNNs have shown state of the art performance in both phone recognition and large vocabulary speech recognition [9, 10, 11], the application of LSTM networks has been limited to phone recognition on the TIMIT database, and it has required using additional techniques and models such as CTC and RNN transducer to obtain better results than DNNs.

In this paper, we show that LSTM based RNN architectures can obtain state of the art performance in a large vocabulary speech recognition system with thousands of context dependent (CD) states. The proposed architectures modify the standard architecture of the LSTM networks to make better use of the model parameters while addressing the computational efficiency problems of large networks.

[1] The original manuscript has been submitted to ICASSP 2014 conference on November 4, 2013 and it has been rejected due to having content on the reference only 5th page. This version has been slightly edited to reflect the latest experimental results.

2. LSTM ARCHITECTURES

In the standard architecture of LSTM networks, there are an input layer, a recurrent LSTM layer and an output layer. The input layer is connected to the LSTM layer. The recurrent connections in the LSTM layer are directly from the cell output units to the cell input units, input gates, output gates and forget gates. The cell output units are connected to the output layer of the network. The total number of parameters W in a standard LSTM network with one cell in each memory block, ignoring the biases, can be calculated as follows:

$$W = n_c \times n_c \times 4 + n_i \times n_c \times 4 + n_c \times n_o + n_c \times 3$$

where n_c is the number of memory cells (and number of memory blocks in this case), n_i is the number of input units, and n_o is the number of output units. The computational complexity of learning LSTM models per weight and time step with the stochastic gradient descent (SGD) optimization technique is $O(1)$. Therefore, the learning computational complexity per time step is $O(W)$. The learning time for a network with a relatively small number of inputs is dominated by the $n_c \times (n_c + n_o)$ factor. For the tasks requiring a large number of output units and a large number of memory cells to store temporal contextual information, learning LSTM models become computationally expensive.

As an alternative to the standard architecture, we propose two novel architectures to address the computational complexity of learning LSTM models. The two architectures are shown in the same Figure 1. In one of them, we connect the cell output units to a recurrent projection layer which connects to the cell input units and gates for recurrency in addition to network output units for the prediction of the outputs. Hence, the number of parameters in this model is $n_c \times n_r \times 4 + n_i \times n_c \times 4 + n_r \times n_o + n_c \times n_r + n_c \times 3$, where n_r is the number of units in the recurrent projection layer. In the other one, in addition to the recurrent projection layer, we add another non-recurrent projection layer which is directly connected to the output layer. This model has $n_c \times n_r \times 4 + n_i \times n_c \times 4 + (n_r + n_p) \times n_o + n_c \times (n_r + n_p) + n_c \times 3$ parameters, where n_p is the number of units in the non-recurrent projection layer and it allows us to increase the number of units in the projection layers without increasing the number of parameters in the recurrent connections ($n_c \times n_r \times 4$). Note that having two projection layers with regard to output units is effectively equivalent to having a single projection layer with $n_r + n_p$ units.

An LSTM network computes a mapping from an input sequence $x = (x_1, ..., x_T)$ to an output sequence $y = (y_1, ..., y_T)$ by calculating the network unit activations using the following equations iteratively from $t = 1$ to T:

$$i_t = \sigma(W_{ix}x_t + W_{im}m_{t-1} + W_{ic}c_{t-1} + b_i) \quad (1)$$
$$f_t = \sigma(W_{fx}x_t + W_{mf}m_{t-1} + W_{cf}c_{t-1} + b_f) \quad (2)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g(W_{cx}x_t + W_{cm}m_{t-1} + b_c) \quad (3)$$
$$o_t = \sigma(W_{ox}x_t + W_{om}m_{t-1} + W_{oc}c_t + b_o) \quad (4)$$
$$m_t = o_t \odot h(c_t) \quad (5)$$
$$y_t = W_{ym}m_t + b_y \quad (6)$$

where the W terms denote weight matrices (e.g. W_{ix} is the matrix of weights from the input gate to the input), the b terms denote bias vectors (b_i is the input gate bias vector), σ is the logistic sigmoid function, and i, f, o and c are respectively the input gate, forget gate, output gate and cell activation vectors, all of which are the same size as the cell output activation vector m, $\odot$ is the element-wise product

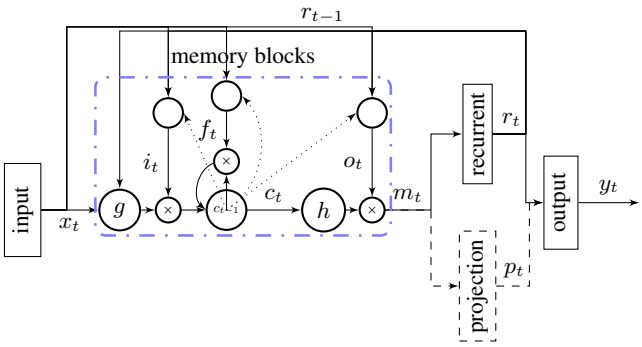


Fig. 1. LSTM based RNN architectures with a recurrent projection layer and an optional non-recurrent projection layer. A single memory block is shown for clarity.

of the vectors and g and h are the cell input and cell output activation functions, generally $tanh$.

With the proposed LSTM architecture with both recurrent and non-recurrent projection layer, the equations are as follows:

$$i_t = \sigma(W_{ix}x_t + W_{ir}r_{t-1} + W_{ic}c_{t-1} + b_i) \quad (7)$$
$$f_t = \sigma(W_{fx}x_t + W_{rf}r_{t-1} + W_{cf}c_{t-1} + b_f) \quad (8)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g(W_{cx}x_t + W_{cr}r_{t-1} + b_c) \quad (9)$$
$$o_t = \sigma(W_{ox}x_t + W_{or}r_{t-1} + W_{oc}c_t + b_o) \quad (10)$$
$$m_t = o_t \odot h(c_t) \quad (11)$$
$$r_t = W_{rm}m_t \quad (12)$$
$$p_t = W_{pm}m_t \quad (13)$$
$$y_t = W_{yr}r_t + W_{yp}p_t + b_y \quad (14)$$
$$(15)$$

where the r and p denote the recurrent and optional non-recurrent unit activations.

2.1. Implementation

We choose to implement the proposed LSTM architectures on multi-core CPU on a single machine rather than on GPU. The decision was based on CPU's relatively simpler implementation complexity and ease of debugging. CPU implementation also allows easier distributed implementation on a large cluster of machines if the learning time of large networks becomes a major bottleneck on a single machine [14]. For matrix operations, we use the Eigen matrix library [15]. This templated C++ library provides efficient implementations for matrix operations on CPU using vectorized instructions (SIMD – single instruction multiple data). We implemented activation functions and gradient calculations on matrices using SIMD instructions to benefit from parallelization.

We use the asynchronous stochastic gradient descent (ASGD) optimization technique. The update of the parameters with the gradients is done asynchronously from multiple threads on a multi-core machine. Each thread operates on a batch of sequences in parallel for computational efficiency – for instance, we can do matrix-matrix multiplications rather than vector-matrix multiplications – and for more stochasticity since model parameters can be updated from multiple input sequence at the same time. In addition to batching of sequences in a single thread, training with multiple threads effectively

results in much larger batch of sequences (number of threads times batch size) to be processed in parallel.

We use the truncated backpropagation through time (BPTT) learning algorithm to update the model parameters [16]. We use a fixed time step T_{bptt} (e.g. 20) to forward-propagate the activations and backward-propagate the gradients. In the learning process, we split an input sequence into a vector of subsequences of size T_{bptt}. The subsequences of an utterance are processed in their original order. First, we calculate and forward-propagate the activations iteratively using the network input and the activations from the previous time step for T_{bptt} time steps starting from the first frame and calculate the network errors using network cost function at each time step. Then, we calculate and back-propagate the gradients from a cross-entropy criterion, using the errors at each time step and the gradients from the next time step starting from the time T_{bptt}. Finally, the gradients for the network parameters (weights) are accumulated for T_{bptt} time steps and the weights are updated. The state of memory cells after processing each subsequence is saved for the next subsequence. Note that when processing multiple subsequences from different input sequences, some subsequences can be shorter than T_{bptt} since we could reach the end of those sequences. In the next batch of subsequences, we replace them with subsequences from a new input sequence, and reset the state of the cells for them.

3. EXPERIMENTS

We evaluate and compare the performance of DNN, RNN and LSTM neural network architectures on a large vocabulary speech recognition task – Google English Voice Search task.

3.1. Systems & Evaluation

All the networks are trained on a 3 million utterance (about 1900 hours) dataset consisting of anonymized and hand-transcribed Google voice search and dictation traffic. The dataset is represented with 25ms frames of 40-dimensional log-filterbank energy features computed every 10ms. The utterances are aligned with a 90 million parameter FFNN with 14247 CD states. We train networks for three different output states inventories: 126, 2000 and 8000. These are obtained by mapping 14247 states down to these smaller state inventories through equivalence classes. The 126 state set are the context independent (CI) states (3 x 42). The weights in all the networks before training are randomly initialized. We try to set the learning rate specific to a network architecture and its configuration to the largest value that results in a stable convergence. The learning rates are exponentially decayed during training.

During training, we evaluate frame accuracies (i.e. phone state labeling accuracy of acoustic frames) on a held out development set of 200,000 frames. The trained models are evaluated in a speech recognition system on a test set of 23,000 hand-transcribed utterances and the word error rates (WERs) are reported. The vocabulary size of the language model used in the decoding is 2.6 million.

The DNNs are trained with SGD with a minibatch size of 200 frames on a Graphics Processing Unit (GPU). Each network is fully connected with logistic sigmoid hidden layers and with a softmax output layer representing phone HMM states. For consistency with the LSTM architectures, some of the networks have a low-rank projection layer [17]. The DNNs inputs consist of stacked frames from an asymmetrical window, with 5 frames on the right and either 10 or 15 frames on the left (denoted 10w5 and 15w5 respectively)

The LSTM and conventional RNN architectures of various configurations are trained with ASGD with 24 threads, each asynchronously processing one partition of data, with each thread computing a gradient step on 4 or 8 subsequences from different utterances. A time step of 20 (T_{bptt}) is used to forward-propagate and the activations and backward-propagate the gradients using the truncated BPTT learning algorithm. The units in the hidden layer of RNNs use the logistic sigmoid activation function. The RNNs with the recurrent projection layer architecture use linear activation units in the projection layer. The LSTMs use hyperbolic tangent activation (tanh) for the cell input units and cell output units, and logistic sigmoid for the input, output and forget gate units. The recurrent projection and optional non-recurrent projection layers in the LSTMs use linear activation units. The input to the LSTMs and RNNs is 25ms frame of 40-dimensional log-filterbank energy features (no window of frames). Since the information from the future frames helps making better decisions for the current frame, consistent with the DNNs, we delay the output state label by 5 frames.

3.2. Results

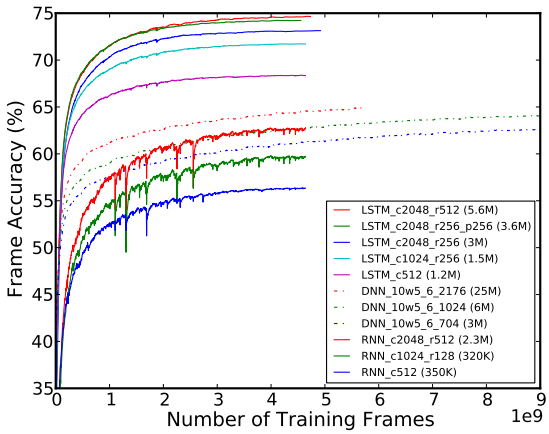


Fig. 2. 126 context independent phone HMM states.

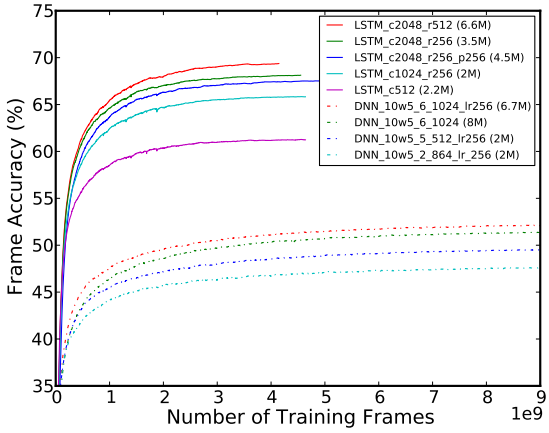


Fig. 3. 2000 context dependent phone HMM states.

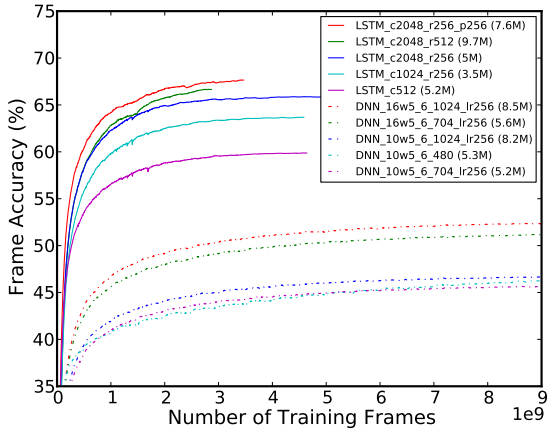


Fig. 4. 8000 context dependent phone HMM states.

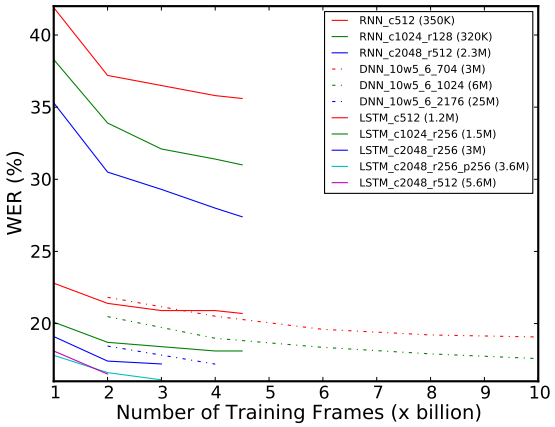


Fig. 5. 126 context independent phone HMM states.

Figure 2, 3, and 4 show the frame accuracy results for 126, 2000 and 8000 state outputs, respectively. In the figures, the name of the network configuration contains the information about the network size and architecture. cN states the number (N) of memory cells in the LSTMs and the number of units in the hidden layer in the RNNs. rN states the number of recurrent projection units in the LSTMs and RNNs. pN states the number of non-recurrent projection units in the LSTMs. The DNN configuration names state the left context and right context size (e.g. 10w5), the number of hidden layers (e.g. 6), the number of units in each of the hidden layers (e.g. 1024) and optional low-rank projection layer size (e.g. 256). The number of parameters in each model is given in parenthesis. We evaluated the RNNs only for 126 state output configuration, since they performed significantly worse than the DNNs and LSTMs. As can be seen from Figure 2, the RNNs were also very unstable at the beginning of the training and, to achieve convergence, we had to limit the activations and the gradients due to the exploding gradient problem. The LSTM networks give much better frame accuracy than the RNNs and DNNs while converging faster. The proposed LSTM projected RNN architectures give significantly better accuracy than the standard LSTM RNN architecture with the same number of parameters – compare *LSTM_512* with *LSTM_1024_256* in Figure 3. The LSTM network with both recurrent and non-recurrent projection layers generally performs better than the LSTM network with only recurrent projection layer except for the 2000 state experiment where we have set the learning rate too small.

Figure 5, 6, and 7 show the WERs for the same models for 126, 2000 and 8000 state outputs, respectively. Note that some of the LSTM networks have not converged yet, we will update the results when the models converge in the final revision of the paper. The speech recognition experiments show that the LSTM networks give improved speech recognition accuracy for the context independent 126 output state model, context dependent 2000 output state embedded size model (constrained to run on a mobile phone processor) and relatively large 8000 output state model. As can be seen from Figure 6, the proposed architectures (compare *LSTM_c1024_r256* with *LSTM_c512*) are essential for obtaining better recognition accuracies than DNNs. We also did an experiment to show that depth is very important for DNNs – compare *DNN_10w5_2_864_lr256* with *DNN_10w5_5_512_lr256* in Figure 6.

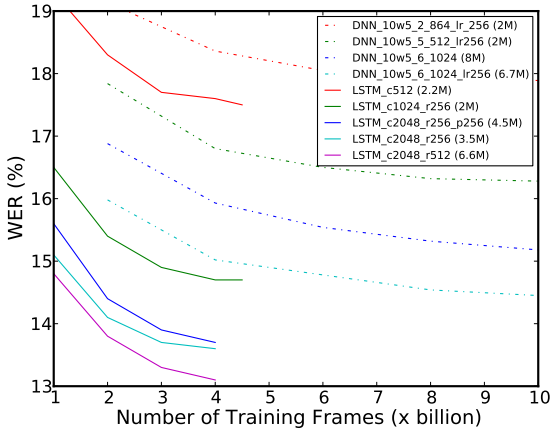


Fig. 6. 2000 context dependent phone HMM states.

4. CONCLUSION

As far as we know, this paper presents the first application of LSTM networks in a large vocabulary speech recognition task. To address the scalability issue of the LSTMs to large networks with large number of output units, we introduce two architecutures that make more effective use of model parameters than the standard LSTM architecture. One of the proposed architectures introduces a recurrent projection layer between the LSTM layer (which itself has no recursion) and the output layer. The other introduces another non-recurrent projection layer to increase the projection layer size without adding more recurrent connections and this decoupling provides more flexibility. We show that the proposed architectures improve the performance of the LSTM networks significantly over the standard LSTM. We also show that the proposed LSTM architectures give better performance than DNNs on a large vocabulary speech recognition task with a large number of output states. Training LSTM networks on a single multi-core machine does not scale well to larger networks. We will investigate GPU- and distributed CPU-implementations similar to [14] to address that.

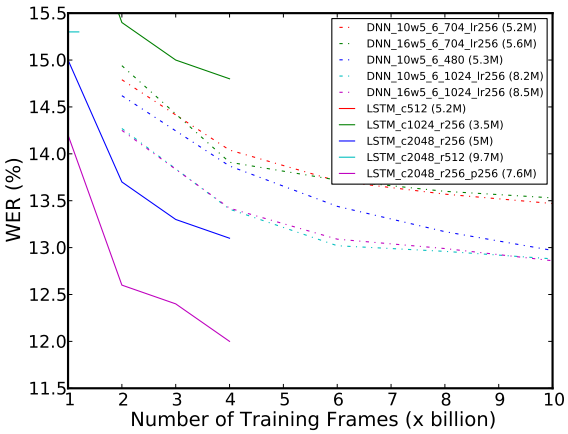


Fig. 7. 8000 context dependent phone HMM states.

5. REFERENCES

[1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi, "Learning long-term dependencies with gradient descent is difficult," *Neural Networks, IEEE Transactions on*, vol. 5, no. 2, pp. 157–166, 1994.

[2] Sepp Hochreiter and Jürgen Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[3] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

[4] Felix A. Gers, Nicol N. Schraudolph, and Jürgen Schmidhuber, "Learning precise timing with LSTM recurrent networks," *Journal of Machine Learning Research*, vol. 3, pp. 115–143, Mar. 2003.

[5] Felix A. Gers and Jürgen Schmidhuber, "LSTM recurrent networks learn simple context free and context sensitive languages," *IEEE Transactions on Neural Networks*, vol. 12, no. 6, pp. 1333–1340, 2001.

[6] Mike Schuster and Kuldip K. Paliwal, "Bidirectional recurrent neural networks," *Signal Processing, IEEE Transactions on*, vol. 45, no. 11, pp. 2673–2681, 1997.

[7] Alex Graves and Jürgen Schmidhuber, "Framewise phoneme classification with bidirectional LSTM and other neural network architectures," *Neural Networks*, vol. 12, pp. 5–6, 2005.

[8] Alex Graves, Marcus Liwicki, Santiago Fernandez, Roman Bertolami, Horst Bunke, and Jürgen Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, vol. 31, no. 5, pp. 855–868, 2009.

[9] Abdel Rahman Mohamed, George E. Dahl, and Geoffrey E. Hinton, "Acoustic modeling using deep belief networks," *IEEE Transactions on Audio, Speech & Language Processing*, vol. 20, no. 1, pp. 14–22, 2012.

[10] George E. Dahl, Dong Yu, Li Deng, and Alex Acero, "Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition," *IEEE Transactions on Audio, Speech & Language Processing*, vol. 20, no. 1, pp. 30–42, Jan. 2012.

[11] Navdeep Jaitly, Patrick Nguyen, Andrew Senior, and Vincent Vanhoucke, "Application of pretrained deep neural networks to large vocabulary speech recognition," in *Proceedings of INTERSPEECH*, 2012.

[12] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton, "Speech recognition with deep recurrent neural networks," in *Proceedings of ICASSP*, 2013.

[13] Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur, "Recurrent neural network based language model," in *Proceedings of INTERSPEECH*. 2010, vol. 2010, pp. 1045–1048, International Speech Communication Association.

[14] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc'Aurelio Ranzato, Andrew W. Senior, Paul A. Tucker, Ke Yang, and Andrew Y. Ng, "Large scale distributed deep networks.," in *NIPS*, 2012, pp. 1232–1240.

[15] Gaël Guennebaud, Benoît Jacob, et al., "Eigen v3," http://eigen.tuxfamily.org, 2010.

[16] Ronald J. Williams and Jing Peng, "An efficient gradient-based algorithm for online training of recurrent network trajectories," *Neural Computation*, vol. 2, pp. 490–501, 1990.

[17] T.N. Sainath, B. Kingsbury, V. Sindhwani, E. Arisoy, and B. Ramabhadran, "Low-rank matrix factorization for deep neural network training with high-dimensional output targets," in *Proc. ICASSP*, 2013.

To appear as a part of an upcoming textbook on deep learning.

Recurrent Neural Networks and Long Short-Term Memory Networks: Tutorial and Survey

Benyamin Ghojogh BGHOJOGH@UWATERLOO.CA
Ali Ghodsi ALI.GHODSI@UWATERLOO.CA
Department of Statistics and Actuarial Science & David R. Cheriton School of Computer Science,
Data Analytics Laboratory, University of Waterloo, Waterloo, ON, Canada

YouTube channel of Benyamin Ghojogh: https://www.youtube.com/@bghojogh
YouTube channel of Ali Ghodsi: https://www.youtube.com/@DataScienceCoursesUW

Abstract

This is a tutorial paper on Recurrent Neural Network (RNN), Long Short-Term Memory Network (LSTM), and their variants. We start with a dynamical system and backpropagation through time for RNN. Then, we discuss the problems of gradient vanishing and explosion in long-term dependencies. We explain close-to-identity weight matrix, long delays, leaky units, and echo state networks for solving this problem. Then, we introduce LSTM gates and cells, history and variants of LSTM, and Gated Recurrent Units (GRU). Finally, we introduce bidirectional RNN, bidirectional LSTM, and the Embeddings from Language Model (ELMo) network, for processing a sequence in both directions.

1. Introduction

Before the era of transformers in deep learning, regular neural networks could not process sequences, such as sentences (sequence of words) or speech (sequence of phonemes), properly without any recurrence. Recurrent Neural Network (RNN), proposed in (Rumelhart et al., 1986), is a dynamical system which considers recurrence. In recurrence, the output of a model is fed as input to the model again in the next time step. One of the main training algorithms for RNN is Backpropagation Through Time (BPTT), developed by several works (Robinson & Fallside, 1987; Werbos, 1988; Williams, 1989; Williams & Zipser, 1995; Mozer, 1995), which is similar to the backpropagation algorithm (Rumelhart et al., 1986) but has also chain rule through time. There were some problems with gradient vanishing or explosion for long-term dependencies in RNN (Bengio et al., 1993; 1994). Several solutions were proposed for this issue, some of which are close-to-identity weight matrix (Mikolov et al., 2015), long delays (Lin et al., 1995), leaky units (Jaeger et al., 2007; Sutskever & Hinton, 2010), and echo state networks (Jaeger & Haas, 2004; Jaeger, 2007).

Sequence modeling requires both short-term and long-term dependencies. For example, consider the sentence "The police is chasing the thief". In this sentence, the words "police" and "thief" are related to each other with short-term dependency because they are close to one another in the sequence of words. Another example is the sentence "I was born in France. My father was working there for many years during my childhood. My family had a great time there while my father was making money in his business there. That is why I know how to speak French". In this second example, the words "France" and "French" are related to each other with long-term dependency because they are far away from one another in the sequence of words. That inspired researchers to propose the Long Short-Term Memory (LSTM) network to handle both short-term and long-tern dependencies (Hochreiter & Schmidhuber, 1995; 1997). Later, Grated Recurrent Unit (GRU) was proposed (Cho et al., 2014) which simplified LSTM to reduce its unnecessary complexity.

RNN and LSTM networks are causal models which condition every sequence element on the *previous* elements in the sequence. Later researches showed that processing the sequence in both directions can perform better for the sequences which can be processed offline; e.g., if the chunks of sequence can be saved and processed and the sequence elements should not be processed as a stream (Graves & Schmidhuber, 2005a;b). Therefore, bidirectional RNN (Schuster & Paliwal, 1997; Baldi et al., 1999) and bidirectional LSTM (Graves & Schmidhuber, 2005a;b)

were proposed to process sequences in both directions. The Embeddings from Language Model (ELMo) network (Peters et al., 2018) is a language model which makes use of the bidirectional LSTM.

This is a tutorial on RNN, LSTM, and their variants. There exist some other tutorials and surveys about this topic, some of which are (Jaeger, 2002; Jozefowicz et al., 2015; Lipton et al., 2015; Schmidhuber, 2015; Greff et al., 2016; Salehinejad et al., 2017; Staudemeyer & Morris, 2019; Yu et al., 2019; Smagulova & James, 2019). A very good survey on the variants of LSTM is (Greff et al., 2016).

2. Recurrent Neural Network

2.1. Dynamical System

A dynamical system is recursive and its classical form is as follows:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}), \tag{1}$$

where t denotes the time step, $\boldsymbol{h}_t$ is the state at time t, and $f_\theta(.)$ is a function fixed between the states of all time steps. Figure 1-a shows such a system. Dynamical systems are widely used in chaos theory (Broer & Takens, 2011).

We can have a dynamical system with external input signal where $\boldsymbol{x}_t$ denotes the input signal at time t. This system is modeled as:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t). \tag{2}$$

This system is depicted in Fig. 1-b.

2.2. Parameter Sharing

The state $\boldsymbol{h}_t$ can be considered as a summary of the past sequence of inputs and states. If a different function f_θ is defined for each possible sequence length, the model will not have generalization. If the same parameters are used for any sequence length, the model will have generalization properties. Therefore, the parameters are shared for all lengths and between all states. Such a dynamical system with parameter sharing can be implemented as a neural network with weights. Such a network is called a Recurrent Neural Network (RNN), which was proposed in (Rumelhart et al., 1986).

RNN is illustrated in Fig. 1-c, where the same weight matrices are used for all time slots. RNN gets a sequence as input and outputs a sequence as a decision for a task such as regression or classification. Suppose the input, output, and state at time slot t are denoted by $\boldsymbol{x}_t \in \mathbb{R}^d$, $\boldsymbol{y}_t \in \mathbb{R}^q$, and $\boldsymbol{h}_t \in \mathbb{R}^p$, respectively. Let $\boldsymbol{W} \in \mathbb{R}^{p \times p}$ be the weight matrix between states, $\boldsymbol{U} \in \mathbb{R}^{p \times d}$ be the weight matrix between the inputs and the states, and $\boldsymbol{V} \in \mathbb{R}^{q \times p}$ denote the weight matrix between the states and outputs. The bias weights for the state and the output are denoted by $\boldsymbol{b}_i \in \mathbb{R}^p$ and $\boldsymbol{b}_y \in \mathbb{R}^q$, respectively. As shown in Fig. 1-c, we have:

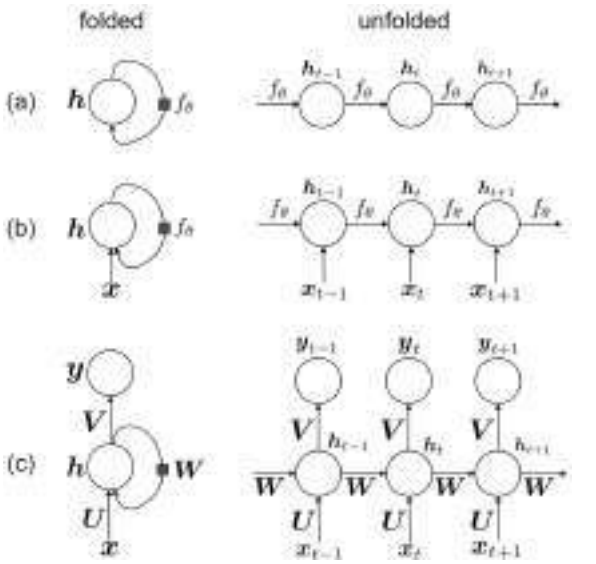


Figure 1. The folded and unfolded structures of (a) a dynamic system without input, (b) a dynamical system with input, and (c) an RNN. Every square on an edge means connection from one time slot before.

$$\mathbb{R}^p \ni \boldsymbol{i}_t = \boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i, \tag{3}$$

$$[-1,1]^p \ni \boldsymbol{h}_t = \tanh(\boldsymbol{i}_t) = \tanh(\boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i), \tag{4}$$

$$\mathbb{R}^q \ni \boldsymbol{y}_t = \boldsymbol{V}\boldsymbol{h}_t + \boldsymbol{b}_y, \tag{5}$$

where $\tanh(.) \in (-1, 1)$ denotes the hyperbolic tangent function, which is used as an element-wise activation function for the states.

If there is an activation function, such as softmax, at the output layer, we denote the output of activation function by:

$$\mathbb{R}^q \ni \widehat{\boldsymbol{y}}_t = \text{softmax}(\boldsymbol{y}_t) = \frac{\exp(y_{t,1})}{\sum_{j=1}^{q} \exp(y_{t,j})}, \tag{6}$$

where $y_{t,j}$ denotes the j-th component of $\boldsymbol{y}_t$.

2.3. Backpropagation Through Time (BPTT)

One of the methods for training RNN is Backpropagation Through Time (BPTT), which is very similar to the backpropagation algorithm (Rumelhart et al., 1986) because it is based on gradient descent and chain rule (Ghojogh et al., 2021), but it has also chain rule through time. BPTT was developed by several works (Robinson & Fallside, 1987; Werbos, 1988; Williams, 1989; Williams & Zipser, 1995; Mozer, 1995). This algorithm is very solid in theory; however, it does not show the best performance in practice.

In BPTT, the loss is considered as a summation of loss functions at the previous time steps until now. As it is im-

practical to consider all time steps from the start of training (especially after a long time of training), we only consider the T previous time steps. In other words, we assume that RNN has T-order Markov property (Ghojogh et al., 2019b). Therefore, the loss function is:

$$\mathbb{R} \ni \mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t, \tag{7}$$

where $\mathcal{L}_1$ is the loss function at the current time slot and $\mathcal{L}_t$ denotes the loss function at the previous $(t-1)$ time slot.

This loss functions needs to be optimized using gradient descent and chain rule. Therefore, we calculate its gradient with respect to the parameters of RNN. These parameters are $\boldsymbol{y}_t$, $\boldsymbol{h}_t$, $\boldsymbol{V}$, $\boldsymbol{W}$, $\boldsymbol{U}$, $\boldsymbol{b}_i$, and $\boldsymbol{b}_y$, based on Eqs. (3), (4), and (5) and Fig. 1-c.

2.3.1. Gradient With Respect to the Output

If there is no activation function at the last layer, the gradient of the loss function of RNN with respect to the output at time t is:

$$\mathbb{R}^q \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \overset{(a)}{=} \frac{\partial \mathcal{L}}{\partial \mathcal{L}_t} \times \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t} \overset{(7)}{=} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t}, \tag{8}$$

where (a) is because of the chain rule.

The gradient of the loss function at time t with respect to the output at time t, i.e., $\partial \mathcal{L}_t / \partial \boldsymbol{y}_t$, is calculated based on the formula of the loss function. The loss function can be any loss function for classification, regression, or other tasks.

If there is an activation function at the last layer (see Eq. (6)), the gradient is:

$$\mathbb{R}^q \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \overset{(a)}{=} \frac{\partial \mathcal{L}}{\partial \mathcal{L}_t} \times \frac{\partial \mathcal{L}_t}{\partial \widehat{\boldsymbol{y}}_t} \times \frac{\partial \widehat{\boldsymbol{y}}_t}{\partial \boldsymbol{y}_t} \overset{(7)}{=} \frac{\partial \mathcal{L}_t}{\partial \widehat{\boldsymbol{y}}_t} \times \frac{\partial \widehat{\boldsymbol{y}}_t}{\partial \boldsymbol{y}_t}, \tag{9}$$

where (a) is because of the chain rule. The derivative $\partial \widehat{\boldsymbol{y}}_t / \partial \boldsymbol{y}_t$ is calculated based on the formula of the activation function. The other derivative, $\partial \mathcal{L}_t / \partial \widehat{\boldsymbol{y}}_t$, is calculated based on the formula of the loss as a function of the output of the activation function.

2.3.2. Gradient With Respect to the State

The gradient of the loss function of RNN with respect to the state at time t is:

$$\begin{aligned}\mathbb{R}^p \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t} &\overset{(a)}{=} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \frac{\partial \boldsymbol{y}_t}{\partial \boldsymbol{h}_t}\Big) + \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_{t+1}} \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}\Big) \\ &\overset{(5)}{=} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \boldsymbol{V}\Big) + \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_{t+1}} \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}\Big),\end{aligned} \tag{10}$$

where (a) is because changing $\boldsymbol{h}_t$ affects both $\boldsymbol{y}_t$ and $\boldsymbol{h}_{t+1}$. We denote $\boldsymbol{\delta}_t := \partial \mathcal{L} / \partial \boldsymbol{h}_t$ so Eq. (10) becomes:

$$\boldsymbol{\delta}_t = \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \boldsymbol{V}\Big) + \Big(\boldsymbol{\delta}_{t+1} \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}\Big). \tag{11}$$

According to Eqs. (3) and (4), we have:

$$\boldsymbol{i}_{t+1} = \boldsymbol{W}\boldsymbol{h}_t + \boldsymbol{U}\boldsymbol{x}_{t+1} + \boldsymbol{b}_i, \tag{12}$$

$$\boldsymbol{h}_{t+1} = \tanh(\boldsymbol{i}_{t+1}) = \tanh(\boldsymbol{W}\boldsymbol{h}_t + \boldsymbol{U}\boldsymbol{x}_{t+1} + \boldsymbol{b}_i). \tag{13}$$

Therefore:

$$\begin{aligned}\mathbb{R}^{p \times p} \ni \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t} &\overset{(a)}{=} \big(\frac{\partial \boldsymbol{i}_{t+1}}{\partial \boldsymbol{h}_t}\big)^\top \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{i}_{t+1}} \\ &\overset{(b)}{=} \boldsymbol{W}(1 - \boldsymbol{h}_{t+1}^\top \boldsymbol{h}_{t+1})\boldsymbol{I}_{p \times p},\end{aligned} \tag{14}$$

where $\boldsymbol{I}_{p \times p}$ is the identity matrix of size $(p \times p)$, (a) is because of the chain rule, and (b) is because $\mathbb{R}^{p \times p} \ni \partial \boldsymbol{i}_{t+1} / \partial \boldsymbol{h}_t = \boldsymbol{W}^\top$ for Eq. (12), and we have:

$$\mathbb{R}^{p \times p} \ni \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{i}_{t+1}} = (1 - \boldsymbol{h}_{t+1}^\top \boldsymbol{h}_{t+1})\boldsymbol{I}_{p \times p}, \tag{15}$$

based on Eq. (13) and the formula for derivative of the hyperbolic tangent function, noticing that the state is multidimensional and not a scalar.

For the time slot $t = T$, the derivative $\partial \mathcal{L} / \partial \boldsymbol{h}_T$ is much simpler:

$$\mathbb{R}^p \ni \boldsymbol{\delta}_T = \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_T} \overset{(a)}{=} \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_T} \times \frac{\partial \boldsymbol{y}_T}{\partial \boldsymbol{h}_T} \overset{(5)}{=} \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_T} \overset{(8)}{=} \frac{\partial \mathcal{L}_T}{\partial \boldsymbol{y}_T}, \tag{16}$$

where (a) is because of the chain rule and the derivative $\partial \mathcal{L}_T / \partial \boldsymbol{y}_T$ is computed based on the formula of loss function.

2.3.3. Gradient With Respect to $\boldsymbol{V}$

The gradient of the loss function of RNN with respect to the weight matrix $\boldsymbol{V}$ is:

$$\mathbb{R}^{q \times p} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{V}} \overset{(a)}{=} \sum_{t=1}^{T} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \frac{\partial \boldsymbol{y}_t}{\partial \boldsymbol{V}}\Big) \overset{(b)}{=} \sum_{t=1}^{T} \Big(\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t} \times \boldsymbol{h}_t^\top\Big), \tag{17}$$

where (a) is because $\boldsymbol{V}$ exists in all time slots and changing $\boldsymbol{V}$ affects the loss $\mathcal{L}$ in all time slots. The equation (b) is because of Eqs. (8) and (5). The derivative $\partial \mathcal{L}_t / \partial \boldsymbol{y}_t \in \mathbb{R}^q$ is calculated based on the formula of the loss function.

2.3.4. Gradient With Respect to $\boldsymbol{W}$

The gradient of the loss function of RNN with respect to the weight matrix $\boldsymbol{W}$ is:

$$\mathbb{R}^{p \times p} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}} \overset{(a)}{=} \sum_{t=1}^{T} \textbf{vec}_{p \times p}^{-1}\Big(\big(\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}}\big)^\top \times \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}\Big), \tag{18}$$

where $\textbf{vec}_{p \times p}^{-1}(.)$ de-vectorizes the vector of length p^2 to a matrix of size $(p \times p)$ and (a) is because $\boldsymbol{W}$ exists in all time slots and changing $\boldsymbol{W}$ affects the loss $\mathcal{L}$ in all time

slots. The derivative $\partial\mathcal{L}/\partial\boldsymbol{h}_t \in \mathbb{R}^p$ in Eq. (18) was computed in Section 2.3.2. The derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{W}$ in Eq. (18) is:

$$\mathbb{R}^{p\times p^2} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{W}} = \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} \times \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{W}},$$

because of the chain rule. The first term is:

$$\mathbb{R}^{p\times p} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} = (1 - \boldsymbol{h}_t^\top \boldsymbol{h}_t)\boldsymbol{I}_{p\times p}, \tag{19}$$

according to Eq. (15). Based on the Magnus-Neudecker convention (Ghojogh et al., 2021), the second term is calculated as:

$$\mathbb{R}^{p\times p^2} \ni \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{W}} = \boldsymbol{h}_{t-1}^\top \otimes \boldsymbol{I}_{p\times p},$$

where $\otimes$ denotes the Kronecker product.

2.3.5. Gradient With Respect to $\boldsymbol{U}$

The gradient of the loss function of RNN with respect to the weight matrix $\boldsymbol{U}$ is:

$$\mathbb{R}^{p\times d} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{U}} \overset{(a)}{=} \sum_{t=1}^{T} \textbf{vec}^{-1}_{p\times d}\Big(\big(\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{U}}\big)^\top \times \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}\Big), \tag{20}$$

where (a) is because $\boldsymbol{U}$ exists in all time slots and changing $\boldsymbol{U}$ affects the loss $\mathcal{L}$ in all time slots. The derivative $\partial\mathcal{L}/\partial\boldsymbol{h}_t \in \mathbb{R}^p$ in Eq. (18) was computed in Section 2.3.2. The derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{U}$ in Eq. (18) is:

$$\mathbb{R}^{p\times (pd)} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{U}} = \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} \times \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{U}},$$

because of the chain rule. The first term is already calculated in Eq. (19). Based on the Magnus-Neudecker convention (Ghojogh et al., 2021), the second term is calculated as:

$$\mathbb{R}^{p\times (pd)} \ni \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{U}} = \boldsymbol{x}_t^\top \otimes \boldsymbol{I}_{p\times p}.$$

2.3.6. Gradient With Respect to $\boldsymbol{b}_i$

The gradient of the loss function of RNN with respect to the bias $\boldsymbol{b}_i$ is:

$$\mathbb{R}^{p} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_i} \overset{(a)}{=} \sum_{t=1}^{T} \Big(\big(\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{b}_i}\big)^\top \times \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}\Big), \tag{21}$$

where (a) is because $\boldsymbol{b}_i$ exists in all time slots and changing $\boldsymbol{b}$ affects the loss $\mathcal{L}$ in all time slots. The derivative $\partial\mathcal{L}/\partial\boldsymbol{h}_t$ was already calculated in Section 2.3.2. The derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{b}_i$ is calculated as:

$$\mathbb{R}^{p\times p} \ni \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{b}_i} \overset{(a)}{=} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t} \times \frac{\partial \boldsymbol{i}_t}{\partial \boldsymbol{b}_i} \overset{(3)}{=} \frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{i}_t},$$

where (a) is because of the chain rule and the derivative $\partial\boldsymbol{h}_t/\partial\boldsymbol{i}_t$ was already calculated in Eq. (19).

2.3.7. Gradient With Respect to $\boldsymbol{b}_y$

The gradient of the loss function of RNN with respect to the bias $\boldsymbol{b}_y$ is:

$$\mathbb{R}^{q} \ni \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_y} \overset{(a)}{=} \sum_{t=1}^{T} \Big(\frac{\partial \mathcal{L}}{\partial \boldsymbol{y}_t} \times \frac{\partial \boldsymbol{y}_t}{\partial \boldsymbol{b}_y}\Big) \overset{(b)}{=} \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{y}_t}, \tag{22}$$

where (a) is because $\boldsymbol{b}_y$ exists in all time slots and changing $\boldsymbol{b}_y$ affects the loss $\mathcal{L}$ in all time slots. The equation (b) is because of Eqs. (8) and (5). The derivative $\partial\mathcal{L}_t/\partial\boldsymbol{y}_t \in \mathbb{R}^q$ is calculated based on the formula of the loss function.

2.3.8. Updates by Gradient Descent

BPPT updates the parameters of RNN by gradient descent (Ghojogh et al., 2021) using the calculated gradients:

$$\begin{aligned}
&\boldsymbol{h}_t := \boldsymbol{h}_t - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{h}_t}, \quad \forall t \in \{1, \dots, T\},\\
&\boldsymbol{V} := \boldsymbol{V} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{V}},\\
&\boldsymbol{W} := \boldsymbol{W} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}},\\
&\boldsymbol{U} := \boldsymbol{U} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{U}},\\
&\boldsymbol{b}_i := \boldsymbol{b}_i - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_i},\\
&\boldsymbol{b}_y := \boldsymbol{b}_y - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}_y},
\end{aligned}$$

where $\eta > 0$ is the learning rate and the gradients are calculated by Eqs. (10), (17), (18), (20), (21), and (22).

3. Gradient Vanishing or Explosion in Long-term Dependencies

In recurrent neural networks, so as in deep neural networks, the final output is the composition of a large number of non-linear transformations. This results in the problem of either vanishing or exploding gradients in recurrent neural networks, especially for capturing long-term dependencies in sequence processing (Bengio et al., 1993; 1994). This problem is explained in the following. Recall Eq. (2) for a dynamical system:

$$\boldsymbol{h}_t = f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t).$$

By induction, the hidden state at time t, i.e., $\boldsymbol{h}_t$, can be written as the previous T time steps. If the subscript t denotes the previous t time steps, we have by induction (Hochreiter, 1998; Hochreiter et al., 2001):

$$\boldsymbol{h}_1 = f_\theta\Big(f_\theta\big(\dots f_\theta(\boldsymbol{h}_T, \boldsymbol{x}_{T+1})\dots, \boldsymbol{x}_2\big), \boldsymbol{x}_1\Big).$$

Then, by the chain rule in derivatives, the derivative loss at time T, i.e., $\mathcal{L}_T$, is:

$$\frac{\partial \mathcal{L}_T}{\partial \theta} = \sum_{t \leq T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_t} \frac{\partial \boldsymbol{h}_t}{\partial \theta} \overset{(a)}{=} \sum_{t \leq T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_T} \frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_t} \frac{\partial \boldsymbol{h}_t}{\partial \theta} \overset{(2)}{=} \sum_{t \leq T} \frac{\partial \mathcal{L}_t}{\partial \boldsymbol{h}_T} \frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_t} \frac{\partial f_\theta(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)}{\partial \theta}, \tag{23}$$

where (a) is because of the chain rule. In this expression, there is the derivative of $\boldsymbol{h}_T$ with respect to $\boldsymbol{h}_t$ which itself can be calculated by the chain rule:

$$\frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_t} = \frac{\partial \boldsymbol{h}_T}{\partial \boldsymbol{h}_{T-1}} \times \frac{\partial \boldsymbol{h}_{T-1}}{\partial \boldsymbol{h}_{T-2}} \times \cdots \times \frac{\partial \boldsymbol{h}_{t+1}}{\partial \boldsymbol{h}_t}. \tag{24}$$

for capturing long-term dependencies in the sequence, T should be large. This means that in Eq. (24), and hence in Eq. (23), the number of multiplicand terms becomes huge. On the one hand, if each derivative is slightly smaller than one, the entire derivative in the chain rule becomes very small for multiplication of many terms smaller than one. This problem is referred to as gradient vanishing. On the other hand, if every derivative is slightly larger than one, the entire derivative in the chain rule explodes, resulting in the problem of exploding gradients. Note that gradient vanishing is more common than gradient explosion in recurrent networks.

There exist various attempts for resolving the problem of gradient vanishing or explosion (Hochreiter, 1998; Bengio et al., 2013). In the following, some of these attempts are introduced.

3.1. Close-to-identity Weight Matrix

As Eq. (4) shows, the state is multiplied by a weight matrix $\boldsymbol{W}$ at every time step and if there is long-term dependency, many of these $\boldsymbol{W}$ matrices are multiplied. Suppose the eigenvalue decomposition (Ghojogh et al., 2019a) of the matrix $\boldsymbol{W}$ is $\boldsymbol{W} = \boldsymbol{A}\boldsymbol{\Lambda}\boldsymbol{A}^\top$ where $\boldsymbol{A} \in \mathbb{R}^{p \times p}$ and $\boldsymbol{\Lambda} := \textbf{diag}([\lambda_1, \dots, \lambda_p]^\top)$ contain the eigenvectors and eigenvalues of $\boldsymbol{W}$, respectively. Eq. (4) is restated as:

$$\boldsymbol{h}_t = \tanh(\boldsymbol{A}\boldsymbol{\Lambda}\boldsymbol{A}^\top \boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i). \tag{25}$$

If a change ε in some element of the state $\boldsymbol{h}_{t-1}$ is aligned with an eigenvector of the weight matrix $\boldsymbol{W}$, then the effect of this change in $\boldsymbol{h}_t$ will be $(\lambda^t\, \varepsilon)$ after t time steps, according to Eq. (25). Two cases may happen:

- If the largest eigenvalue is less than one, i.e., $\lambda < 1$, then the change $(\lambda^t\, \varepsilon)$ is contrastive because $\lambda^t \ll 1$ for long-term dependencies. In this case, gradient vanishing occurs in long-term dependency and the network forgets very long time ago.
- If the largest eigenvalue is less than one, i.e., $\lambda > 1$, then the change $(\lambda^t\, \varepsilon)$ is diverging because $\lambda^t \gg 1$ for long-term dependencies. In this case, the gradient network forgets very long time ago. In this case, gradient explosion occurs in long-term dependency and remembering very long time ago dominates the short-term memories.

As remembering short-term memories is usually more important than remembering very past time in different tasks, it is recommended to use the weight matrix $\boldsymbol{W}$ whose largest eigenvalue is less than one; this makes the RNN have the Markovian property because it forgets very past after some point. However, if the largest eigenvalue of $\boldsymbol{W}$ is much less than one, i.e., $\lambda \ll 1$, gradient vanishing happens very sooner than expected. Therefore, it is recommended to use the weight matrix $\boldsymbol{W}$ whose largest eigenvalue is *slightly* less than one, i.e., $\lambda \lesssim 1$. This makes the RNN slightly contrastive. One way to have the weight matrix $\boldsymbol{W}$ whose largest eigenvalue is slightly less than one is to make this matrix close to the identity matrix (Mikolov et al., 2015).

There exist some other ways to determine the weight matrix $\boldsymbol{W}$. For example, the wight matrix can be set to be an orthogonal matrix (Arjovsky et al., 2016). Another approach is to copy the previous state exactly to the current state. In this approach, the Eq. (4) is modified to (Hu et al., 2018):

$$\begin{aligned} \boldsymbol{h}_t &= \tanh(\boldsymbol{W}\boldsymbol{h}_{t-1} + \boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i) \\ &= \tanh\big((\boldsymbol{W} + \boldsymbol{I})\boldsymbol{h}_{t-1} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i\big), \end{aligned} \tag{26}$$

where $\boldsymbol{I}$ is the identity matrix. This prevents gradient vanishing because it brings a copy of the previous step to the current state. This can also be interpreted as strengthening the diagonal of the weight matrix $\boldsymbol{W}$; hence, increasing the largest eigenvalue of $\boldsymbol{W}$ for preventing gradient vanishing.

3.2. Long Delays

As Eq. (4) and Fig. 1-c show, in the regular RNN, every state $\boldsymbol{h}_t$ is fed by its previous state $\boldsymbol{h}_{t-1}$ through the weight matrix $\boldsymbol{W}$. As discussed in Section 3.1, in the regular RNN, the effect of the change ε in a state results in $(\lambda^t\, \varepsilon)$ after t time steps, where λ is the largest eigenvalue of $\boldsymbol{W}$.

As shown in Fig. 1-c, the regular RNN has one-step connections or delays between the states. It is possible to have longer delays between the states in addition to the one-step delays (Lin et al., 1995). In other words, it is possible to have higher levels of Markov property in the network. Let $\boldsymbol{W}_k$ denote the weight matrix for k-step delays between the states. Then, Eq. (4) can be modified to:

$$\boldsymbol{h}_t = \tanh\Big(\sum_k \boldsymbol{W}_k \boldsymbol{h}_{t-k} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i\Big), \tag{27}$$

where the summation is over the k values for the existing

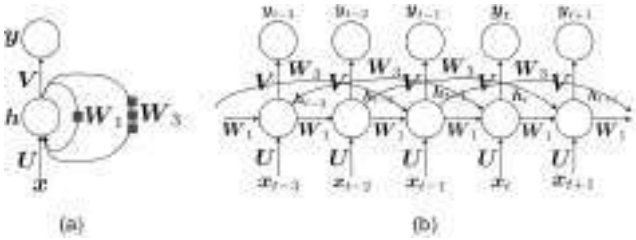


Figure 2. The (a) folded and (b) unfolded structures of an RNN with long delays. Every square on an edge means connection from one time slot before.

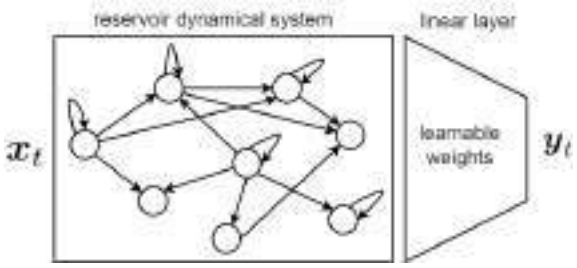


Figure 3. The echo state network.

delays in the RNN structure. An example for an RNN network with one-step and three-step delays is:

$$\boldsymbol{h}_t = \tanh\Big(\boldsymbol{W}_1\boldsymbol{h}_{t-1} + \boldsymbol{W}_3\boldsymbol{h}_{t-3} + \boldsymbol{U}\boldsymbol{x}_t + \boldsymbol{b}_i\Big),$$

which is illustrated in Fig. 2.

Having long delays in RNN is one of the attempts for preventing gradient vanishing (Lin et al., 1995). This is justified because every state is having impact not only from the previous state but also from the more previous states. Therefore, in backpropagation through time, there is some skip in gradient flow from a state to more previous states without the need to go through the middle states in the chain rule.

3.3. Leaky Units

Another way to resolve the problem of gradient vanishing is leaky units (Jaeger et al., 2007; Sutskever & Hinton, 2010). Let $h_{t,j}$ denote the j-th element of the state $\boldsymbol{h}_t \in [-1, 1]^p$. In leaky units, Eq. (4) is modified to the following element-wise equation:

$$\begin{aligned} h_{t,j} = &(1 - \frac{1}{\tau_j})\, h_{t-1,j} \\ &+ \frac{1}{\tau_j}\tanh(\boldsymbol{W}_{j:}\boldsymbol{h}_{t-1} + \boldsymbol{U}_{j:}\boldsymbol{x}_t + b_{i,j}), \end{aligned} \tag{28}$$

where $1 \leq \tau_j < \infty$ and $\boldsymbol{W}_{j:}$ is the j-th row of $\boldsymbol{W}$ and $\boldsymbol{U}_{j:}$ is the j-th row of $\boldsymbol{U}$ and $b_{i,j}$ is the j-th element of $\boldsymbol{b}_i$. When $\tau_i = 1$, then Eq. (28) becomes:

$$h_{t,j} = \frac{1}{\tau_j}\tanh(\boldsymbol{W}_{j:}\boldsymbol{h}_{t-1} + \boldsymbol{U}_{j:}\boldsymbol{x}_t + b_{i,j}),$$

which gives back Eq. (4) in the regular RNN. However, when $\tau_i \gg 1$, then Eq. (28) becomes:

$$h_{t,j} = h_{t-1,j},$$

which means that the previous state is copied to the current state. The larger the τ_i, the easier the gradient propagates for $h_{t,i}$. Therefore, by tuning τ_i, it is possible to control how much of the past should be directly copied and how much should be passed through the weight matrix. This can control the amount of gradient vanishing. Note that leaky units use different τ_i's because there may be a need to keep some of the directions of states (with $\tau_i = 1$) or forget some of the directions (with $\tau_i \gg 1$). In other words, it decides about the p directions of states separately.

3.4. Echo State Networks

One of the approaches to handle the problem of gradient vanishing in RNN is to use echo state networks (Jaeger & Haas, 2004; Jaeger, 2007). These networks consider the recurrent neural network as a black box having hidden units with nonlinear activation functions and connections between them. This black box of recurrent connections is called the *reservoir dynamical system* which models the internal structure of a computer or brain. The connections in the reservoir system are usually sparse and the weights of these connections are considered to be fixed. The output of the reservoir system is connected to an additional linear output layer whose weights are learnable. The echo state network minimizes the mean squared error in the output layer; hence, it performs linear regression in the last layer (Jaeger & Haas, 2004). This network is shown in Fig. 3. Because of not learning the recurrent weights in the reservoir system and sufficing to learn the weights of the output layer, the echo state network does not face the gradient vanishing problem. A tutorial on this topic is (Jaeger, 2002).

3.5. Other methods

There exist some other methods for not having gradient vanishing in recurrent networks. For example, hierarchical RNN (Hihi & Bengio, 1995) and deep RNN (Graves, 2013) have been proposed which stack several recurrent networks. Another related category of networks is the time-delay neural networks (Lang et al., 1990; Peddinti et al., 2015) which are used for shift-invariant sequence processing, especially used in speech recognition (Sugiyama et al., 1991). In these networks, every neuron in a layer receives a contextual window of the neurons in the previous layer as well as their delayed outputs in several time slots ago. Moreover, these networks apply backpropagation on several copies of the network shifted across the sequence

(time) so that the network becomes time-invariant.

4. Long Short-Term Memory Network

As the examples in Section 1 showed, we need short-term relations in some cases and long-term relations in some other cases. RNN learns the sequence based on one or several previous states, depending on its structure and the level of its Markov property (see Fig. 2). Therefore, we need to decide on the structure of RNN to be able to handle short-term or long-term dependencies in the sequence. Instead of manual design of the RNN structure or deciding manually when to clear the state, we can let the neural network learn by itself when to clear the state based on its input sequence. Long Short-Term Memory (LSTM), initially proposed in (Hochreiter & Schmidhuber, 1995; 1997), is able to do this; it learns from its input sequence when to use short-term dependency (i.e., when to clear the state) and when to use the long-term memory (i.e., when not to clear the state).

4.1. LSTM Gates and Cells

LSTM consists of several cells, each of which corresponds to a time slot (see Fig. 4). Every LSTM cell contains several gates for learning different aspects of the input time series (see Fig. 5). These gates are introduced in the following.

4.1.1. The Input Gate

One of the gates in the LSTM cell is the *input gate*, first proposed in (Hochreiter & Schmidhuber, 1995; 1997). This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{i}_t \in [0,1]^p$:

$$\boldsymbol{i}_t = \text{sig}\big(\boldsymbol{W}_i\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_i\, \boldsymbol{x}_t + (\boldsymbol{p}_i \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_i\big), \tag{29}$$

where $\boldsymbol{W}_i \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_i \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_i \in \mathbb{R}^p$ are the learnable weights for the input gate, $\odot$ denotes the Hadamard (element-wise) product, $\boldsymbol{c}_{t-1} \in \mathbb{R}^p$ is the final memory of the last time slot (which will be explained in Section 4.1.5), and $\boldsymbol{p}_i \in \mathbb{R}^p$ is the learnable *peephole weight* (Gers & Schmidhuber, 2000) letting a possible leak of information from the previous final memory. The function $\text{sig}(.) \in (0,1)$ is the sigmoid function which is applied element-wise:

$$\text{sig}(x) = \frac{1}{1+\exp(-x)}. \tag{30}$$

As Eq. (29) demonstrates, the input gate considers the effect of the input and the previous hidden state. It may also use a leak of information from the previous memory through the peephole. This gate carries the importance of the information of the input at the current time slot. The input gate is depicted in Fig. 5.

4.1.2. The Forget Gate

Another gate in the LSTM cell is the *forget gate*, first proposed in (Gers et al., 2000). This gate also takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{f}_t \in [0,1]^p$:

$$\boldsymbol{f}_t = \text{sig}\big(\boldsymbol{W}_f\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_f\, \boldsymbol{x}_t + (\boldsymbol{p}_f \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_f\big), \tag{31}$$

where $\boldsymbol{W}_f \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_f \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_f \in \mathbb{R}^p$ are the learnable weights for the forget gate, and $\boldsymbol{p}_f \in \mathbb{R}^p$ is the learnable *peephole weight* (Gers & Schmidhuber, 2000) letting a possible leak of information from the previous final memory.

As Eq. (31) shows, the forget gate considers the effect of the input and the previous hidden state, and perhaps a leak of information from the previous memory. This gate controls the amount of forgetting the previous information with respect to the new-coming information. the forget gate is illustrated in Fig. 5.

4.1.3. The Output Gate

The next gate in the LSTM cell is the *output gate* first proposed in (Hochreiter & Schmidhuber, 1995; 1997). This gate also takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{o}_t \in [0,1]^p$:

$$\boldsymbol{o}_t = \text{sig}\big(\boldsymbol{W}_o\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_o\, \boldsymbol{x}_t + (\boldsymbol{p}_o \odot \boldsymbol{c}_t) + \boldsymbol{b}_o\big), \tag{32}$$

where $\boldsymbol{W}_o \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_o \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_o \in \mathbb{R}^p$ are the learnable weights for the output gate, and $\boldsymbol{p}_o \in \mathbb{R}^p$ is the learnable *peephole weight* (Gers & Schmidhuber, 2000) letting a possible leak of information from the current final memory.

As shown in Eq. (32), the output gate considers the effect of the input and the previous hidden state, and a possible information leak from the current memory. The putput gate is shown in Fig. 5.

4.1.4. The New Memory Cell (Block Input)

The LSTM cell includes a gate named the *new memory cell*. This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\widetilde{\boldsymbol{c}}_t \in [-1,1]^p$. This gate considers the effect of the input and the previous hidden state to represent the new information of current input. It is formulated as:

$$\widetilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{W}_c\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_c\, \boldsymbol{x}_t + \boldsymbol{b}_c), \tag{33}$$

where $\boldsymbol{W}_c \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_c \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_c$ are the learnable weights for the new memory cell. The new memory cell is also referred to as the *block input* in the literature (Greff et al., 2016). The signal $\widetilde{\boldsymbol{c}}_t$ is sometimes denoted by

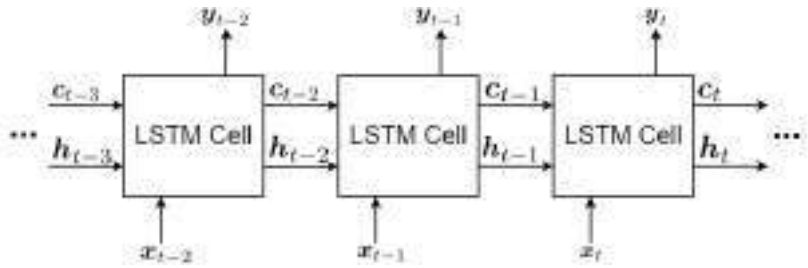


Figure 4. A sequence of LSTM cells processing the input sequence.

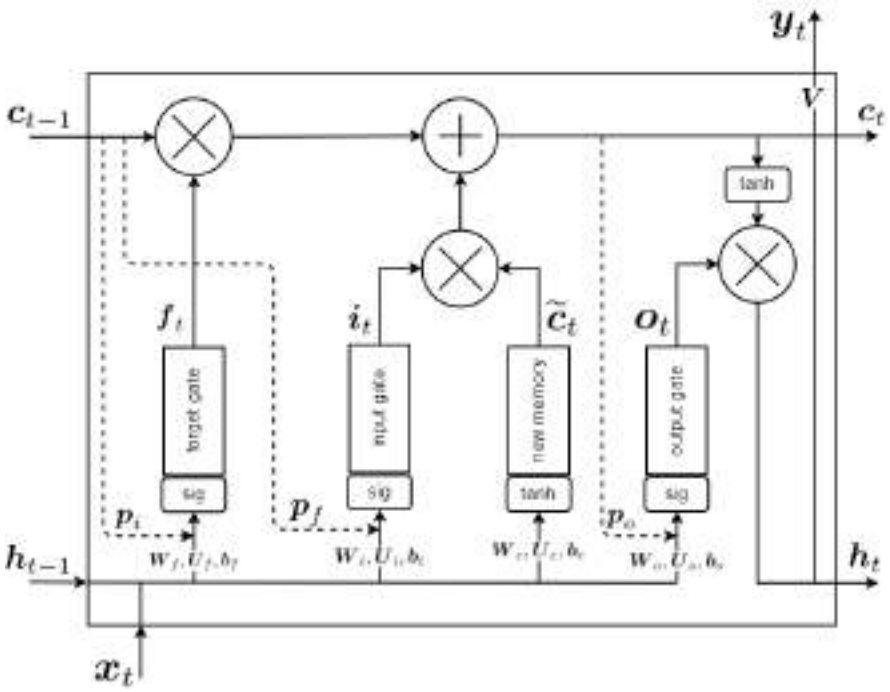


Figure 5. The conveyor belt in the LSTM cell.

$\boldsymbol{z}_t$ in the literature. The new memory cell is illustrated in Fig. 5.

4.1.5. The Final Memory Calculation

After computation of the outputs of the input gate $\boldsymbol{i}_t$, the forget gate $\boldsymbol{f}_t$, and the new memory cell $\widetilde{\boldsymbol{c}}_t$, we calculate the *final memory* $\boldsymbol{c}_t \in \mathbb{R}^p$:

$$\boldsymbol{c}_t = (\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}) + (\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t), \tag{34}$$

where $\boldsymbol{c}_{t-1} \in \mathbb{R}^p$ is the final memory of the previous time slot.

As Eq. (34) demonstrates, the final memory considers the effect of the forget gate, the previous memory, the input, and the new memory. In the first term, i.e., $\boldsymbol{f}_t \odot \boldsymbol{c}_{t-1}$, the forget gate $\boldsymbol{f}_t \in [0,1]^p$ controls how much of the previous memory $\boldsymbol{c}_{t-1}$ should be forgotten. The closer the $\boldsymbol{f}_t$ is to zero, the more the network forgets the previous memory $\boldsymbol{c}_{t-1}$. In the second term, i.e., $\boldsymbol{i}_t \odot \widetilde{\boldsymbol{c}}_t$, the input gate $\boldsymbol{i}_t \in [0,1]^p$ and the new memory cell $\widetilde{\boldsymbol{c}}_t \in [-1,1]^p$ both control how much of the new input information should be used. The closer the input gate $\boldsymbol{i}_t$ is to one and the closer the new memory cell $\widetilde{\boldsymbol{c}}_t$ is to ± 1, the more the input information is used.

In other words, the first and second terms in Eq. (34) determine the trade-off of usage of old versus new information in the sequence. The weights of these gates are trained in a way that they pass or block the input/previous information based on the input sequence and the time step in the sequence. The final memory calculation is depicted in Fig. 5.

4.1.6. The Hidden State (Block Output)

After computation of the output of the output gate $\boldsymbol{o}_t$ and the final memory $\boldsymbol{c}_t$, we calculate the *hidden state* $\boldsymbol{h}_t \in$

$[-1, 1]^p$:

$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t). \tag{35}$$

This hidden state is also considered as the *block output* of the LSTM cell, depicted in Fig. 5.

4.1.7. The Output

The output $\boldsymbol{y}_t \in \mathbb{R}^q$ of the LSTM cell is as follows:

$$\boldsymbol{y}_t = \boldsymbol{V}\boldsymbol{h}_t + \boldsymbol{b}_y, \tag{36}$$

where $\boldsymbol{V} \in \mathbb{R}^{q \times p}$ and the bias $\boldsymbol{b}_y \in \mathbb{R}^q$ are the learnable weights for the output. It is possible to use an activation function, such as Eq. (6), after this output signal. Figure 5 shows the output signal in the LSTM cell.

Note that in the literature, the output is sometimes considered to be equal to the hidden state, i.e., $\boldsymbol{y}_t = \boldsymbol{h}_t$, by setting $\boldsymbol{V} = \boldsymbol{I}$ (the identity matrix) and $\boldsymbol{b}_y = \boldsymbol{0}$ (the zero vector).

4.2. History and Variants of LSTM

LSTM has gone through various developments and improvements gradually (Greff et al., 2016). Some of the variants of LSTM do not have the peepholes. In this case, the Eqs. (29), (31), and (32) are simplified to:

$$\boldsymbol{i}_t = \text{sig}\big(\boldsymbol{W}_i\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_i\, \boldsymbol{x}_t + \boldsymbol{b}_i\big), \tag{37}$$

$$\boldsymbol{f}_t = \text{sig}\big(\boldsymbol{W}_f\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_f\, \boldsymbol{x}_t + \boldsymbol{b}_f\big), \tag{38}$$

$$\boldsymbol{o}_t = \text{sig}\big(\boldsymbol{W}_o\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_o\, \boldsymbol{x}_t + \boldsymbol{b}_o\big), \tag{39}$$

respectively. In the following, we review a history of variants of the LSTM networks.

4.2.1. Original LSTM

LSTM was originally proposed by Hochreiter and Schmidhuber in years 1995 to 1997 (Hochreiter & Schmidhuber, 1995; 1997). We call it the *original LSTM* (Hochreiter & Schmidhuber, 1997). The original LSTM had only the input and output gates, introduced in Sections 4.1.1 and 4.1.3, and it did not have a forget gate. It also did not contain the peepholes; therefore, its gates were Eqs. (37) and (39). The original LSTM trained the network using BPTT (introduced in Section 2.3) and a mixture of real-time recurrent learning (Robinson & Fallside, 1987; Williams, 1989).

4.2.2. Vanilla LSTM

Later, Gers *et. al.* (Gers et al., 2000; Gers & Schmidhuber, 2000) applied some changes to the original LSTM. The forget gate, introduced in Section 4.1.2, was proposed for the first time in (Gers et al., 2000) to let the network forget its previous states either completely or partially. The peephole connections, introduced in Sections 4.1.1, 4.1.2, and 4.1.3, were first proposed in (Gers & Schmidhuber, 2000). The peepholes let a possible leak of information from the previous or current final memory. This lets the memory control the gates.

These two papers (Gers et al., 2000; Gers & Schmidhuber, 2000) also incorporated the *full gate recurrence*, in which all gates receive additional recurrent inputs from all gates at the previous time step. In full gate recurrence, the Eqs. (29), (31), and (32) become:

$$\begin{aligned}\boldsymbol{i}_t = \text{sig}\big(&\boldsymbol{W}_i\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_i\, \boldsymbol{x}_t + (\boldsymbol{p}_i \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_i \\ &+ \boldsymbol{R}_{ii}\, \boldsymbol{i}_{t-1} + \boldsymbol{R}_{if}\, \boldsymbol{f}_{t-1} + \boldsymbol{R}_{io}\, \boldsymbol{o}_{t-1}\big).\end{aligned} \tag{40}$$

$$\begin{aligned}\boldsymbol{f}_t = \text{sig}\big(&\boldsymbol{W}_f\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_f\, \boldsymbol{x}_t + (\boldsymbol{p}_f \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_f \\ &+ \boldsymbol{R}_{fi}\, \boldsymbol{i}_{t-1} + \boldsymbol{R}_{ff}\, \boldsymbol{f}_{t-1} + \boldsymbol{R}_{fo}\, \boldsymbol{o}_{t-1}\big).\end{aligned} \tag{41}$$

$$\begin{aligned}\boldsymbol{o}_t = \text{sig}\big(&\boldsymbol{W}_o\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_o\, \boldsymbol{x}_t + (\boldsymbol{p}_o \odot \boldsymbol{c}_t) + \boldsymbol{b}_o \\ &+ \boldsymbol{R}_{oi}\, \boldsymbol{i}_{t-1} + \boldsymbol{R}_{of}\, \boldsymbol{f}_{t-1} + \boldsymbol{R}_{oo}\, \boldsymbol{o}_{t-1}\big),\end{aligned} \tag{42}$$

where $\boldsymbol{R}_{ii}, \boldsymbol{R}_{if}, \boldsymbol{R}_{io}, \boldsymbol{R}_{fi}, \boldsymbol{R}_{ff}, \boldsymbol{R}_{fo}, \boldsymbol{R}_{oi}, \boldsymbol{R}_{of}, \boldsymbol{R}_{oo} \in \mathbb{R}^{p \times p}$ are the learnable recurrent weights. Note that the full gate recurrence often disappeared in later papers on LSTM.

Later, Graves and Schmidhuber adapted the original LSTM and proposed the *vanilla LSTM* in 2005 (Graves & Schmidhuber, 2005a), which is one of the most common LSTMs in the literature. The vanilla LSTM incorporated the structures of the original LSTM (Hochreiter & Schmidhuber, 1997) and the papers (Gers et al., 2000; Gers & Schmidhuber, 2000). The full BPTT, introduced in Section 2.3, was used for LSTM in the vanilla LSTM (Graves & Schmidhuber, 2005a).

4.2.3. Other LSTM Variants

There are other variants of LSTM (Greff et al., 2016; Jozefowicz et al., 2015); although, the most common used LSTM is the vanilla LSTM (Graves & Schmidhuber, 2005a). BPTT was used for LSTM training in (Graves & Schmidhuber, 2005a); however, Kalman filtering was used for its training (Gers et al., 2002) before that. Another training method for LSTM was evolutionary learning (Schmidhuber et al., 2007). Context-sensitive evolutionary learning was also used for LSTM training (Bayer et al., 2009).

Later works tried to improve the performance of LSTM. For example, Sak *et. al.* added a linear layer which projects the output of the LSTM to smaller number of parameters before the recurrent connections (Sak et al., 2014). Doetsch *et. al.* converted the slope of activation functions of the LSTM gates to learnable parameters for performance improvement (Doetsch et al., 2014). *Dynamic cortex memory* was another LSTM variant which added connections between the gates within every LSTM cell (Otte et al., 2014). Finally in 2014, one the biggest improvements of LSTM was proposed, which was named the Gated Recurrent Units (GRU) (Cho et al., 2014). The philosophy of GRU was to simplify the LSTM cell because we may no need to have a very complicated cell to learn the sequence information. In

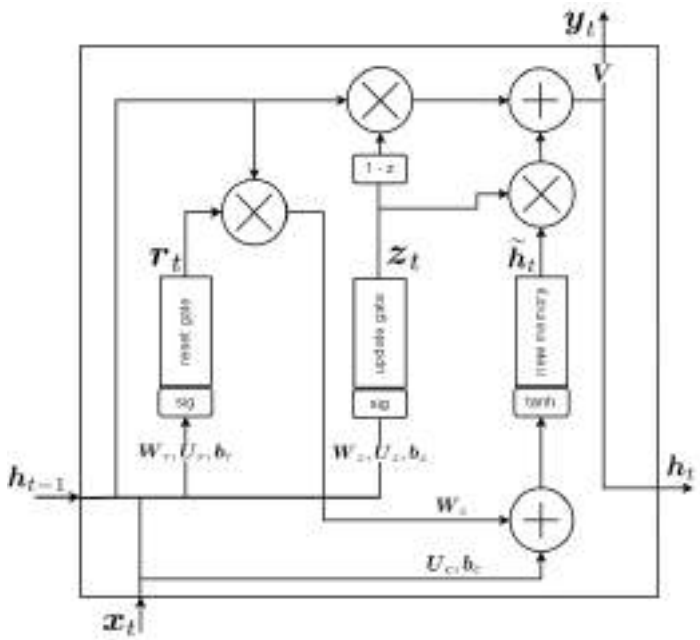


Figure 6. The conveyor belt in the GRU cell (the fully gated unit).

other words, GRU raised the question of whether we need to be that flexible like LSTM to learn the sequence. GRU is less flexible than LSTM but it is good enough for sequence learning.

GRU redesigned the LSTM cell by introducing reset gate, update gate, and new memory cell; therefore, the number of gates were reduced from four to three. It was empirically shown in (Chung et al., 2014) that the performance of LSTM improves by using GRU cells. Later in 2017, the GRU was further simplified by merging the reset and update gates into a forget gate (Heck & Salem, 2017). Nowadays, GRU is the most commonly used LSTM structure. Section 4.3 introduces the details of the GRU cell.

4.3. Gated Recurrent Units (GRU)

GRU was first proposed in (Cho et al., 2014). As was mentioned in Section 4.2.3, GRU simplified the LSTM cell in order to make the flexibility of LSTM.

4.3.1. Fully Gated Unit

The main GRU was the *fully gated unit* (Cho et al., 2014), whose gates are introduced in the following. Its gates are illustrated in Fig. 6.

– The Reset Gate: One of the gates in the GRU cell is the *reset gate*. This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1, 1]^p$, and outputs the signal $\boldsymbol{r}_t \in [0, 1]^p$:

$$\boldsymbol{r}_t = \text{sig}(\boldsymbol{W}_r\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_r\, \boldsymbol{x}_t + \boldsymbol{b}_r), \tag{43}$$

where $\boldsymbol{W}_r \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_r \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_r \in \mathbb{R}^p$ are the learnable weights for the reset gate. The reset gate considers the effect of the input and the previous hidden state, and it controls the amount of forgetting/resetting the previous information with respect to the new-coming information. Comparing Eqs. (38) and (43) shows that the reset gate in the GRU cell is similar to the forget gate in the LSTM cell. The reset gate is depicted in Fig. 6.

– The Update Gate: Another gate in the GRU cell is the *update gate*. This gate also takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1, 1]^p$, and outputs the signal $\boldsymbol{z}_t \in [0, 1]^p$:

$$\boldsymbol{z}_t = \text{sig}(\boldsymbol{W}_z\, \boldsymbol{h}_{t-1} + \boldsymbol{U}_z\, \boldsymbol{x}_t + \boldsymbol{b}_z), \tag{44}$$

where $\boldsymbol{W}_z \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_z \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_z \in \mathbb{R}^p$ are the learnable weights for the update gate. The update gate considers the effect of the input and the previous hidden state, and it controls the amount of using the new input data for updating the cell by the coming information of sequence. Comparing Eqs. (37) and (44) shows that the update gate in the GRU cell is similar to the input gate in the LSTM cell. Figure 6 shows the update gate in the GRU cell.

– The New Memory Cell: The GRU cell includes a gate named the *new memory cell*. This gate takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1, 1]^p$, and outputs the signal

$\widetilde{\boldsymbol{h}}_t \in [-1,1]^p$:

$$\widetilde{\boldsymbol{h}}_t = \tanh\Big(\boldsymbol{W}_c\,\big(\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}\big)\Big) + \boldsymbol{U}_c\,\boldsymbol{x}_t + \boldsymbol{b}_c\Big), \quad (45)$$

where $\boldsymbol{W}_c \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_c \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_c$ are the learnable weights for the new memory cell. This gate considers the effect of the input and the previous hidden state to represent the new information of current input. Comparing Eqs. (33) and (45) shows that the new memory cell in the GRU cell is similar to the new memory cell in the LSTM cell. Note that, in the LSTM cell, the hidden state (see Eq. (35)) and the new memory cell (see Eq. (33)) were different; however, the hidden state of the GRU cell (see Eq. (45)) replaces the new memory signal in the LSTM cell. The new memory cell is shown in Fig. 6.

– The Final Memory (Hidden State): After computation of the outputs of the update gate $\boldsymbol{z}_t$ and the new memory cell $\widetilde{\boldsymbol{h}}_t$, we calculate the *final memory* or the *hidden state* $\boldsymbol{h}_t \in \mathbb{R}^p$:

$$\boldsymbol{h}_t = \big((\boldsymbol{1} - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1}\big) + (\boldsymbol{z}_t \odot \widetilde{\boldsymbol{h}}_t), \quad (46)$$

where $\boldsymbol{h}_{t-1} \in \mathbb{R}^p$ is the hidden state of the previous time slot. The final memory block is illustrated in Fig. 6.

As Eq. (46) demonstrates, the final memory considers the effect of the update gate, the previous memory, and the new memory. In the first term, i.e., $(\boldsymbol{1} - \boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1}$, the update gate $\boldsymbol{z}_t \in [0,1]^p$ controls how much of the previous state $\boldsymbol{h}_{t-1}$ should be used based on the input data. The closer the $\boldsymbol{z}_t$ is to one (resp. zero), the more the network forgets (resp. considers) the previous state $\boldsymbol{h}_{t-1}$. In the second term, i.e., $\boldsymbol{z}_t \odot \widetilde{\boldsymbol{h}}_t$, the update gate $\boldsymbol{z}_t \in [0,1]^p$ and the new memory cell $\widetilde{\boldsymbol{h}}_t \in [-1,1]^p$ both control how much of the new input information should be used. In other words, it controls how much the information should be updated by the new information. The closer the update gate $\boldsymbol{z}_t$ is to one and the closer the new memory cell $\widetilde{\boldsymbol{h}}_t$ is to ± 1, the more the input information is used.

Overall, the first and second terms in Eq. (46) determine the trade-off of usage of old versus new information in the sequence. The weights of these gates are trained in a way that they pass or block the input/previous information based on the input sequence and the time step in the sequence. Comparing Eqs. (34) and (46) shows that the final memory in the GRU cell is in the form of the final memory in the LSTM cell; however, they have somewhat different functionality.

4.3.2. Minimal Gated Unit

Minimal gated unit (Heck & Salem, 2017) is another variant of GRU which has simplified the gate by merging the reset and update gates into a forget gate. This merging is possible because the forget gate can control both the previous and new information of the sequence.

– The Forget Gate: The *forget gate* takes the input at the current time slot, $\boldsymbol{x}_t \in \mathbb{R}^d$, and the hidden state of the last time slot, $\boldsymbol{h}_{t-1} \in [-1,1]^p$, and outputs the signal $\boldsymbol{r}_t \in [0,1]^p$:

$$\boldsymbol{f}_t = \text{sig}(\boldsymbol{W}_f\,\boldsymbol{h}_{t-1} + \boldsymbol{U}_f\,\boldsymbol{x}_t + \boldsymbol{b}_f), \quad (47)$$

where $\boldsymbol{W}_f \in \mathbb{R}^{p\times p}$, $\boldsymbol{U}_f \in \mathbb{R}^{p\times d}$, and the bias $\boldsymbol{b}_f \in \mathbb{R}^p$ are the learnable weights for the forget gate. The forget gate considers the effect of the input and the previous hidden state, and it controls the amount of forgetting the previous information with respect to the new-coming information. Therefore, it controls both forgetting or using the previous memory and using the new coming information.

– The New Memory Cell and the Final Memory: Because the forget gate replaces the reset and the update gate in the minimal gate unit, Eqs. (45) and (46) are changed to:

$$\widetilde{\boldsymbol{h}}_t = \tanh\Big(\boldsymbol{W}_c\,\big(\boldsymbol{f}_t \odot \boldsymbol{h}_{t-1}\big)\Big) + \boldsymbol{U}_c\,\boldsymbol{x}_t + \boldsymbol{b}_c\Big), \quad (48)$$

$$\boldsymbol{h}_t = \big((\boldsymbol{1} - \boldsymbol{f}_t) \odot \boldsymbol{h}_{t-1}\big) + (\boldsymbol{f}_t \odot \widetilde{\boldsymbol{h}}_t), \quad (49)$$

respectively, to be the new memory cell and the final memory in the minimal gate unit.

5. Bidirectional RNN and LSTM

5.1. Justification of Bidirectional Processing

A bidirectional RNN or LSTM network processes the sequence in both directions; left to right and right to left. In the first glance, online causal tasks such as reading a text or listening to a speech do not have access to the future. Therefore, bidirectional networks seem to violate causality in them. However, in many of these tasks, it is possible to wait for the completion of a part of the sequence such as a sentence and then decide about it. For example, it is normal to wait for the completion of sentence in speech recognition and then recognize it (Graves & Schmidhuber, 2005a;b). In text processing, the text is usually available except in a streaming text. Even in streaming text, it is possible to wait for a sentence to complete. Therefore, it makes sense to use bidirectional networks for processing sequences because, sometimes, the important related word comes after a word and not necessarily before it. An example for such a case is the sentence "The police is chasing the thief" where the word "thief" is a strongly related (opposite) word for the word "police". In this sentence, both the words "thief" and "police" are related and it is worth to process the sentence in both directions.

5.2. Bidirectional RNN

The bidirectional RNN was first proposed in (Schuster & Paliwal, 1997) and further exploited in (Baldi et al., 1999). It uses two sets of states each for one of the directions in

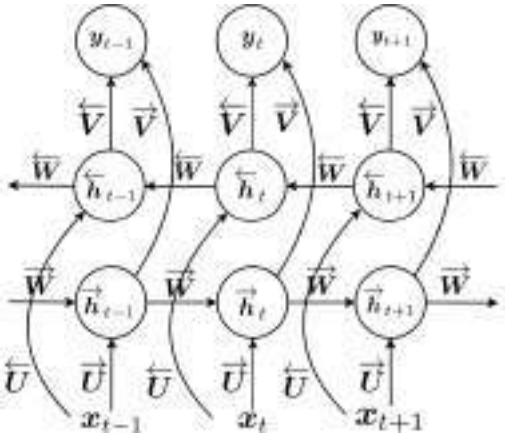


Figure 7. The bidirectional RNN.

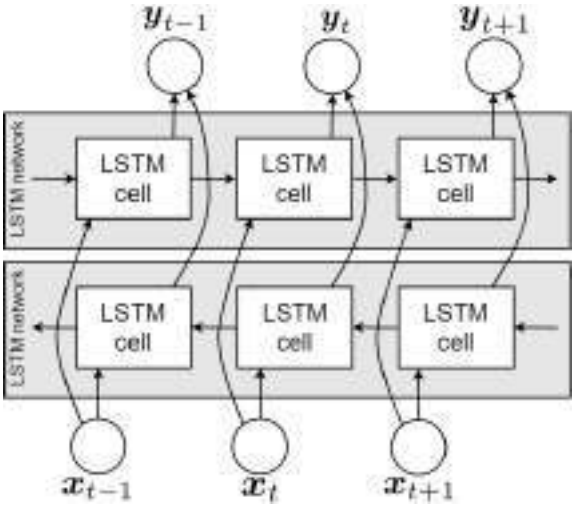


Figure 8. The bidirectional LSTM.

the sequence. Let the states for left-to-right and right-to-left processing be denoted by $\overrightarrow{\boldsymbol{h}}_t$ and $\overleftarrow{\boldsymbol{h}}_t$, respectively. In the bidirectional RNN, Eq. (4) is replaced by two equations (Graves et al., 2013):

$$\overrightarrow{\boldsymbol{h}}_t = \tanh(\overrightarrow{\boldsymbol{W}}\overrightarrow{\boldsymbol{h}}_{t-1} + \overrightarrow{\boldsymbol{U}}\boldsymbol{x}_t + \overrightarrow{\boldsymbol{b}}_i), \tag{50}$$

$$\overleftarrow{\boldsymbol{h}}_t = \tanh(\overleftarrow{\boldsymbol{W}}\overleftarrow{\boldsymbol{h}}_{t+1} + \overleftarrow{\boldsymbol{U}}\boldsymbol{x}_t + \overleftarrow{\boldsymbol{b}}_i), \tag{51}$$

and Eq. (5) is replaced by:

$$\boldsymbol{y}_t = \overrightarrow{\boldsymbol{V}}\overrightarrow{\boldsymbol{h}}_t + \overleftarrow{\boldsymbol{V}}\overleftarrow{\boldsymbol{h}}_t + \boldsymbol{b}_y, \tag{52}$$

where the arrows show the parameters for each direction of processing. The unfolding schematic of the bidirectional RNN is illustrated in Fig. 7. As this figure shows, the outputs of both directions are connected to an output layer. In some cases, this output layer may be replaced by a third multi-layer neural network. All weights of the bidirectional RNN are trained using backpropagation through time similarly to what was explained in Section 2.3. It is noteworthy that the deep variant of bidirectional RNN has been proposed in (Graves et al., 2013).

5.3. Bidirectional LSTM

Bidirectional LSTM was first proposed in (Graves & Schmidhuber, 2005a;b). As obvious from its name, the bidirectional LSTM includes two LSTM networks each of which processes the sequence from one direction. In other words, there are two LSTM networks which are fed with the sequence in opposite orders. This structure is depicted in Fig. 8. Experiments have shown that the bidirectional LSTM outperforms the unidirectional LSTM (Graves & Schmidhuber, 2005a; Graves et al., 2005). This is expected according to the justification in Section 5.1. Because of its advantages, various methods have combined the bidirectional LSTM with other methods. For example, a hybrid of the bidirectional LSTM and HMM (Ghojogh et al., 2019b) has shown its merit (Graves et al., 2005). Another example is the hybrid of bidirectional LSTM and Conditional Random Fields (CRF) (Lafferty et al., 2001) for the sequence tagging task (Huang et al., 2015). Other extensions of bidirectional LSTM networks exist such as (Peters et al., 2017).

5.4. Embeddings from Language Model (ELMo)

The Embeddings from Language Model (ELMo) network, first proposed in (Peters et al., 2018), is a language model which makes use of bidirectional LSTM networks. It is one of the successful context-aware language modeling networks. It has been widely used in various applications such as in medical interview processing (Sarzynska-Wawer et al., 2021).

The structure of ELMo is illustrated in Fig. 9. As shown in this figure, ELMo contains L layers of bidirectional LSTM networks where the output of each bidirectional LSTM is fed to the next bidirectional LSTM. in The bidirectional LSTM networks of ELMo, $\boldsymbol{V} = \boldsymbol{I}$ is set so that the outputs $\boldsymbol{y}$ becomes equal to the hidden states $\boldsymbol{h}$. At time slot t and layer l, the outputs (or hidden states) of the two directions of LSTM are concatenated together to make $\boldsymbol{h}_t^{(l)}$:

$$\boldsymbol{h}_t^{(l)} := [\overrightarrow{\boldsymbol{h}}_t^{(l)\top}, \overleftarrow{\boldsymbol{h}}_t^{(l)\top}]^\top.$$

Then, a linear combination of these hidden states of layers is considered to be the embedding vector of ELMo network at time t (Peters et al., 2018):

$$\boldsymbol{y}_t^{\text{ELMo}} := \gamma \sum_{l=1}^{L} s_l\, \boldsymbol{h}_t^{(l)}, \tag{53}$$

where γ and $\{s_l\}_{l=1}^L$ are the hyperparameter scalar weights which are determined according to the specific task (e.g.,

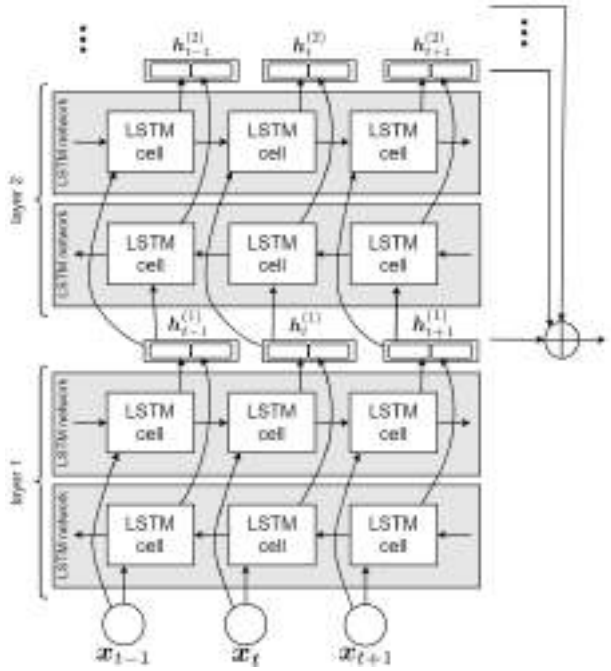


Figure 9. The ELMo network.

question answering, translation, etc) in natural language processing.

6. Conclusion

This was a tutorial paper on RNN, LSTM, and its variants. We covered dynamical system, backpropagation through time, LSTM gates and cells, history and variants of LSTM, the GRU cell, bidirectional RNN, bidirectional LSTM, and ELMo network.

References

Arjovsky, Martin, Shah, Amar, and Bengio, Yoshua. Unitary evolution recurrent neural networks. In *International conference on machine learning*, pp. 1120–1128. PMLR, 2016.

Baldi, Pierre, Brunak, Søren, Frasconi, Paolo, Soda, Giovanni, and Pollastri, Gianluca. Exploiting the past and the future in protein secondary structure prediction. *Bioinformatics*, 15(11):937–946, 1999.

Bayer, Justin, Wierstra, Daan, Togelius, Julian, and Schmidhuber, Jürgen. Evolving memory cell structures for sequence learning. In *International conference on artificial neural networks*, pp. 755–764. Springer, 2009.

Bengio, Yoshua, Frasconi, Paolo, and Simard, Patrice. The problem of learning long-term dependencies in recurrent networks. In *IEEE international conference on neural networks*, pp. 1183–1188. IEEE, 1993.

Bengio, Yoshua, Simard, Patrice, and Frasconi, Paolo. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2): 157–166, 1994.

Bengio, Yoshua, Boulanger-Lewandowski, Nicolas, and Pascanu, Razvan. Advances in optimizing recurrent networks. In *2013 IEEE international conference on acoustics, speech and signal processing*, pp. 8624–8628. IEEE, 2013.

Broer, Hendrik Wolter and Takens, Floris. *Dynamical systems and chaos*, volume 172. Springer, 2011.

Cho, Kyunghyun, Van Merriënboer, Bart, Bahdanau, Dzmitry, and Bengio, Yoshua. On the properties of neural machine translation: Encoder-decoder approaches. In *Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation (SSST-8)*, 2014.

Chung, Junyoung, Gulcehre, Caglar, Cho, KyungHyun, and Bengio, Yoshua. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.

Doetsch, Patrick, Kozielski, Michal, and Ney, Hermann. Fast and robust training of recurrent neural networks for offline handwriting recognition. In *2014 14th international conference on frontiers in handwriting recognition*, pp. 279–284. IEEE, 2014.

Gers, Felix A and Schmidhuber, Jürgen. Recurrent nets that time and count. In *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium*, volume 3, pp. 189–194. IEEE, 2000.

Gers, Felix A, Schmidhuber, Jürgen, and Cummins, Fred. Learning to forget: Continual prediction with LSTM. *Neural computation*, 12(10):2451–2471, 2000.

Gers, Felix A, Pérez-Ortiz, Juan Antonio, Eck, Douglas, and Schmidhuber, Jürgen. DEKF-LSTM. In *10th European Symposium on Artificial Neural Networks (ESANN)*, 2002.

Ghojogh, Benyamin, Karray, Fakhri, and Crowley, Mark. Eigenvalue and generalized eigenvalue problems: Tutorial. *arXiv preprint arXiv:1903.11240*, 2019a.

Ghojogh, Benyamin, Karray, Fakhri, and Crowley, Mark. Hidden Markov model: Tutorial. *Engineering Archive*, 2019b.

Ghojogh, Benyamin, Ghodsi, Ali, Karray, Fakhri, and Crowley, Mark. KKT conditions, first-order and second-order optimization, and distributed optimization: Tutorial and survey. *arXiv preprint arXiv:2110.01858*, 2021.

Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling

Junyoung Chung **Caglar Gulcehre** **KyungHyun Cho** **Yoshua Bengio**
Université de Montréal
Université de Montréal
CIFAR Senior Fellow

Abstract

In this paper we compare different types of recurrent units in recurrent neural networks (RNNs). Especially, we focus on more sophisticated units that implement a gating mechanism, such as a long short-term memory (LSTM) unit and a recently proposed gated recurrent unit (GRU). We evaluate these recurrent units on the tasks of polyphonic music modeling and speech signal modeling. Our experiments revealed that these advanced recurrent units are indeed better than more traditional recurrent units such as tanh units. Also, we found GRU to be comparable to LSTM.

1 Introduction

Recurrent neural networks have recently shown promising results in many machine learning tasks, especially when input and/or output are of variable length [see, e.g., Graves, 2012]. More recently, Sutskever et al. [2014] and Bahdanau et al. [2014] reported that recurrent neural networks are able to perform as well as the existing, well-developed systems on a challenging task of machine translation.

One interesting observation we make from these recent successes is that almost none of these successes were achieved with a vanilla recurrent neural network. Rather, it was a recurrent neural network with sophisticated recurrent hidden units, such as long short-term memory units [Hochreiter and Schmidhuber, 1997], that was used in those successful applications.

Among those sophisticated recurrent units, in this paper, we are interested in evaluating two closely related variants. One is a long short-term memory (LSTM) unit, and the other is a gated recurrent unit (GRU) proposed more recently by Cho et al. [2014]. It is well established in the field that the LSTM unit works well on sequence-based tasks with long-term dependencies, but the latter has only recently been introduced and used in the context of machine translation.

In this paper, we evaluate these two units and a more traditional tanh unit on the task of sequence modeling. We consider three polyphonic music datasets [see, e.g., Boulanger-Lewandowski et al., 2012] as well as two internal datasets provided by Ubisoft in which each sample is a raw speech representation.

Based on our experiments, we concluded that by using fixed number of parameters for all models on some datasets GRU, can outperform LSTM units both in terms of convergence in CPU time and in terms of parameter updates and generalization.

2 Background: Recurrent Neural Network

A recurrent neural network (RNN) is an extension of a conventional feedforward neural network, which is able to handle a variable-length sequence input. The RNN handles the variable-length

sequence by having a recurrent hidden state whose activation at each time is dependent on that of the previous time.

More formally, given a sequence $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_T)$, the RNN updates its recurrent hidden state h_t by

$$\mathbf{h}_t = \begin{cases} 0, & t = 0 \\ \phi\left(\mathbf{h}_{t-1}, \mathbf{x}_t\right), & \text{otherwise} \end{cases} \tag{1}$$

where ϕ is a nonlinear function such as composition of a logistic sigmoid with an affine transformation. Optionally, the RNN may have an output $\mathbf{y} = (y_1, y_2, \ldots, y_T)$ which may again be of variable length.

Traditionally, the update of the recurrent hidden state in Eq. (1) is implemented as

$$\mathbf{h}_t = g\left(W\mathbf{x}_t + U\mathbf{h}_{t-1}\right), \tag{2}$$

where g is a smooth, bounded function such as a logistic sigmoid function or a hyperbolic tangent function.

A generative RNN outputs a probability distribution over the next element of the sequence, given its current state $\mathbf{h}_t$, and this generative model can capture a distribution over sequences of variable length by using a special output symbol to represent the end of the sequence. The sequence probability can be decomposed into

$$p(x_1, \ldots, x_T) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_T \mid x_1, \ldots, x_{T-1}), \tag{3}$$

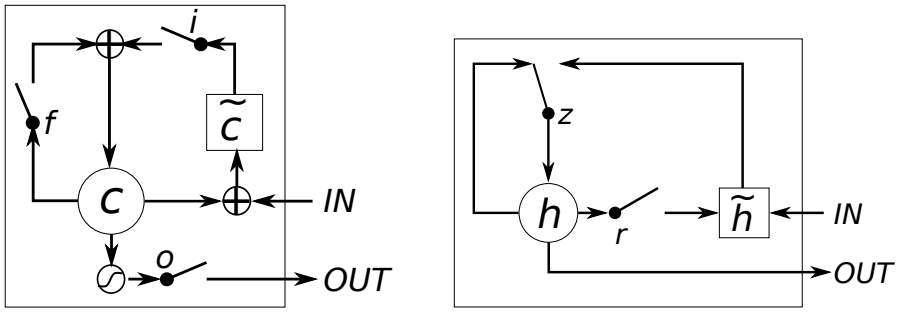
where the last element is a special end-of-sequence value. We model each conditional probability distribution with

$$p(x_t \mid x_1, \ldots, x_{t-1}) = g(h_t),$$

where h_t is from Eq. (1). Such generative RNNs are the subject of this paper.

Unfortunately, it has been observed by, e.g., Bengio et al. [1994] that it is difficult to train RNNs to capture long-term dependencies because the gradients tend to either vanish (most of the time) or explode (rarely, but with severe effects). This makes gradient-based optimization method struggle, not just because of the variations in gradient magnitudes but because of the effect of long-term dependencies is hidden (being exponentially smaller with respect to sequence length) by the effect of short-term dependencies. There have been two dominant approaches by which many researchers have tried to reduce the negative impacts of this issue. One such approach is to devise a better learning algorithm than a simple stochastic gradient descent [see, e.g., Bengio et al., 2013, Pascanu et al., 2013, Martens and Sutskever, 2011], for example using the very simple *clipped gradient*, by which the norm of the gradient vector is clipped, or using second-order methods which may be less sensitive to the issue if the second derivatives follow the same growth pattern as the first derivatives (which is not guaranteed to be the case).

The other approach, in which we are more interested in this paper, is to design a more sophisticated activation function than a usual activation function, consisting of affine transformation followed by a simple element-wise nonlinearity by using gating units. The earliest attempt in this direction resulted in an activation function, or a recurrent unit, called a long short-term memory (LSTM) unit [Hochreiter and Schmidhuber, 1997]. More recently, another type of recurrent unit, to which we refer as a gated recurrent unit (GRU), was proposed by Cho et al. [2014]. RNNs employing either of these recurrent units have been shown to perform well in tasks that require capturing long-term dependencies. Those tasks include, but are not limited to, speech recognition [see, e.g., Graves et al., 2013] and machine translation [see, e.g., Sutskever et al., 2014, Bahdanau et al., 2014].



(a) Long Short-Term Memory (b) Gated Recurrent Unit

Figure 1: Illustration of (a) LSTM and (b) gated recurrent units. (a) i, f and o are the input, forget and output gates, respectively. c and $\tilde{c}$ denote the memory cell and the new memory cell content. (b) r and z are the reset and update gates, and h and $\tilde{h}$ are the activation and the candidate activation.

3 Gated Recurrent Neural Networks

In this paper, we are interested in evaluating the performance of those recently proposed recurrent units (LSTM unit and GRU) on sequence modeling. Before the empirical evaluation, we first describe each of those recurrent units in this section.

3.1 Long Short-Term Memory Unit

The Long Short-Term Memory (LSTM) unit was initially proposed by Hochreiter and Schmidhuber [1997]. Since then, a number of minor modifications to the original LSTM unit have been made. We follow the implementation of LSTM as used in Graves [2013].

Unlike to the recurrent unit which simply computes a weighted sum of the input signal and applies a nonlinear function, each j-th LSTM unit maintains a memory c_t^j at time t. The output h_t^j, or the activation, of the LSTM unit is then

$$h_t^j = o_t^j \tanh\left(c_t^j\right),$$

where o_t^j is an *output gate* that modulates the amount of memory content exposure. The output gate is computed by

$$o_t^j = \sigma\left(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + V_o \mathbf{c}_t\right)^j,$$

where σ is a logistic sigmoid function. V_o is a diagonal matrix.

The memory cell c_t^j is updated by partially forgetting the existing memory and adding a new memory content $\tilde{c}_t^j$:

$$c_t^j = f_t^j c_{t-1}^j + i_t^j \tilde{c}_t^j, \tag{4}$$

where the new memory content is

$$\tilde{c}_t^j = \tanh\left(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1}\right)^j.$$

The extent to which the existing memory is forgotten is modulated by a *forget gate* f_t^j, and the degree to which the new memory content is added to the memory cell is modulated by an *input gate* i_t^j. Gates are computed by

$$\begin{aligned} f_t^j =& \sigma\left(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + V_f \mathbf{c}_{t-1}\right)^j, \\ i_t^j =& \sigma\left(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + V_i \mathbf{c}_{t-1}\right)^j. \end{aligned}$$

Note that V_f and V_i are diagonal matrices.

Unlike to the traditional recurrent unit which overwrites its content at each time-step (see Eq. (2)), an LSTM unit is able to decide whether to keep the existing memory via the introduced gates. Intuitively, if the LSTM unit detects an important feature from an input sequence at early stage, it easily carries this information (the existence of the feature) over a long distance, hence, capturing potential long-distance dependencies.

See Fig. 1 (a) for the graphical illustration.

3.2 Gated Recurrent Unit

A gated recurrent unit (GRU) was proposed by Cho et al. [2014] to make each recurrent unit to adaptively capture dependencies of different time scales. Similarly to the LSTM unit, the GRU has gating units that modulate the flow of information inside the unit, however, without having a separate memory cells.

The activation h_t^j of the GRU at time t is a linear interpolation between the previous activation h_{t-1}^j and the candidate activation $\tilde{h}_t^j$:

$$h_t^j = (1 - z_t^j)h_{t-1}^j + z_t^j \tilde{h}_t^j, \tag{5}$$

where an *update gate* z_t^j decides how much the unit updates its activation, or content. The update gate is computed by

$$z_t^j = \sigma \left(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} \right)^j .$$

This procedure of taking a linear sum between the existing state and the newly computed state is similar to the LSTM unit. The GRU, however, does not have any mechanism to control the degree to which its state is exposed, but exposes the whole state each time.

The candidate activation $\tilde{h}_t^j$ is computed similarly to that of the traditional recurrent unit (see Eq. (2)) and as in [Bahdanau et al., 2014],

$$\tilde{h}_t^j = \tanh \left(W \mathbf{x}_t + U \left(\mathbf{r}_t \odot \mathbf{h}_{t-1} \right) \right)^j ,$$

where $\mathbf{r}_t$ is a set of reset gates and $\odot$ is an element-wise multiplication. [1] When off (r_t^j close to 0), the reset gate effectively makes the unit act as if it is reading the first symbol of an input sequence, allowing it to *forget* the previously computed state.

The reset gate r_t^j is computed similarly to the update gate:

$$r_t^j = \sigma \left(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} \right)^j .$$

See Fig. 1 (b) for the graphical illustration of the GRU.

3.3 Discussion

It is easy to notice similarities between the LSTM unit and the GRU from Fig. 1.

The most prominent feature shared between these units is the additive component of their update from t to $t + 1$, which is lacking in the traditional recurrent unit. The traditional recurrent unit always replaces the activation, or the content of a unit with a new value computed from the current input and the previous hidden state. On the other hand, both LSTM unit and GRU keep the existing content and add the new content on top of it (see Eqs. (4) and (5)).

[1] Note that we use the reset gate in a slightly different way from the original GRU proposed in Cho et al. [2014]. Originally, the candidate activation was computed by

$$\tilde{h}_t^j = \tanh \left(W \mathbf{x}_t + \mathbf{r}_t \odot \left(U \mathbf{h}_{t-1} \right) \right)^j ,$$

where r_t^j is a *reset gate*. We found in our preliminary experiments that both of these formulations performed as well as each other.

This additive nature has two advantages. First, it is easy for each unit to remember the existence of a specific feature in the input stream for a long series of steps. Any important feature, decided by either the forget gate of the LSTM unit or the update gate of the GRU, will not be overwritten but be maintained as it is.

Second, and perhaps more importantly, this addition effectively creates shortcut paths that bypass multiple temporal steps. These shortcuts allow the error to be back-propagated easily without too quickly vanishing (if the gating unit is nearly saturated at 1) as a result of passing through multiple, bounded nonlinearities, thus reducing the difficulty due to vanishing gradients [Hochreiter, 1991, Bengio et al., 1994].

These two units however have a number of differences as well. One feature of the LSTM unit that is missing from the GRU is the controlled exposure of the memory content. In the LSTM unit, the amount of the memory content that is seen, or used by other units in the network is controlled by the output gate. On the other hand the GRU exposes its full content without any control.

Another difference is in the location of the input gate, or the corresponding reset gate. The LSTM unit computes the new memory content without any separate control of the amount of information flowing from the previous time step. Rather, the LSTM unit controls the amount of the new memory content being added to the memory cell *independently* from the forget gate. On the other hand, the GRU controls the information flow from the previous activation when computing the new, candidate activation, but does not independently control the amount of the candidate activation being added (the control is tied via the update gate).

From these similarities and differences alone, it is difficult to conclude which types of gating units would perform better in general. Although Bahdanau et al. [2014] reported that these two units performed comparably to each other according to their preliminary experiments on machine translation, it is unclear whether this applies as well to tasks other than machine translation. This motivates us to conduct more thorough empirical comparison between the LSTM unit and the GRU in this paper.

4 Experiments Setting

4.1 Tasks and Datasets

We compare the LSTM unit, GRU and $\tanh$ unit in the task of sequence modeling. Sequence modeling aims at learning a probability distribution over sequences, as in Eq. (3), by maximizing the log-likelihood of a model given a set of training sequences:

$$\max_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} \log p\left(x_t^n \mid x_1^n, \dots, x_{t-1}^n; \boldsymbol{\theta}\right),$$

where $\boldsymbol{\theta}$ is a set of model parameters. More specifically, we evaluate these units in the tasks of polyphonic music modeling and speech signal modeling.

For the polyphonic music modeling, we use three polyphonic music datasets from [Boulanger-Lewandowski et al., 2012]: Nottingham, JSB Chorales, MuseData and Piano-midi. These datasets contain sequences of which each symbol is respectively a 93-, 96-, 105-, and 108-dimensional binary vector. We use logistic sigmoid function as output units.

We use two internal datasets provided by Ubisoft[2] for speech signal modeling. Each sequence is an one-dimensional raw audio signal, and at each time step, we design a recurrent neural network to look at 20 consecutive samples to predict the following 10 consecutive samples. We have used two different versions of the dataset: One with sequences of length 500 (Ubisoft A) and the other with sequences of length $8,000$ (Ubisoft B). Ubisoft A and Ubisoft B have $7,230$ and 800 sequences each. We use mixture of Gaussians with 20 components as output layer. [3]

[2] http://www.ubi.com/

[3] Our implementation is available at https://github.com/jych/librnn.git

4.2 Models

For each task, we train three different recurrent neural networks, each having either LSTM units (LSTM-RNN, see Sec. 3.1), GRUs (GRU-RNN, see Sec. 3.2) or $\tanh$ units ($\tanh$-RNN, see Eq. (2)). As the primary objective of these experiments is to compare all three units fairly, we choose the size of each model so that each model has approximately the same number of parameters. We intentionally made the models to be small enough in order to avoid overfitting which can easily distract the comparison. This approach of comparing different types of hidden units in neural networks has been done before, for instance, by Gulcehre et al. [2014]. See Table 1 for the details of the model sizes.

Unit	# of Units	# of Parameters
Polyphonic music modeling		
LSTM	36	$\approx 19.8 \times 10^3$
GRU	46	$\approx 20.2 \times 10^3$
$\tanh$	100	$\approx 20.1 \times 10^3$
Speech signal modeling		
LSTM	195	$\approx 169.1 \times 10^3$
GRU	227	$\approx 168.9 \times 10^3$
$\tanh$	400	$\approx 168.4 \times 10^3$

Table 1: The sizes of the models tested in the experiments.

			$\tanh$	GRU	LSTM
Music Datasets	Nottingham	train	3.22	2.79	3.08
		test	**3.13**	3.23	3.20
	JSB Chorales	train	8.82	6.94	8.15
		test	9.10	**8.54**	8.67
	MuseData	train	5.64	5.06	5.18
		test	6.23	**5.99**	6.23
	Piano-midi	train	5.64	4.93	6.49
		test	9.03	**8.82**	9.03
Ubisoft Datasets	Ubisoft dataset A	train	6.29	2.31	1.44
		test	6.44	3.59	**2.70**
	Ubisoft dataset B	train	7.61	0.38	0.80
		test	7.62	**0.88**	1.26

Table 2: The average negative log-probabilities of the training and test sets.

We train each model with RMSProp [see, e.g., Hinton, 2012] and use weight noise with standard deviation fixed to 0.075 [Graves, 2011]. At every update, we rescale the norm of the gradient to 1, if it is larger than 1 [Pascanu et al., 2013] to prevent exploding gradients. We select a learning rate (scalar multiplier in RMSProp) to maximize the validation performance, out of 10 randomly chosen log-uniform candidates sampled from $\mathcal{U}(-12, -6)$ [Bergstra and Bengio, 2012]. The validation set is used for early-stop training as well.

5 Results and Analysis

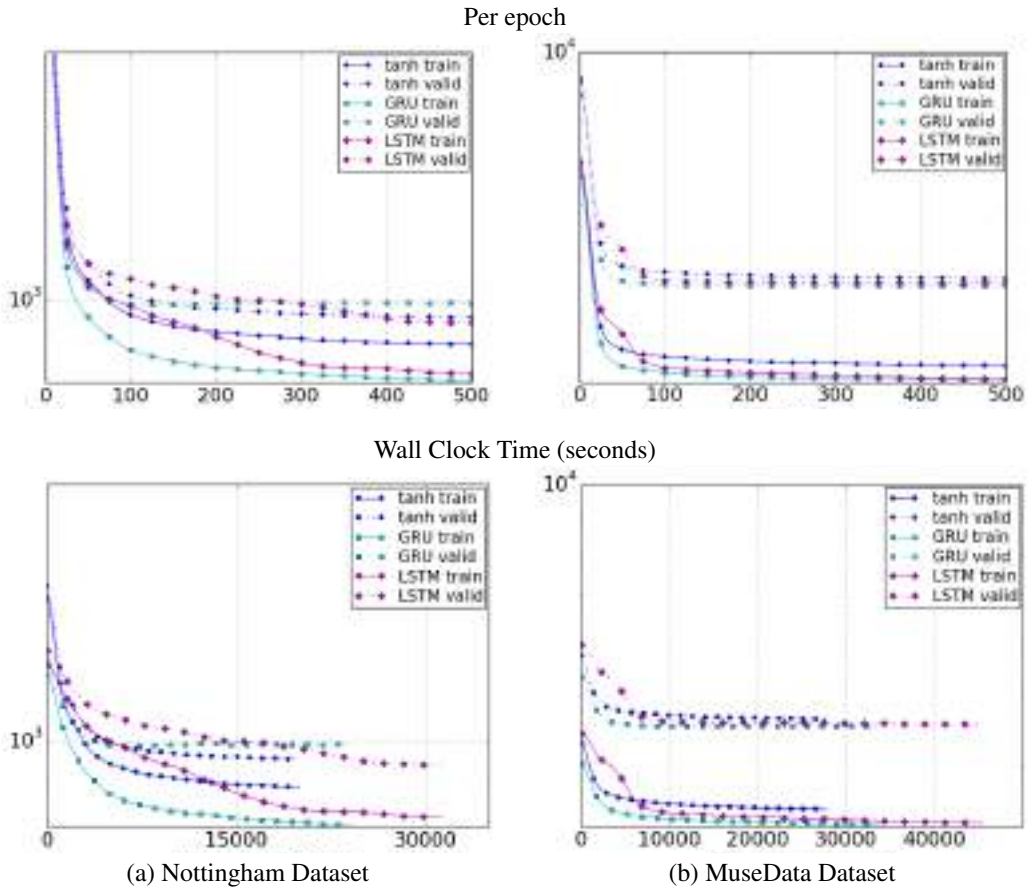
Table 2 lists all the results from our experiments. In the case of the polyphonic music datasets, the GRU-RNN outperformed all the others (LSTM-RNN and $\tanh$-RNN) on all the datasets except for the Nottingham. However, we can see that on these music datasets, all the three models performed closely to each other.

On the other hand, the RNNs with the gating units (GRU-RNN and LSTM-RNN) clearly outperformed the more traditional $\tanh$-RNN on both of the Ubisoft datasets. The LSTM-RNN was best with the Ubisoft A, and with the Ubisoft B, the GRU-RNN performed best.

In Figs. 2–3, we show the learning curves of the best validation runs. In the case of the music datasets (Fig. 2), we see that the GRU-RNN makes faster progress in terms of both the number of

updates and actual CPU time. If we consider the Ubisoft datasets (Fig. 3), it is clear that although the computational requirement for each update in the tanh-RNN is much smaller than the other models, it did not make much progress each update and eventually stopped making any progress at much worse level.

These results clearly indicate the advantages of the gating units over the more traditional recurrent units. Convergence is often faster, and the final solutions tend to be better. However, our results are not conclusive in comparing the LSTM and the GRU, which suggests that the choice of the type of gated recurrent unit may depend heavily on the dataset and corresponding task.



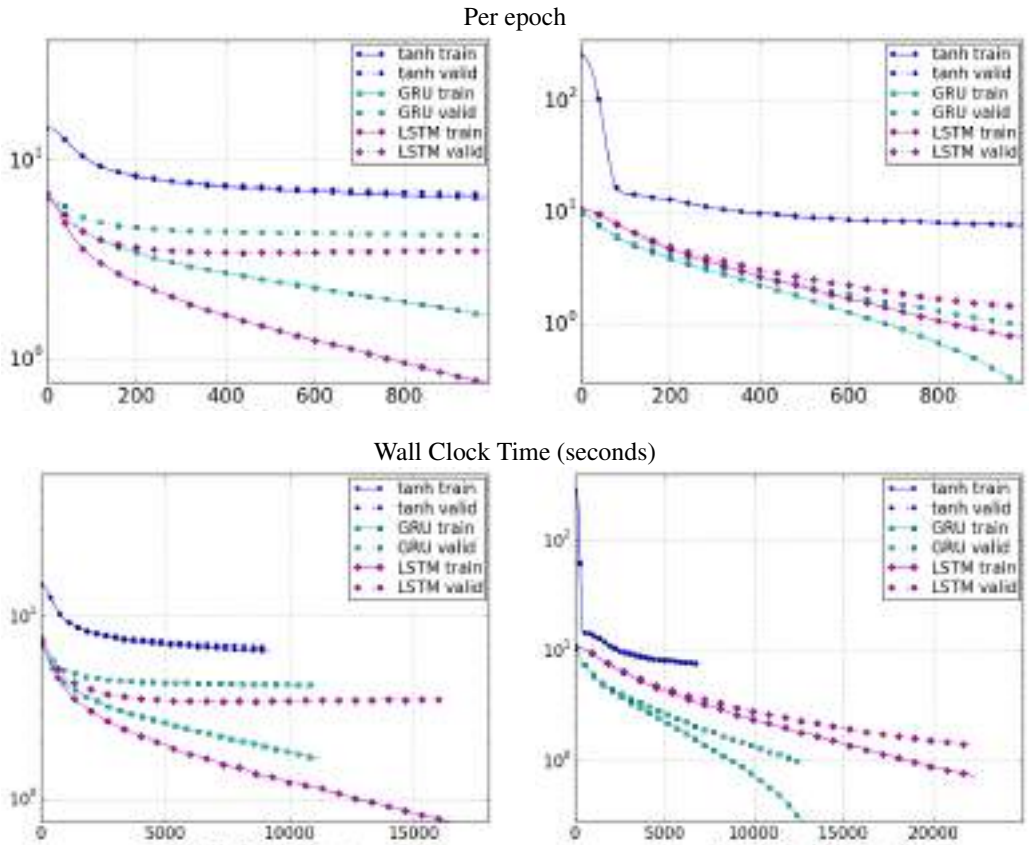
(a) Nottingham Dataset (b) MuseData Dataset

Figure 2: Learning curves for training and validation sets of different types of units with respect to (top) the number of iterations and (bottom) the wall clock time. y-axis corresponds to the negative-log likelihood of the model shown in log-scale.

6 Conclusion

In this paper we empirically evaluated recurrent neural networks (RNN) with three widely used recurrent units; (1) a traditional tanh unit, (2) a long short-term memory (LSTM) unit and (3) a recently proposed gated recurrent unit (GRU). Our evaluation focused on the task of sequence modeling on a number of datasets including polyphonic music data and raw speech signal data.

The evaluation clearly demonstrated the superiority of the gated units; both the LSTM unit and GRU, over the traditional tanh unit. This was more evident with the more challenging task of raw speech signal modeling. However, we could not make concrete conclusion on which of the two gating units was better.



(a) Ubisoft Dataset A (b) Ubisoft Dataset B

Figure 3: Learning curves for training and validation sets of different types of units with respect to (top) the number of iterations and (bottom) the wall clock time. x-axis is the number of epochs and y-axis corresponds to the negative-log likelihood of the model shown in log-scale.

We consider the experiments in this paper as preliminary. In order to understand better how a gated unit helps learning and to separate out the contribution of each component, for instance gating units in the LSTM unit or the GRU, of the gating units, more thorough experiments will be required in the future.

Acknowledgments

The authors would like to thank Ubisoft for providing the datasets and for the support. The authors would like to thank the developers of Theano [Bergstra et al., 2010, Bastien et al., 2012] and Pylearn2 [Goodfellow et al., 2013]. We acknowledge the support of the following agencies for research funding and computing support: NSERC, Calcul Québec, Compute Canada, the Canada Research Chairs and CIFAR.

A Critical Review of Recurrent Neural Networks for Sequence Learning

Zachary C. Lipton
zlipton@cs.ucsd.edu

John Berkowitz
jaberkow@physics.ucsd.edu

Charles Elkan
elkan@cs.ucsd.edu

June 5th, 2015

Abstract

Countless learning tasks require dealing with sequential data. Image captioning, speech synthesis, and music generation all require that a model produce outputs that are sequences. In other domains, such as time series prediction, video analysis, and musical information retrieval, a model must learn from inputs that are sequences. Interactive tasks, such as translating natural language, engaging in dialogue, and controlling a robot, often demand both capabilities. Recurrent neural networks (RNNs) are connectionist models that capture the dynamics of sequences via cycles in the network of nodes. Unlike standard feedforward neural networks, recurrent networks retain a state that can represent information from an arbitrarily long context window. Although recurrent neural networks have traditionally been difficult to train, and often contain millions of parameters, recent advances in network architectures, optimization techniques, and parallel computation have enabled successful large-scale learning with them. In recent years, systems based on long short-term memory (LSTM) and bidirectional (BRNN) architectures have demonstrated ground-breaking performance on tasks as varied as image captioning, language translation, and handwriting recognition. In this survey, we review and synthesize the research that over the past three decades first yielded and then made practical these powerful learning models. When appropriate, we reconcile conflicting notation and nomenclature. Our goal is to provide a self-contained explication of the state of the art together with a historical perspective and references to primary research.

1 Introduction

Neural networks are powerful learning models that achieve state-of-the-art results in a wide range of supervised and unsupervised machine learning tasks.

They are suited especially well for machine perception tasks, where the raw underlying features are not individually interpretable. This success is attributed to their ability to learn hierarchical representations, unlike traditional methods that rely upon hand-engineered features [Farabet et al., 2013]. Over the past several years, storage has become more affordable, datasets have grown far larger, and the field of parallel computing has advanced considerably. In the setting of large datasets, simple linear models tend to under-fit, and often under-utilize computing resources. Deep learning methods, in particular those based on deep belief networks (DNNs), which are greedily built by stacking restricted Boltzmann machines, and convolutional neural networks, which exploit the local dependency of visual information, have demonstrated record-setting results on many important applications.

However, despite their power, standard neural networks have limitations. Most notably, they rely on the assumption of independence among the training and test examples. After each example (data point) is processed, the entire state of the network is lost. If each example is generated independently, this presents no problem. But if data points are related in time or space, this is unacceptable. Frames from video, snippets of audio, and words pulled from sentences, represent settings where the independence assumption fails. Additionally, standard networks generally rely on examples being vectors of fixed length. Thus it is desirable to extend these powerful learning tools to model data with temporal or sequential structure and varying length inputs and outputs, especially in the many domains where neural networks are already the state of the art. Recurrent neural networks (RNNs) are connectionist models with the ability to selectively pass information across sequence steps, while processing sequential data one element at a time. Thus they can model input and/or output consisting of sequences of elements that are not independent. Further, recurrent neural networks can simultaneously model sequential and time dependencies on multiple scales.

In the following subsections, we explain the fundamental reasons why recurrent neural networks are worth investigating. To be clear, we are motivated by a desire to achieve empirical results. This motivation warrants clarification because recurrent networks have roots in both cognitive modeling and supervised machine learning. Owing to this difference of perspectives, many published papers have different aims and priorities. In many foundational papers, generally published in cognitive science and computational neuroscience journals, such as [Hopfield, 1982, Jordan, 1986, Elman, 1990], biologically plausible mechanisms are emphasized. In other papers [Schuster and Paliwal, 1997, Socher et al., 2014, Karpathy and Fei-Fei, 2014], biological inspiration is downplayed in favor of achieving empirical results on important tasks and datasets. This review is motivated by practical results rather than biological plausibility, but where appropriate, we draw connections to relevant concepts in neuroscience. Given the empirical aim, we now address three significant questions that one might reasonably want answered before reading further.

1.1 Why model sequentiality explicitly?

In light of the practical success and economic value of sequence-agnostic models, this is a fair question. Support vector machines, logistic regression, and feedforward networks have proved immensely useful without explicitly modeling time. Arguably, it is precisely the assumption of independence that has led to much recent progress in machine learning. Further, many models implicitly capture time by concatenating each input with some number of its immediate predecessors and successors, presenting the machine learning model with a sliding window of context about each point of interest. This approach has been used with deep belief nets for speech modeling by Maas et al. [2012].

Unfortunately, despite the usefulness of the independence assumption, it precludes modeling long-range dependencies. For example, a model trained using a finite-length context window of length 5 could never be trained to answer the simple question, *"what was the data point seen six time steps ago?"* For a practical application such as call center automation, such a limited system might learn to route calls, but could never participate with complete success in an extended dialogue. Since the earliest conception of artificial intelligence, researchers have sought to build systems that interact with humans in time. In Alan Turing's groundbreaking paper *Computing Machinery and Intelligence*, he proposes an "imitation game" which judges a machine's intelligence by its ability to convincingly engage in dialogue [Turing, 1950]. Besides dialogue systems, modern interactive systems of economic importance include self-driving cars and robotic surgery, among others. Without an explicit model of sequentiality or time, it seems unlikely that any combination of classifiers or regressors can be cobbled together to provide this functionality.

1.2 Why not use Markov models?

Recurrent neural networks are not the only models capable of representing time dependencies. Markov chains, which model transitions between states in an observed sequence, were first described by the mathematician Andrey Markov in 1906. Hidden Markov models (HMMs), which model an observed sequence as probabilistically dependent upon a sequence of unobserved states, were described in the 1950s and have been widely studied since the 1960s [Stratonovich, 1960]. However, traditional Markov model approaches are limited because their states must be drawn from a modestly sized discrete state space S. The dynamic programming algorithm that is used to perform efficient inference with hidden Markov models scales in time $O(|S|^2)$ [Viterbi, 1967]. Further, the transition table capturing the probability of moving between any two time-adjacent states is of size $|S|^2$. Thus, standard operations become infeasible with an HMM when the set of possible hidden states grows large. Further, each hidden state can depend only on the immediately previous state. While it is possible to extend a Markov model to account for a larger context window by creating a new state space equal to the cross product of the possible states at each time in the window, this procedure grows the state space exponentially with the size of the

window, rendering Markov models computationally impractical for modeling long-range dependencies [Graves et al., 2014].

Given the limitations of Markov models, we ought to explain why it is reasonable that connectionist models, i.e., artificial neural networks, should fare better. First, recurrent neural networks can capture long-range time dependencies, overcoming the chief limitation of Markov models. This point requires a careful explanation. As in Markov models, any state in a traditional RNN depends only on the current input as well as on the state of the network at the previous time step.[1] However, the hidden state at any time step can contain information from a nearly arbitrarily long context window. This is possible because the number of distinct states that can be represented in a hidden layer of nodes grows exponentially with the number of nodes in the layer. Even if each node took only binary values, the network could represent 2^N states where N is the number of nodes in the hidden layer. When the value of each node is a real number, a network can represent even more distinct states. While the potential expressive power of a network grows exponentially with the number of nodes, the complexity of both inference and training grows at most quadratically.

1.3 Are RNNs too expressive?

Finite-sized RNNs with nonlinear activations are a rich family of models, capable of nearly arbitrary computation. A well-known result is that a finite-sized recurrent neural network with sigmoidal activation functions can simulate a universal Turing machine [Siegelmann and Sontag, 1991]. The capability of RNNs to perform arbitrary computation demonstrates their expressive power, but one could argue that the C programming language is equally capable of expressing arbitrary programs. And yet there are no papers claiming that the invention of C represents a panacea for machine learning. A fundamental reason is there is no simple way of efficiently exploring the space of C programs. In particular, there is no general way to calculate the gradient of an arbitrary C program to minimize a chosen loss function. Moreover, given any finite dataset, there exist countless programs which overfit the dataset, generating desired training output but failing to generalize to test examples.

Why then should RNNs suffer less from similar problems? First, given any fixed architecture (set of nodes, edges, and activation functions), the recurrent neural networks with this architecture are differentiable end to end. The derivative of the loss function can be calculated with respect to each of the parameters (weights) in the model. Thus, RNNs are amenable to gradient-based training. Second, while the Turing-completeness of RNNs is an impressive property, given a fixed-size RNN with a specific architecture, it is not actually possible to reproduce any arbitrary program. Further, unlike a program composed in C, a recurrent neural network can be regularized via standard techniques that help

[1] While traditional RNNs only model the dependence of the current state on the previous state, bidirectional recurrent neural networks (BRNNs) [Schuster and Paliwal, 1997] extend RNNs to model dependence on both past states and future states.

prevent overfitting, such as weight decay, dropout, and limiting the degrees of freedom.

1.4 Comparison to prior literature

The literature on recurrent neural networks can seem impenetrable to the uninitiated. Shorter papers assume familiarity with a large body of background literature, while diagrams are frequently underspecified, failing to indicate which edges span time steps and which do not. Jargon abounds, and notation is inconsistent across papers or overloaded within one paper. Readers are frequently in the unenviable position of having to synthesize conflicting information across many papers in order to understand just one. For example, in many papers subscripts index both nodes and time steps. In others, h simultaneously stands for a link function and a layer of hidden nodes. The variable t simultaneously stands for both time indices and targets, sometimes in the same equation. Many excellent research papers have appeared recently, but clear reviews of the recurrent neural network literature are rare.

Among the most useful resources are a recent book on supervised sequence labeling with recurrent neural networks [Graves, 2012] and an earlier doctoral thesis [Gers, 2001]. A recent survey covers recurrent neural nets for language modeling [De Mulder et al., 2015]. Various authors focus on specific technical aspects; for example Pearlmutter [1995] surveys gradient calculations in continuous time recurrent neural networks. In the present review paper, we aim to provide a readable, intuitive, consistently notated, and reasonably comprehensive but selective survey of research on recurrent neural networks for learning with sequences. We emphasize architectures, algorithms, and results, but we aim also to distill the intuitions that have guided this largely heuristic and empirical field. In addition to concrete modeling details, we offer qualitative arguments, a historical perspective, and comparisons to alternative methodologies where appropriate.

2 Background

This section introduces formal notation and provides a brief background on neural networks in general.

2.1 Sequences

The input to an RNN is a sequence, and/or its target is a sequence. An input sequence can be denoted $(\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, ..., \boldsymbol{x}^{(T)})$ where each data point $\boldsymbol{x}^{(t)}$ is a real-valued vector. Similarly, a target sequence can be denoted $(\boldsymbol{y}^{(1)}, \boldsymbol{y}^{(2)}, ..., \boldsymbol{y}^{(T)})$. A training set typically is a set of examples where each example is an (input sequence, target sequence) pair, although commonly either the input or the output may be a single data point. Sequences may be of finite or countably infinite length. When they are finite, the maximum time index of the sequence

is called T. RNNs are not limited to time-based sequences. They have been used successfully on non-temporal sequence data, including genetic data [Baldi and Pollastri, 2003]. However, in many important applications of RNNs, the sequences have an explicit or implicit temporal aspect. While we often refer to time in this survey, the methods described here are applicable to non-temporal as well as to temporal tasks.

Using temporal terminology, an input sequence consists of data points $\boldsymbol{x}^{(t)}$ that arrive in a discrete sequence of *time steps* indexed by t. A target sequence consists of data points $\boldsymbol{y}^{(t)}$. We use superscripts with parentheses for time, and not subscripts, to prevent confusion between sequence steps and indices of nodes in a network. When a model produces predicted data points, these are labeled $\hat{\boldsymbol{y}}^{(t)}$.

The time-indexed data points may be equally spaced samples from a continuous real-world process. Examples include the still images that comprise the frames of videos or the discrete amplitudes sampled at fixed intervals that comprise audio recordings. The time steps may also be ordinal, with no exact correspondence to durations. In fact, RNNs are frequently applied to domains where sequences have a defined order but no explicit notion of time. This is the case with natural language. In the word sequence "John Coltrane plays the saxophone", $\boldsymbol{x}^{(1)}$ = John, $\boldsymbol{x}^{(2)}$ = Coltrane, etc.

2.2 Neural networks

Neural networks are biologically inspired models of computation. Generally, a neural network consists of a set of *artificial neurons*, commonly referred to as *nodes* or *units*, and a set of directed edges between them, which intuitively represent the *synapses* in a biological neural network. Associated with each neuron j is an activation function $l_j(\cdot)$, which is sometimes called a link function. We use the notation l_j and not h_j, unlike some other papers, to distinguish the activation function from the values of the hidden nodes in a network, which, as a vector, is commonly notated $\boldsymbol{h}$ in the literature.

Associated with each edge from node j' to j is a weight $w_{jj'}$. Following the convention adopted in several foundational papers [Hochreiter and Schmidhuber, 1997, Gers et al., 2000, Gers, 2001, Sutskever et al., 2011], we index neurons with j and j', and $w_{jj'}$ denotes the "to-from" weight corresponding to the directed edge to node j from node j'. It is important to note that in many references the indices are flipped and $w_{j'j} \neq w_{jj'}$ denotes the "from-to" weight on the directed edge from the node j' to the node j, as in lecture notes by Elkan [2015] and in Wikipedia [2015].

The value v_j of each neuron j is calculated by applying its activation function to a weighted sum of the values of its input nodes (Figure 1):

$$v_j = l_j\left(\sum_{j'} w_{jj'} \cdot v_{j'}\right). \qquad \square$$

For convenience, we term the weighted sum inside the parentheses the *incoming*

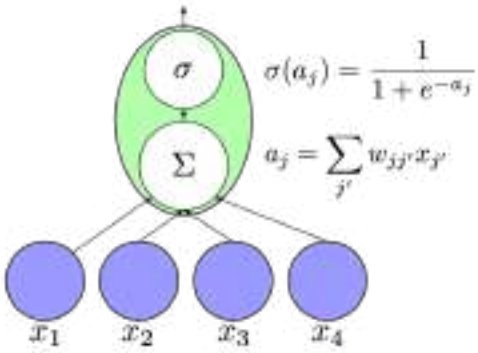


Figure 1: An artificial neuron computes a nonlinear function of a weighted sum of its inputs.

activation and notate it as a_j. We represent this computation in diagrams by depicting neurons as circles and edges as arrows connecting them. When appropriate, we indicate the exact activation function with a symbol, e.g., σ for sigmoid.

Common choices for the activation function include the sigmoid $\sigma(z) = 1/(1+e^{-z})$ and the *tanh* function $\phi(z) = (e^z - e^{-z})/(e^z + e^{-z})$. The latter has become common in feedforward neural nets and was applied to recurrent nets by Sutskever et al. [2011]. Another activation function which has become prominent in deep learning research is the rectified linear unit (ReLU) whose formula is $l_j(z) = \max(0, z)$. This type of unit has been demonstrated to improve the performance of many deep neural networks [Nair and Hinton, 2010, Maas et al., 2012, Zeiler et al., 2013] on tasks as varied as speech processing and object recognition, and has been used in recurrent neural networks by Bengio et al. [2013].

The activation function at the output nodes depends upon the task. For multiclass classification with K alternative classes, we apply a softmax nonlinearity in an output layer of K nodes. The softmax function calculates

$$\hat{y}_k = \frac{e^{a_k}}{\sum_{k'=1}^{K} e^{a_{k'}}} \text{ for } k = 1 \text{ to } k = K.$$

The denominator is a normalizing term consisting of the sum of the numerators, ensuring that the outputs of all nodes sum to one. For multilabel classification the activation function is simply a point-wise sigmoid, and for regression we typically have linear output.

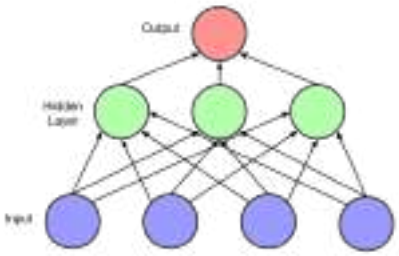


Figure 2: A feedforward neural network. An example is presented to the network by setting the values of the blue (bottom) nodes. The values of the nodes in each layer are computed successively as a function of the prior layers until output is produced at the topmost layer.

2.3 Feedforward networks and backpropagation

With a neural model of computation, one must determine the order in which computation should proceed. Should nodes be sampled one at a time and updated, or should the value of all nodes be calculated at once and then all updates applied simultaneously? Feedforward networks (Figure 2) are a restricted class of networks which deal with this problem by forbidding cycles in the directed graph of nodes. Given the absence of cycles, all nodes can be arranged into layers, and the outputs in each layer can be calculated given the outputs from the lower layers.

The input $\boldsymbol{x}$ to a feedforward network is provided by setting the values of the lowest layer. Each higher layer is then successively computed until output is generated at the topmost layer $\hat{\boldsymbol{y}}$. Feedforward networks are frequently used for supervised learning tasks such as classification and regression. Learning is accomplished by iteratively updating each of the weights to minimize a loss function, $\mathcal{L}(\hat{\boldsymbol{y}}, \boldsymbol{y})$, which penalizes the distance between the output $\hat{\boldsymbol{y}}$ and the target $\boldsymbol{y}$.

The most successful algorithm for training neural networks is backpropagation, introduced for this purpose by Rumelhart et al. [1985]. Backpropagation uses the chain rule to calculate the derivative of the loss function $\mathcal{L}$ with respect to each parameter in the network. The weights are then adjusted by gradient descent. Because the loss surface is non-convex, there is no assurance that backpropagation will reach a global minimum. Moreover, exact optimization is known to be an NP-hard problem. However, a large body of work on heuristic

pre-training and optimization techniques has led to impressive empirical success on many supervised learning tasks. In particular, convolutional neural networks, popularized by Le Cun et al. [1990], are a variant of feedforward neural network that holds records since 2012 in many computer vision tasks such as object detection [Krizhevsky et al., 2012].

Nowadays, neural networks are usually trained with stochastic gradient descent (SGD) using mini-batches. With batch size equal to one, the stochastic gradient update equation is

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} F_i$$

where η is the learning rate and $\nabla_{\boldsymbol{w}} F_i$ is the gradient of the objective function with respect to the parameters $\boldsymbol{w}$ as calculated on a single example (x_i, y_i). Many variants of SGD are used to accelerate learning. Some popular heuristics, such as AdaGrad [Duchi et al., 2011], AdaDelta [Zeiler, 2012], and RMSprop [Tieleman and Hinton, 2012], tune the learning rate adaptively for each feature. AdaGrad, arguably the most popular, adapts the learning rate by caching the sum of squared gradients with respect to each parameter at each time step. The step size for each feature is multiplied by the inverse of the square root of this cached value. AdaGrad leads to fast convergence on convex error surfaces, but because the cached sum is monotonically increasing, the method has a monotonically decreasing learning rate, which may be undesirable on highly non-convex loss surfaces. RMSprop modifies AdaGrad by introducing a decay factor in the cache, changing the monotonically growing value into a moving average. Momentum methods are another common SGD variant used to train neural networks. These methods add to each update a decaying sum of the previous updates. When the momentum parameter is tuned well and the network is initialized well, momentum methods can train deep nets and recurrent nets competitively with more computationally expensive methods like the Hessian-free optimizer of Sutskever et al. [2013].

To calculate the gradient in a feedforward neural network, backpropagation proceeds as follows. First, an example is propagated forward through the network to produce a value v_j at each node and outputs $\hat{\boldsymbol{y}}$ at the topmost layer. Then, a loss function value $\mathcal{L}(\hat{y}_k, y_k)$ is computed at each output node k. Subsequently, for each output node k, we calculate

$$\delta_k = \frac{\partial \mathcal{L}(\hat{y}_k, y_k)}{\partial \hat{y}_k} \cdot l'_k(a_k).$$

Given these values δ_k, for each node in the immediately prior layer we calculate

$$\delta_j = l'(a_j) \sum_k \delta_k \cdot w_{kj}.$$

This calculation is performed successively for each lower layer to yield δ_j for every node j given the δ values for each node connected to j by an outgoing edge. Each value δ_j represents the derivative $\partial \mathcal{L} / \partial a_j$ of the total loss function with respect to that node's incoming activation. Given the values v_j calculated

during the forward pass, and the values δ_j calculated during the backward pass, the derivative of the loss $\mathcal{L}$ with respect a given parameter $w_{jj'}$ is

$$\frac{\partial \mathcal{L}}{\partial w_{jj'}} = \delta_j v_{j'}.$$

Other methods have been explored for learning the weights in a neural network. A number of papers from the 1990s [Belew et al., 1990, Gruau et al., 1994] championed the idea of learning neural networks with genetic algorithms, with some even claiming that achieving success on real-world problems only by applying many small changes to the weights of a network was impossible. Despite the subsequent success of backpropagation, interest in genetic algorithms continues. Several recent papers explore genetic algorithms for neural networks, especially as a means of learning the architecture of neural networks, a problem not addressed by backpropagation [Bayer et al., 2009, Harp and Samad, 2013]. By *architecture* we mean the number of layers, the number of nodes in each, the connectivity pattern among the layers, the choice of activation functions, etc.

One open question in neural network research is how to exploit sparsity in training. In a neural network with sigmoidal or *tanh* activation functions, the nodes in each layer never take value exactly zero. Thus, even if the inputs are sparse, the nodes at each hidden layer are not. However, rectified linear units (ReLUs) introduce sparsity to hidden layers [Glorot et al., 2011]. In this setting, a promising path may be to store the sparsity pattern when computing each layer's values and use it to speed up computation of the next layer in the network. Some recent work shows that given sparse inputs to a linear model with a standard regularizer, sparsity can be fully exploited even if regularization makes the gradient be not sparse [Carpenter, 2008, Langford et al., 2009, Singer and Duchi, 2009, Lipton and Elkan, 2015].

3 Recurrent neural networks

Recurrent neural networks are feedforward neural networks augmented by the inclusion of edges that span adjacent time steps, introducing a notion of time to the model. Like feedforward networks, RNNs may not have cycles among conventional edges. However, edges that connect adjacent time steps, called recurrent edges, may form cycles, including cycles of length one that are self-connections from a node to itself across time. At time t, nodes with recurrent edges receive input from the current data point $\boldsymbol{x}^{(t)}$ and also from hidden node values $\boldsymbol{h}^{(t-1)}$ in the network's previous state. The output $\hat{\boldsymbol{y}}^{(t)}$ at each time t is calculated given the hidden node values $\boldsymbol{h}^{(t)}$ at time t. Input $\boldsymbol{x}^{(t-1)}$ at time $t-1$ can influence the output $\hat{\boldsymbol{y}}^{(t)}$ at time t and later by way of the recurrent connections.

Two equations specify all calculations necessary for computation at each time step on the forward pass in a simple recurrent neural network as in Figure 3:

$$\boldsymbol{h}^{(t)} = \sigma(W^{\text{hx}}\boldsymbol{x}^{(t)} + W^{\text{hh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h)$$

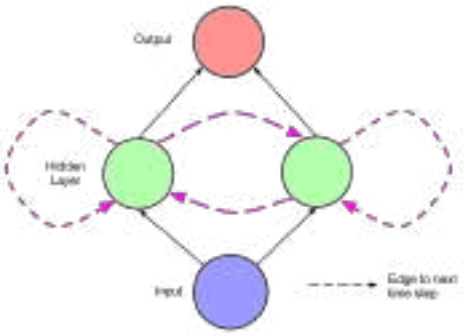


Figure 3: A simple recurrent network. At each time step t, activation is passed along solid edges as in a feedforward network. Dashed edges connect a source node at each time t to a target node at each following time $t + 1$.

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}(W^{\text{yh}}\boldsymbol{h}^{(t)} + \boldsymbol{b}_y).$$

Here W_{hx} is the matrix of conventional weights between the input and the hidden layer and W_{hh} is the matrix of recurrent weights between the hidden layer and itself at adjacent time steps. The vectors $\boldsymbol{b}_h$ and $\boldsymbol{b}_y$ are bias parameters which allow each node to learn an offset.

The dynamics of the network depicted in Figure 3 across time steps can be visualized by *unfolding* it as in Figure 4. Given this picture, the network can be interpreted not as cyclic, but rather as a deep network with one layer per time step and shared weights across time steps. It is then clear that the unfolded network can be trained across many time steps using backpropagation. This algorithm, called *backpropagation through time* (BPTT), was introduced by Werbos [1990]. All recurrent networks in common current use apply it.

3.1 Early recurrent network designs

The foundational research on recurrent networks took place in the 1980s. In 1982, Hopfield introduced a family of recurrent neural networks that have pattern recognition capabilities [Hopfield, 1982]. They are defined by the values of the weights between nodes and the link functions are simple thresholding at zero. In these nets, a pattern is placed in the network by setting the values of the nodes. The network then runs for some time according to its update rules, and eventually another pattern is read out. Hopfield networks are useful for recovering a stored pattern from a corrupted version and are the forerunners of Boltzmann machines and auto-encoders.

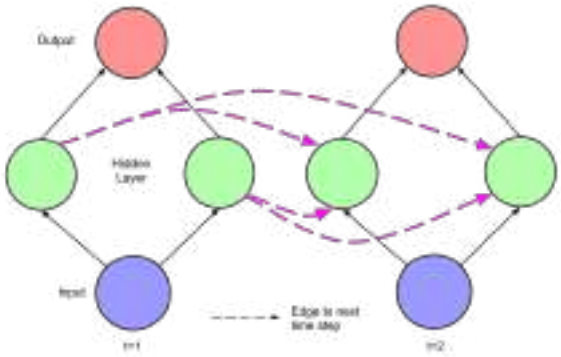


Figure 4: The recurrent network of Figure 3 unfolded across time steps.

An early architecture for supervised learning on sequences was introduced by Jordan [1986]. Such a network (Figure 5) is a feedforward network with a single hidden layer that is extended with special units.[2] Output node values are fed to the special units, which then feed these values to the hidden nodes at the following time step. If the output values are actions, the special units allow the network to remember actions taken at previous time steps. Several modern architectures use a related form of direct transfer from output nodes; Sutskever et al. [2014] translates sentences between natural languages, and when generating a text sequence, the word chosen at each time step is fed into the network as input at the following time step. Additionally, the special units in a Jordan network are self-connected. Intuitively, these edges allow sending information across multiple time steps without perturbing the output at each intermediate time step.

The architecture introduced by Elman [1990] is simpler than the earlier Jordan architecture. Associated with each unit in the hidden layer is a context unit. Each such unit j' takes as input the state of the corresponding hidden node j at the previous time step, along an edge of fixed weight $w_{j'j} = 1$. This value then feeds back into the same hidden node j along a standard edge. This architecture is equivalent to a simple RNN in which each hidden node has a single self-connected recurrent edge. The idea of fixed-weight recurrent edges that make hidden nodes self-connected is fundamental in subsequent work on LSTM networks [Hochreiter and Schmidhuber, 1997].

Elman [1990] trains the network using backpropagation and demonstrates that the network can learn time dependencies. The paper features two sets of experiments. The first extends the logical operation *exclusive or* (XOR) to

[2] Jordan [1986] calls the special units "state units" while Elman [1990] calls a corresponding structure "context units." In this paper we simplify terminology by using only "context units".

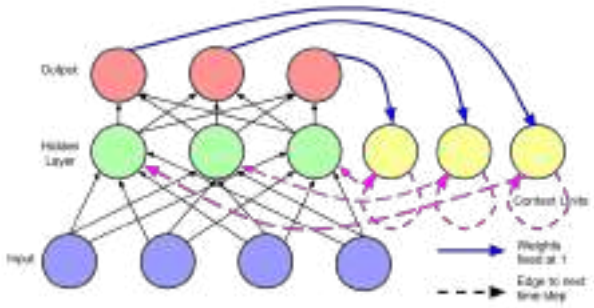


Figure 5: A recurrent neural network as proposed by Jordan [1986]. Output units are connected to special units that at the next time step feed into themselves and into hidden units.

the time domain by concatenating sequences of three tokens. For each three-token segment, e.g. "011", the first two tokens ("01") are chosen randomly and the third ("1") is set by performing xor on the first two. Random guessing should achieve accuracy of 50%. A perfect system should perform the same as random for the first two tokens, but guess the third token perfectly, achieving accuracy of 66.7%. The simple network of Elman [1990] does in fact approach this maximum achievable score.

3.2 Training recurrent networks

Learning with recurrent networks has long been considered to be difficult. Even for standard feedforward networks, the optimization task is NP-complete Blum and Rivest [1993]. But learning with recurrent networks can be especially challenging due to the difficulty of learning long-range dependencies, as described by Bengio et al. [1994] and expanded upon by Hochreiter et al. [2001]. The problems of *banishing* and *exploding* gradients occur when backpropagating errors across many time steps. As a toy example, consider a network with a single input node, a single output node, and a single recurrent hidden node (Figure 7). Now consider an input passed to the network at time τ and an error calculated at time t, assuming input of zero in the intervening time steps. The tying of weights across time steps means that the recurrent edge at the hidden node j always has the same weight. Therefore, the contribution of the input at time τ to the output at time t will either explode or approach zero, exponentially fast, as $t - \tau$ grows large. Hence the derivative of the error with respect to the input will either explode or vanish.

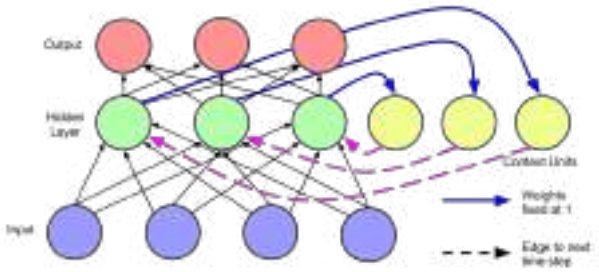


Figure 6: A recurrent neural network as described by Elman [1990]. Hidden units are connected to context units, which feed back into the hidden units at the next time step.

Which of the two phenomena occurs depends on whether the weight of the recurrent edge $|w_{jj}| > 1$ or $|w_{jj}| < 1$ and on the activation function in the hidden node (Figure 8). Given a sigmoid activation function, the vanishing gradient problem is more pressing, but with a rectified linear unit $\max(0, x)$, it is easier to imagine the exploding gradient. Pascanu et al. [2012] give a thorough mathematical treatment of the vanishing and exploding gradient problems, characterizing exact conditions under which these problems may occur. Given these conditions, they suggest an approach to training via a regularization term that forces the weights to values where the gradient neither vanishes nor explodes.

Truncated backpropagation through time (TBPTT) is one solution to the exploding gradient problem for continuously running networks [Williams and Zipser, 1989]. With TBPTT, some maximum number of time steps is set along which error can be propagated. While TBPTT with a small cutoff can be used to alleviate the exploding gradient problem, it requires that one sacrifice the ability to learn long-range dependencies. The LSTM architecture described below uses carefully designed nodes with recurrent edges with fixed unit weight as a solution to the vanishing gradient problem.

The issue of local optima is an obstacle to effective training that cannot be dealt with simply by modifying the network architecture. Optimizing even a single hidden-layer feedforward network is an NP-complete problem [Blum and Rivest, 1993]. However, recent empirical and theoretical studies suggest that in practice, the issue may not be as important as once thought. Dauphin et al. [2014] show that while many critical points exist on the error surfaces of large neural networks, the ratio of saddle points to true local minima increases exponentially with the size of the network, and algorithms can be designed to

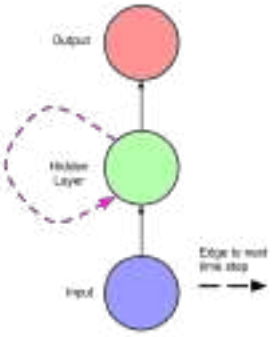


Figure 7: A simple recurrent net with one input unit, one output unit, and one recurrent hidden unit.

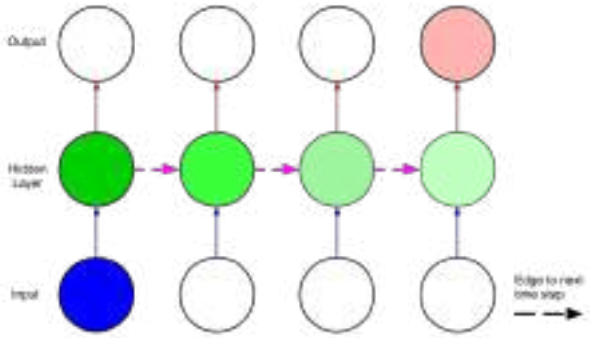


Figure 8: A visualization of the vanishing gradient problem, using the network depicted in Figure 7, adapted from Graves [2012]. If the weight along the recurrent edge is less than one, the contribution of the input at the first time step to the output at the final time step will decrease exponentially fast as a function of the length of the time interval in between.

escape from saddle points.

Overall, along with the improved architectures explained below, fast implementations and better gradient-following heuristics have rendered RNN training feasible. Implementations of forward and backward propagation using GPUs, such as the Theano [Bergstra et al., 2010] and Torch [Collobert et al., 2011] packages, have made it straightforward to implement fast training algorithms. In 1996, prior to the introduction of the LSTM, attempts to train recurrent nets to bridge long time gaps were shown to perform no better than random guessing [Hochreiter and Schmidhuber, 1996]. However, RNNs are now frequently trained successfully.

For some tasks, freely available software can be run on a single GPU and produce compelling results in hours [Karpathy, 2015]. Martens and Sutskever [2011] reported success training recurrent neural networks with a Hessian-free truncated Newton approach, and applied the method to a network which learns to generate text one character at a time in [Sutskever et al., 2011]. In the paper that describes the abundance of saddle points on the error surfaces of neural networks [Dauphin et al., 2014], the authors present a saddle-free version of Newton's method. Unlike Newton's method, which is attracted to critical points, including saddle points, this variant is specially designed to escape from them. Experimental results include a demonstration of improved performance on recurrent networks. Newton's method requires computing the Hessian, which is prohibitively expensive for large networks, scaling quadratically with the number of parameters. While their algorithm only approximates the Hessian, it is still computationally expensive compared to SGD. Thus the authors describe a hybrid approach in which the saddle-free Newton method is applied only in places where SGD appears to be stuck.

4 Modern RNN architectures

The most successful RNN architectures for sequence learning stem from two papers published in 1997. The first paper, *Long Short-Term Memory* by Hochreiter and Schmidhuber [1997], introduces the memory cell, a unit of computation that replaces traditional nodes in the hidden layer of a network. With these memory cells, networks are able to overcome difficulties with training encountered by earlier recurrent networks. The second paper, *Bidirectional Recurrent Neural Networks* by Schuster and Paliwal [1997], introduces an architecture in which information from both the future and the past are used to determine the output at any point in the sequence. This is in contrast to previous networks, in which only past input can affect the output, and has been used successfully for sequence labeling tasks in natural language processing, among others. Fortunately, the two innovations are not mutually exclusive, and have been successfully combined for phoneme classification [Graves and Schmidhuber, 2005] and handwriting recognition [Graves et al., 2009]. In this section we explain the LSTM and BRNN and we describe the *neural Turing machine* (NTM), which extends RNNs with an addressable external memory [Graves et al., 2014].

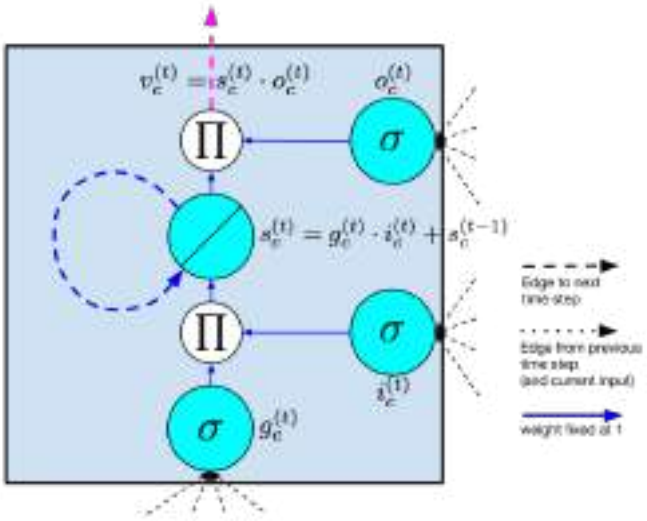


Figure 9: One LSTM memory cell as proposed by Hochreiter and Schmidhuber [1997]. The self-connected node is the internal state s. The diagonal line indicates that it is linear, i.e. the identity link function is applied. The blue dashed line is the recurrent edge, which has fixed unit weight. Nodes marked Π output the product of their inputs. All edges into and from Π nodes also have fixed unit weight.

4.1 Long short-term memory (LSTM)

Hochreiter and Schmidhuber [1997] introduced the LSTM model primarily in order to overcome the problem of vanishing gradients. This model resembles a standard recurrent neural network with a hidden layer, but each ordinary node (Figure 1) in the hidden layer is replaced by a *memory cell* (Figure 9). Each memory cell contains a node with a self-connected recurrent edge of fixed weight one, ensuring that the gradient can pass across many time steps without vanishing or exploding. To distinguish references to a memory cell and not an ordinary node, we use the subscript c.

The term "long short-term memory" comes from the following intuition. Simple recurrent neural networks have *long-term memory* in the form of weights. The weights change slowly during training, encoding general knowledge about the data. They also have *short-term memory* in the form of ephemeral activations, which pass from each node to successive nodes. The LSTM model introduces an intermediate type of storage via the memory cell. A memory cell is a composite unit, built from simpler nodes in a specific connectivity pattern, with the novel inclusion of multiplicative nodes, represented in diagrams by the letter Π. All elements of the LSTM cell are enumerated and described below. Note that when we use vector notation, we are referring to the values of the

nodes in an entire layer of cells. For example, $\boldsymbol{s}$ is a vector containing the value of s_c at each memory cell c in a layer. When the subscript c is used, it is to index an individual memory cell.

- *Input node:* This unit, labeled g_c, is a node that takes activation in the standard way from the input layer $\boldsymbol{x}^{(t)}$ at the current time step and (along recurrent edges) from the hidden layer at the previous time step $\boldsymbol{h}^{(t-1)}$. Typically, the summed weighted input is run through a *tanh* activation function, although in the original LSTM paper, the activation function is a *sigmoid*.

- *Input gate:* Gates are a distinctive feature of the LSTM approach. A gate is a sigmoidal unit that, like the input node, takes activation from the current data point $\boldsymbol{x}^{(t)}$ as well as from the hidden layer at the previous time step. A gate is so-called because its value is used to multiply the value of another node. It is a *gate* in the sense that if its value is zero, then flow from the other node is cut off. If the value of the gate is one, all flow is passed through. The value of the *input gate* i_c multiplies the value of the *input node*.

- *Internal state:* At the heart of each memory cell is a node s_c with linear activation, which is referred to in the original paper as the "internal state" of the cell. The internal state s_c has a self-connected recurrent edge with fixed unit weight. Because this edge spans adjacent time steps with constant weight, error can flow across time steps without vanishing or exploding. This edge is often called the *constant error carousel*. In vector notation, the update for the internal state is $\boldsymbol{s}^{(t)} = \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(t)} + \boldsymbol{s}^{(t-1)}$ where $\odot$ is pointwise multiplication.

- *Forget gate:* These gates f_c were introduced by Gers et al. [2000]. They provide a method by which the network can learn to flush the contents of the internal state. This is especially useful in continuously running networks. With forget gates, the equation to calculate the internal state on the forward pass is

$$\boldsymbol{s}^{(t)} = \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(t)} + \boldsymbol{f}^{(t)} \odot \boldsymbol{s}^{(t-1)}.$$

- *Output gate:* The value v_c ultimately produced by a memory cell is the value of the internal state s_c multiplied by the value of the *output gate* o_c. It is customary that the internal state first be run through a *tanh* activation function, as this gives the output of each cell the same dynamic range as an ordinary *tanh* hidden unit. However, in other neural network research, rectified linear units, which have a greater dynamic range, are easier to train. Thus it seems plausible that the nonlinear function on the internal state might be omitted.

In the original paper and in most subsequent work, the input node is labeled g. We adhere to this convention but note that it may be confusing as g does

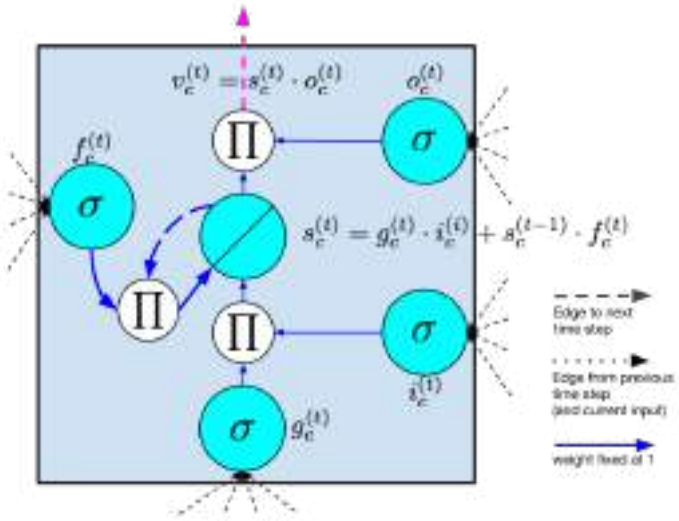


Figure 10: LSTM memory cell with a forget gate as described by Gers et al. [2000].

not stand for *gate*. In the original paper, the gates are called y_{in} and y_{out} but this is confusing because y generally stands for output in the machine learning literature. Seeking comprehensibility, we break with this convention and use i, f, and o to refer to input, forget and output gates respectively, as in Sutskever et al. [2014].

Since the original LSTM was introduced, several variations have been proposed. Forget gates, described above, were proposed in 2000 and were not part of the original LSTM design. However, they have proven effective and are standard in most modern implementations. That same year, Gers and Schmidhuber [2000] proposed peephole connections that pass from the internal state directly to the input and output gates of that same node without first having to be modulated by the output gate. They report that these connections improve performance on timing tasks where the network must learn to measure precise intervals between events. The intuition of the peephole connection can be captured by the following example. Consider a network which must learn to count objects and emit some desired output when n objects have been seen. The network might learn to let some fixed amount of activation into the internal state after each object is seen. This activation is trapped in the internal state s_c by the constant error carousel, and is incremented iteratively each time another object is seen. When the nth object is seen, the network needs to know to let out content from the internal state so that it can affect the output. To accomplish this, the output gate o_c must know the content of the internal state s_c. Thus s_c should be an input to o_c.

Put formally, computation in the LSTM model proceeds according to the

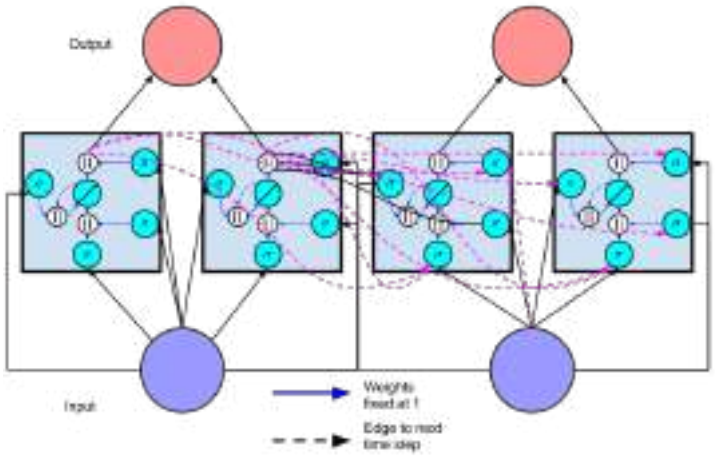


Figure 11: A recurrent neural network with a hidden layer consisting of two memory cells. The network is shown unfolded across two time steps.

following calculations, which are performed at each time step. These equations give the full algorithm for a modern LSTM with forget gates:

$$\boldsymbol{g}^{(t)} = \phi(W^{\mathrm{gx}}\boldsymbol{x}^{(t)} + W^{\mathrm{gh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_g)$$

$$\boldsymbol{i}^{(t)} = \sigma(W^{\mathrm{ix}}\boldsymbol{x}^{(t)} + W^{\mathrm{ih}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_i)$$

$$\boldsymbol{f}^{(t)} = \sigma(W^{\mathrm{fx}}\boldsymbol{x}^{(t)} + W^{\mathrm{fh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_f)$$

$$\boldsymbol{o}^{(t)} = \sigma(W^{\mathrm{ox}}\boldsymbol{x}^{(t)} + W^{\mathrm{oh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_o)$$

$$\boldsymbol{s}^{(t)} = \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(i)} + \boldsymbol{s}^{(t-1)} \odot \boldsymbol{f}^{(t)}$$

$$\boldsymbol{h}^{(t)} = \phi(\boldsymbol{s}^{(t)}) \odot \boldsymbol{o}^{(t)}.$$

The value of the hidden layer of the LSTM at time t is the vector $\boldsymbol{h}^{(t)}$, while $\boldsymbol{h}^{(t-1)}$ is the values output by each memory cell in the hidden layer at the previous time. Note that these equations include the forget gate, but not peephole connections. The calculations for the simpler LSTM without forget gates are obtained by setting $\boldsymbol{f}^{(t)} = 1$ for all t. We use the *tanh* function ϕ for the input node g following the state-of-the-art design of Zaremba and Sutskever [2014]. However, in the original LSTM paper, the activation function for g is the sigmoid σ.

Intuitively, in terms of the forward pass, the LSTM can learn when to let activation into the internal state. As long as the input gate takes value zero, no activation can get in. Similarly, the output gate learns when to let the

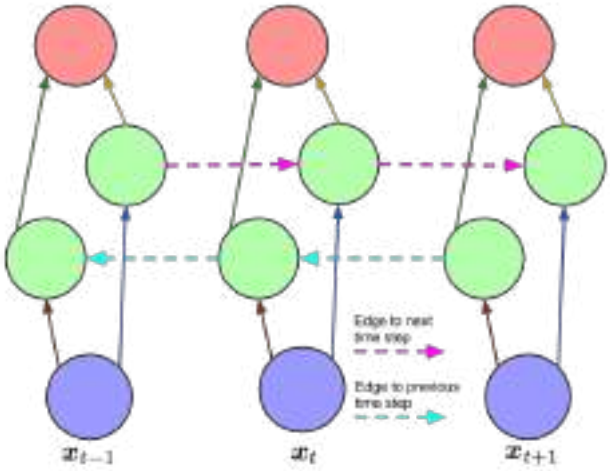


Figure 12: A bidirectional recurrent neural network as described by Schuster and Paliwal [1997], unfolded in time.

value out. When both gates are *closed*, the activation is trapped in the memory cell, neither growing nor shrinking, nor affecting the output at intermediate time steps. In terms of the backwards pass, the constant error carousel enables the gradient to propagate back across many time steps, neither exploding nor vanishing. In this sense, the gates are learning when to let error in, and when to let it out. In practice, the LSTM has shown a superior ability to learn long-range dependencies as compared to simple RNNs. Consequently, the majority of state-of-the-art application papers covered in this review use the LSTM model.

One frequent point of confusion is the manner in which multiple memory cells are used together to comprise the hidden layer of a working neural network. To alleviate this confusion, we depict in Figure 11 a simple network with two memory cells, analogous to Figure 4. The output from each memory cell flows in the subsequent time step to the input node and all gates of each memory cell. It is common to include multiple layers of memory cells [Sutskever et al., 2014]. Typically, in these architectures each layer takes input from the layer below at the same time step and from the same layer in the previous time step.

4.2 Bidirectional recurrent neural networks (BRNNs)

Along with the LSTM, one of the most used RNN architectures is the bidirectional recurrent neural network (BRNN) (Figure 12) first described by Schuster and Paliwal [1997]. In this architecture, there are two layers of hidden nodes. Both hidden layers are connected to input and output. The two hidden layers are differentiated in that the first has recurrent connections from the past time

steps while in the second the direction of recurrent of connections is flipped, passing activation backwards along the sequence. Given an input sequence and a target sequence, the BRNN can be trained by ordinary backpropagation after unfolding across time. The following three equations describe a BRNN:

$$\boldsymbol{h}^{(t)} = \sigma(W^{\mathrm{hx}}\boldsymbol{x}^{(t)} + W^{\mathrm{hh}}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h)$$

$$\boldsymbol{z}^{(t)} = \sigma(W^{\mathrm{zx}}\boldsymbol{x}^{(t)} + W^{\mathrm{zz}}\boldsymbol{z}^{(t+1)} + \boldsymbol{b}_z)$$

$$\hat{\boldsymbol{y}}^{(t)} = \mathrm{softmax}(W^{\mathrm{yh}}\boldsymbol{h}^{(t)} + W^{\mathrm{yz}}\boldsymbol{z}^{(t)} + \boldsymbol{b}_y)$$

where $\boldsymbol{h}^{(t)}$ and $\boldsymbol{z}^{(t)}$ are the values of the hidden layers in the forwards and backwards directions respectively.

One limitation of the BRNN is that cannot run continuously, as it requires a fixed endpoint in both the future and in the past. Further, it is not an appropriate machine learning algorithm for the online setting, as it is implausible to receive information from the future, i.e., to know sequence elements that have not been observed. But for prediction over a sequence of fixed length, it is often sensible to take into account both past and future sequence elements. Consider the natural language task of part-of-speech tagging. Given any word in a sentence, information about both the words which precede and those which follow it is useful for predicting that word's part-of-speech.

The LSTM and BRNN are in fact compatible ideas. The former introduces a new basic unit from which to compose a hidden layer, while the latter concerns the wiring of the hidden layers, regardless of what nodes they contain. Such an approach, termed a BLSTM has been used to achieve state of the art results on handwriting recognition and phoneme classification [Graves and Schmidhuber, 2005, Graves et al., 2009].

4.3 Neural Turing machines

The neural Turing machine (NTM) extends recurrent neural networks with an addressable external memory [Graves et al., 2014]. This work improves upon the ability of RNNs to perform complex algorithmic tasks such as sorting. The authors take inspiration from theories in cognitive science, which suggest humans possess a "central executive" that interacts with a memory buffer [Baddeley et al., 1996]. By analogy to a Turing machine, in which a program directs *read heads* and *write heads* to interact with external memory in the form of a tape, the model is named a Neural Turing Machine. While technical details of the read/write heads are beyond the scope of this review, we aim to convey a high-level sense of the model and its applications.

The two primary components of an NTM are a *controller* and *memory matrix*. The controller, which may be a recurrent or feedforward neural network, takes input and returns output to the outside world, as well as passing instructions to and reading from the memory. The memory is represented by a large matrix of N memory locations, each of which is a vector of dimension M. Additionally, a number of read and write heads facilitate the interaction between

the controller and the memory matrix. Despite these additional capabilities, the NTM is differentiable end-to-end and can be trained by variants of stochastic gradient descent using BPTT.

Graves et al. 2014 select five algorithmic tasks to test the performance of the NTM model. By *algorithmic* we mean that for each task, the target output for a given input can be calculated by following a simple program, as might be easily implemented in any universal programming language. One example is the *copy* task, where the input is a sequence of fixed length binary vectors followed by a delimiter symbol. The target output is a copy of the input sequence. In another task, *priority sort*, an input consists of a sequence of binary vectors together with a distinct scalar priority value for each vector. The target output is the sequence of vectors sorted by priority. The experiments test whether an NTM can be trained via supervised learning to implement these common algorithms correctly and efficiently. Interestingly, solutions found in this way generalize reasonably well to inputs longer than those presented in the training set. In contrast, the LSTM without external memory does not generalize well to longer inputs. The authors compare three different architectures, namely an LSTM RNN, an NTM with a feedforward controller, and an NTM with an LSTM controller. On each task, both NTM architectures significantly outperform the LSTM RNN both in training set performance and in generalization to test data.

5 Applications of LSTMs and BRNNs

The previous sections introduced the building blocks from which nearly all state-of-the-art recurrent neural networks are composed. This section looks at several application areas where recurrent networks have been employed successfully. Before describing state of the art results in detail, it is appropriate to convey a concrete sense of the precise architectures with which many important tasks can be expressed clearly as sequence learning problems with recurrent neural networks. Figure 13 demonstrates several common RNN architectures and associates each with corresponding well-documented tasks.

In the following subsections, we first introduce the representations of natural language used for input and output to recurrent neural networks and the commonly used performance metrics for sequence prediction tasks. Then we survey state-of-the-art results in machine translation, image captioning, video captioning, and handwriting recognition. Many applications of RNNs involve processing written language. Some applications, such as image captioning, involve generating strings of text. Others, such as machine translation and dialogue systems, require both inputting and outputting text. Systems which output text are more difficult to evaluate empirically than those which produce binary predictions or numerical output. As a result several methods have been developed to assess the quality of translations and captions. In the next subsection, we provide the background necessary to understand how text is represented in most modern recurrent net applications. We then explain the commonly reported evaluation metrics.

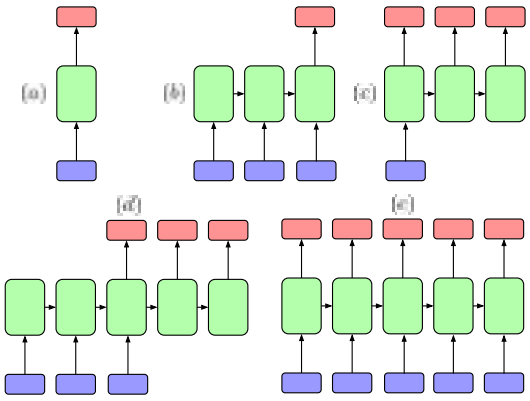


Figure 13: Recurrent neural networks have been used successfully to model both sequential inputs and sequential outputs as well as mappings between single data points and sequences (in both directions). This figure, based on a similar figure in Karpathy [2015], shows how numerous tasks can be modeled with RNNs with sequential inputs and/or sequential outputs. In each subfigure, blue rectangles correspond to inputs, red rectangles to outputs and green rectangles to the entire hidden state of the neural network. (a) This is the conventional independent case, as assumed by standard feedforward networks. (b) Text and video classification are tasks in which a sequence is mapped to one fixed length vector. (c) Image captioning presents the converse case, where the input image is a single non-sequential data point. (d) This architecture has been used for natural language translation, a sequence-to-sequence task in which the two sequences may have varying and different lengths. (e) This architecture has been used to learn a generative model for text, predicting at each step the following character.

5.1 Representations of natural language inputs and outputs

When words are output at each time step, generally the output consists of a softmax vector $\boldsymbol{y}^{(t)} \in \mathbb{R}^K$ where K is the size of the vocabulary. A softmax layer is an element-wise logistic function that is normalized so that all of its components sum to one. Intuitively, these outputs correspond to the probabilities that each word is the correct output at that time step.

For application where an input consists of a sequence of words, typically the words are fed to the network one at a time in consecutive time steps. In these cases, the simplest way to represent words is a *one-hot* encoding, using binary vectors with a length equal to the size of the vocabulary, so "1000" and "0100" would represent the first and second words in the vocabulary respectively. Such an encoding is discussed by Elman [1990] among others. However, this encoding is inefficient, requiring as many bits as the vocabulary is large. Further, it offers no direct way to capture different aspects of similarity between words in the encoding itself. Thus it is common now to model words with a distributed representation using a *meaning vector*. In some cases, these meanings for words are learned given a large corpus of supervised data, but it is more usual to initialize the *meaning vectors* using an embedding based on word co-occurrence statistics. Freely available code to produce word vectors from these statistics include *GloVe* [Pennington et al., 2014], and *word2vec* [Goldberg and Levy, 2014], which implements a word embedding algorithm from Mikolov et al. [2013].

Distributed representations for textual data were described by Hinton [1986], used extensively for natural language by Bengio et al. [2003], and more recently brought to wider attention in the deep learning community in a number of papers describing recursive auto-encoder (RAE) networks [Socher et al., 2010, 2011a,b,c]. For clarity we point out that these *recursive* networks are *not* recurrent neural networks, and in the present survey the abbreviation RNN always means recurrent neural network. While they are distinct approaches, recurrent and recursive neural networks have important features in common, namely that they both involve extensive weight tying and are both trained end-to-end via backpropagation.

In many experiments with recurrent neural networks [Elman, 1990, Sutskever et al., 2011, Zaremba and Sutskever, 2014], input is fed in one character at a time, and output generated one character at a time, as opposed to one word at a time. While the output is nearly always a softmax layer, many papers omit details of how they represent single-character inputs. It seems reasonable to infer that characters are encoded with a one-hot encoding. We know of no cases of paper using a distributed representation at the single-character level.

5.2 Evaluation methodology

A serious obstacle to training systems well to output variable length sequences of words is the flaws of the available performance metrics. In the case of captioning or translation, there maybe be multiple correct translations. Further,

a labeled dataset may contain multiple *reference translations* for each example. Comparing against such a gold standard is more fraught than applying standard performance measure to binary classification problems.

One commonly used metric for structured natural language output with multiple references is *BLEU* score. Developed in 2002, *BLEU* score is related to modified unigram precision [Papineni et al., 2002]. It is the geometric mean of the n-gram precisions for all values of n between 1 and some upper limit N. In practice, 4 is a typical value for N, shown to maximize agreement with human raters. Because precision can be made high by offering excessively short translations, the *BLEU* score includes a brevity penalty B. Where c is average the length of the candidate translations and r the average length of the reference translations, the brevity penalty is

$$B = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}.$$

Then the *BLEU* score is

$$BLEU = B \cdot \exp\left(\frac{1}{N}\sum_{n=1}^{N} \log p_n\right)$$

where p_n is the modified n-gram precision, which is the number of n-grams in the candidate translation that occur in any of the reference translations, divided by the total number of n-grams in the candidate translation. This is called *modified* precision because it is an adaptation of precision to the case of multiple references.

BLEU scores are commonly used in recent papers to evaluate both translation and captioning systems. While *BLEU* score does appear highly correlated with human judgments, there is no guarantee that any given translation with a higher *BLEU* score is superior to another which receives a lower *BLEU* score. In fact, while *BLEU* scores tend to be correlated with human judgement across large sets of translations, they are not accurate predictors of human judgement at the single sentence level.

METEOR is an alternative metric intended to overcome the weaknesses of the *BLEU* score [Banerjee and Lavie, 2005]. *METEOR* is based on explicit word to word matches between candidates and reference sentences. When multiple references exist, the best score is used. Unlike *BLEU*, *METEOR* exploits known synonyms and stemming. The first step is to compute an F-score

$$F_\alpha = \frac{P \cdot R}{\alpha \cdot P + (1-\alpha) \cdot R}$$

based on single word matches where P is the precision and R is the recall. The next step is to calculate a fragmentation penalty $M \propto c/m$ where c is the smallest number of *chunks* of consecutive words such that the words are adjacent in both the candidate and the reference, and m is the total number of matched unigrams yielding the score. Finally, the score is

$$METEOR = (1 - M) \cdot F_\alpha.$$

Empirically, this metric has been found to agree with human raters more than *BLEU* score. However, *METEOR* is less straightforward to calculate than *BLEU*. To replicate the *METEOR* score reported by another party, one must exactly replicate their stemming and synonym matching, as well as the calculations. Both metrics rely upon having the exact same set of reference translations.

Even in the straightforward case of binary classification, without sequential dependencies, commonly used performance metrics like F1 give rise to optimal thresholding strategies which may not accord with intuition about what should constitute good performance [Lipton et al., 2014]. Along the same lines, given that performance metrics such as the ones above are weak proxies for true objectives, it may be difficult to distinguish between systems which are truly stronger and those which most overfit the performance metrics in use.

5.3 Natural language translation

Translation of text is a fundamental problem in machine learning that resists solutions with shallow methods. Some tasks, like document classification, can be performed successfully with a bag-of-words representation that ignores word order. But word order is essential in translation. The sentences "Scientist killed by raging virus" and "Virus killed by raging scientist" have identical bag-of-words representations.

Sutskever et al. [2014] present a translation model using two multilayered LSTMs that demonstrates impressive performance translating from English to French. The first LSTM is used for *encoding* an input phrase from the source language and the second LSTM for *decoding* the output phrase in the target language. The model works according to the following procedure (Figure 14):

- The source phrase is fed to the encoding LSTM one word at a time, which does not output anything. The authors found that significantly better results are achieved when the input sentence is fed into the network in reverse order.
- When the end of the phrase is reached, a special symbol that indicates the beginning of the output sentence is sent to the decoding LSTM. Additionally, the decoding LSTM receives as input the final state of the first LSTM. The second LSTM outputs softmax probabilities over the vocabulary at each time step.
- At inference time, beam search is used to choose the most likely words from the distribution at each time step, running the second LSTM until the end-of-sentence (*EOS*) token is reached.

For training, the true inputs are fed to the encoder, the true translation is fed to the decoder, and loss is propagated back from the outputs of the decoder across the entire sequence to sequence model. The network is trained to maximize the likelihood of the correct translation of each sentence in the training set. At inference time, a left to right beam search is used to determine

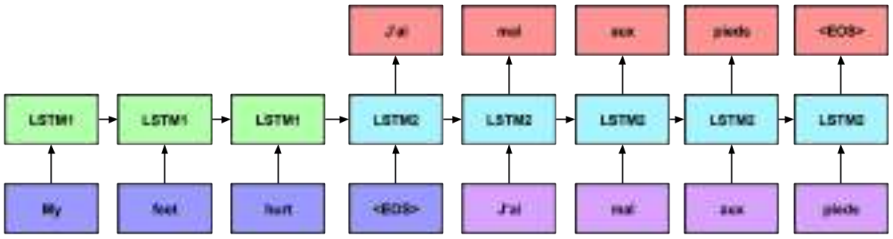


Figure 14: Sequence to sequence LSTM model of Sutskever et al. [2014]. The network consists of an encoding model (first LSTM) and a decoding model (second LSTM). The input blocks (blue and purple) correspond to word vectors, which are fully connected to the corresponding hidden state. Red nodes are softmax outputs. Weights are tied among all encoding steps and among all decoding time steps.

which words to output. A few among the most likely next words are chosen for expansion after each time step. The beam search ends when the network outputs an end-of-sentence (*EOS*) token. Sutskever et al. [2014] train the model using stochastic gradient descent without momentum, halving the learning rate twice per epoch, after the first five epochs. The approach achieves a *BLEU* score of 34.81, outperforming the best previous neural network NLP systems, and matching the best published results for non-neural network approaches, including systems that have explicitly programmed domain expertise. When their system is used to rerank candidate translations from another system, it achieves a *BLEU* score of 36.5.

The implementation which achieved these results involved eight GPUS. Nevertheless, training took 10 days to complete. One GPU was assigned to each layer of the LSTM, and an additional four GPUs were used simply to calculate softmax. The implementation was coded in C++, and each hidden layer of the LSTM contained 1000 nodes. The input vocabulary contained 160,000 words and the output vocabulary contained 80,000 words. Weights were initialized uniformly randomly in the range between -0.08 and 0.08.

Another RNN approach to language translation is presented by Auli et al. [2013]. Their RNN model uses the word embeddings of Mikolov and a lattice representation of the decoder output to facilitate search over the space of possible translations. In the lattice, each node corresponds to a sequence of words. They report a *BLEU* score of 28.5 on French-English translation tasks. Both papers provide results on similar datasets but Sutskever et al. [2014] only report on English to French translation while Auli et al. [2013] only report on French to English translation, so it is not possible to compare the performance of the two models.

5.4 Image captioning

Recently, recurrent neural networks have been used successfully for generating sentences that describe photographs [Vinyals et al., 2015, Karpathy and Fei-Fei, 2014, Mao et al., 2014]. In this task, a training set consists of input images $\boldsymbol{x}$ and target captions $\boldsymbol{y}$. Given a large set of image-caption pairs, a model is trained to predict the appropriate caption for an image.

Vinyals et al. [2015] follow up on the success in language to language translation by considering captioning as a case of image to language translation. Instead of both encoding and decoding with LSTMs, they introduce the idea of encoding an image with a convolutional neural network, and then decoding it with an LSTM. Mao et al. [2014] independently developed a similar RNN image captioning network, and achieved then state-of-the-art results on the Pascal, Flickr30K, and COCO datasets.

Karpathy and Fei-Fei [2014] follows on this work, using a convolutional network to encode images together with a bidirectional network attention mechanism and standard RNN to decode captions, using word2vec embeddings as word representations. They consider both full-image captioning and a model that captures correspondences between image regions and text snippets. At

inference time, their procedure resembles the one described by Sutskever et al. [2014], where sentences are decoded one word at a time. The most probable word is chosen and fed to the network at the next time step. This process is repeated until an *EOS* token is produced.

To convey a sense of the scale of these problems, Karpathy and Fei-Fei [2014] focus on three datasets of captioned images: Flickr8K, Flickr30K, and COCO, of size 50MB (8000 images), 200MB (30,000 images), and 750MB (328,000 images) respectively. The implementation uses the Caffe library [Jia et al., 2014], and the convolutional network is pretrained on ImageNet data. In a revised version, the authors report that LSTMs outperform simpler RNNs and that learning word representations from random initializations is often preferable to word2vec embeddings. As an explanation, they say that word2vec embeddings may cluster words like colors together in the embedding space, which can be not suitable for visual descriptions of images.

5.5 Further applications

Handwriting recognition is an application area where bidirectional LSTMs have been used to achieve state of the art results. In work by Liwicki et al. [2007] and Graves et al. [2009], data is collected from an interactive whiteboard. Sensors record the (x, y) coordinates of the pen at regularly sampled time steps. In the more recent paper, they use a bidirectional LSTM model, outperforming an HMM model by achieving 81.5% word-level accuracy, compared to 70.1% for the HMM.

In the last year, a number of papers have emerged that extend the success of recurrent networks for translation and image captioning to new domains. Among the most interesting of these applications are unsupervised video encoding [Srivastava et al., 2015], video captioning [Venugopalan et al., 2015], and program execution [Zaremba and Sutskever, 2014]. Venugopalan et al. [2015] demonstrate a sequence to sequence architecture that encodes frames from a video and decode words. At each time step the input to the encoding LSTM is the topmost hidden layer of a convolutional neural network. At decoding time, the network outputs probabilities over the vocabulary at each time step.

Zaremba and Sutskever [2014] experiment with networks which read computer programs one character at a time and predict their output. They focus on programs which output integers and find that for simple programs, including adding two nine-digit numbers, their network, which uses LSTM cells in several stacked hidden layers and makes a single left to right pass through the program, can predict the output with 99% accuracy.

6 Discussion

Over the past thirty years, recurrent neural networks have gone from models primarily of interest for cognitive modeling and computational neuroscience, to

powerful and practical tools for large-scale supervised learning from sequences. This progress owes to advances in model architectures, training algorithms, and parallel computing. Recurrent networks are especially interesting because they overcome many of the restrictions placed on input and output data by traditional machine learning approaches. With recurrent networks, the assumption of independence between consecutive examples is broken, and hence also the assumption of fixed-dimension inputs and outputs.

While LSTMs and BRNNs have set records in accuracy on many tasks in recent years, it is noteworthy that advances come from novel architectures rather than from fundamentally novel algorithms. Therefore, automating exploration of the space of possible models, for example via genetic algorithms or a Markov chain Monte Carlo approach, could be promising. Neural networks offer a wide range of transferable and combinable techniques. New activation functions, training procedures, initializations procedures, etc. are generally transferable across networks and tasks, often conferring similar benefits. As the number of such techniques grows, the practicality of testing all combinations diminishes. It seems reasonable to infer that as a community, neural network researchers are exploring the space of model architectures and configurations much as a genetic algorithm might, mixing and matching techniques, with a fitness function in the form of evaluation metrics on major datasets of interest.

This inference suggests two corollaries. First, as just stated, this body of research could benefit from automated procedures to explore the space of models. Second, as we build systems designed to perform more complex tasks, we would benefit from improved fitness functions. A *BLEU* score inspires less confidence than the accuracy reported on a binary classification task. To this end, when possible, it seems prudent to individually test techniques first with classic feedforward networks on datasets with established benchmarks before applying them to recurrent networks in settings with less reliable evaluation criteria.

Lastly, the rapid success of recurrent neural networks on natural language tasks leads us to believe that extensions of this work to longer texts would be fruitful. Additionally, we imagine that dialogue systems could be built along similar principles to the architectures used for translation, encoding prompts and generating responses, while retaining the entirety of conversation history as contextual information.

7 Acknowledgements

The first author's research is funded by generous support from the Division of Biomedical Informatics at UCSD, via a training grant from the National Library of Medicine. This review has benefited from insightful comments from Vineet Bafna, Julian McAuley, Balakrishnan Narayanaswamy, Stefanos Poulis, Lawrence Saul, Zhuowen Tu, and Sharad Vikram.

RotLSTM: rotating memories in Recurrent Neural Networks

Vlad Velici
School of Electronics and Computer Science
University of Southampton
Southampton, UK

Adam Prügel-Bennett
School of Electronics and Computer Science
University of Southampton
Southampton, UK

Abstract

Long Short-Term Memory (LSTM) units have the ability to memorise and use long-term dependencies between inputs to generate predictions on time series data. We introduce the concept of modifying the cell state (memory) of LSTMs using rotation matrices parametrised by a new set of trainable weights. This addition shows significant increases of performance on some of the tasks from the bAbI dataset.

1 Introduction

In the recent years, Recurrent Neural Networks (RNNs) have been successfully used to tackle problems with data that can be represented in the shape of time series. Application domains include Natural Language Processing (NLP) (translation (Rosca & Breuel, 2016), summarisation (Nallapati et al., 2016), question answering and more), speech recogition (Hannun et al., 2014; Graves et al., 2013), text to speech systems (Arik et al., 2017), computer vision tasks (Stewart et al., 2016; Wu et al., 2017), and differentiable programming language interpreters (Riedel et al., 2016; Rocktäschel & Riedel, 2017).

An intuitive explanation for the success of RNNs in fields such as natural language understanding is that they allow words at the beginning of a sentence or paragraph to be memorised. This can be crucial to understanding the semantic content. Thus in the phrase *"The cat ate the fish"* it is important to memorise the subject (cat). However, often later words can change the meaning of a senstence in subtle ways. For example, *"The cat ate the fish,* ***didn't it****"* changes a simple statement into a question. In this paper, we study a mechanism to enhance a standard RNN to enable it to modify its memory, with the hope that this will allow it to capture in the memory cells sequence information using a shorter and more robust representation.

One of the most used RNN units is the Long Short-Term Memory (LSTM) (Hochreiter & Schmidhuber, 1997). The core of the LSTM is that each unit has a *cell state* that is modified in a gated fashion at every time step. At a high level, the cell state has the role of providing the neural network with *memory* to hold long-term relationships between inputs. There are many small variations of LSTM units in the literature and most of them yield similar performance (Greff et al., 2017).

The memory (cell state) is expected to encode information necessary to make the next prediction. Currently the ability of the LSTMs to rotate and swap memory positions is limited to what can be achieved using the available gates. In this work we introduce a new operation on the memory that explicitly enables rotations and swaps of pairwise memory elements. Our preliminary tests show performance improvements on some of the bAbI tasks (Weston et al., 2015) compared with LSTM based architectures.

2 Related work

Adding rotations to RNN cells has been previously studied in Henaff et al. (2016). They do compare their results to LSTM models but do not directly add a new rotation gates to the LSTM like in our work. Their results suggest that rotations do not generalise well for multiple tasks.

Using unitary constraints for weight matrices in RNN units is shown in Arjovsky et al. (2015), where they use the weights in the complex domain (they show a way to only use real numbers in implementation). The EURNN (Jing et al., 2017b) presents a more efficient method of parametrising a weights matrix such that it is unitary and uses this parametrisation to add unitary constraints to weights of RNN cells. A similar work is GORU (Jing et al., 2017a) which adds the same unitary constraints on one of the gate weights in the GRU cell.

A new RNN cell, the Rotational Unit of Memory (RUM) (Dangovski et al., 2017), adds a rotation operation in a gated RNN cell. The rotation parametrisation is passed through to the next cell, similar to how the memory state of an LSTM is. They show increased performance on the bAbI tasks when compared to LSTM and GORU based models.

3 The rotation gate

In this section we introduce the idea of adding a new set of parameters for the RNN cell that enable rotation of the cell state. The following subsection shows how this is implemented in the LSTM unit.

One of the key innovations of LSTMs was the introduction of gated modified states so that if the gate neuron i is saturated then the memory $c_i(t-1)$ would be unaltered. That is, $c_i(t-1) \approx c_i(t)$ with high accuracy. The fact that the amplification factor is very close to 1 prevents the memory vanishing or exploding over many epochs.

To modify the memory, but retain an amplification factor of 1 we take the output after appling the forget and add gates (we call it $\mathbf{d_t}$), and apply a rotation matrix $\mathbf{U}$ to obtain a modified memory $\mathbf{c}_t = \mathbf{U}\mathbf{d}_t$. Note that, for a rotation matrix $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ so that $\|\mathbf{d_t}\| = \|\mathbf{c_t}\|$.

We parametrise the rotation by a vector of angles

$$\mathbf{u} = 2\pi\sigma(\mathbf{W}_{\text{rot}}\mathbf{x} + \mathbf{b}_{\text{rot}}), \tag{1}$$

where $\mathbf{W}_{\text{rot}}$ is a weight matrix and $\mathbf{b}_{\text{rot}}$ is a bias vector which we learn along with the other parameters. $\mathbf{x}$ is the vector of our concatenated inputs (in LSTMs given by concatenating the input for the current timestep with the output from the previous time step).

A full rotation matrix is parametrisable by $n(n-1)/2$ parameters (angles). Using all of these would introduce a huge number of weights, which is likely to over-fit. Instead, we have limited ourselves to considering rotations between pairs of inputs $d_i(t)$ and $d_{i+1}(t)$.

Our rotation matrix is a block-diagonal matrix of 2D rotations

$$\mathbf{U}(\mathbf{u}) = \begin{bmatrix} \cos u_1 & -\sin u_1 & & & \\ \sin u_1 & \cos u_1 & & & \\ & & \ddots & & \\ & & & \cos u_{n/2} & -\sin u_{n/2} \\ & & & \sin u_{n/2} & \cos u_{n/2} \end{bmatrix}, \tag{2}$$

where the cell state is of size n. Our choice of rotations only needs $n/2$ angles.

3.1 RotLSTM

In this section we show how to add memory rotation to the LSTM unit. The rotation is applied after the forget and add gates and before using the current cell state to produce an output.

The RotLSTM equations are as follows:

$$\mathbf{x} = [\mathbf{h_{t-1}}, \mathbf{x_t}], \tag{3}$$
$$\mathbf{f_t} = \sigma(\mathbf{W}_\mathrm{f}\mathbf{x} + \mathbf{b}_\mathrm{f}), \tag{4}$$
$$\mathbf{i_t} = \sigma(\mathbf{W}_\mathrm{i}\mathbf{x} + \mathbf{b}_\mathrm{i}), \tag{5}$$
$$\mathbf{o_t} = \sigma(\mathbf{W}_\mathrm{o}\mathbf{x} + \mathbf{b}_\mathrm{o}), \tag{6}$$
$$\mathbf{u_t} = 2\pi\sigma(\mathbf{W}_\mathrm{rot}\mathbf{x} + \mathbf{b}_\mathrm{rot}), \tag{7}$$
$$\mathbf{d_t} = \mathbf{f_t} \circ \mathbf{c_{t-1}} + \mathbf{i_t} \circ \tanh(\mathbf{W}_\mathrm{c}\mathbf{x} + \mathbf{b}_\mathrm{c}), \tag{8}$$
$$\mathbf{c_t} = \mathbf{U}(\mathbf{u_t})\mathbf{d_t}, \tag{9}$$
$$\mathbf{h_t} = \mathbf{o_t} \circ \tanh(\mathbf{c_t}), \tag{10}$$

where $\mathbf{W}_{\{\mathrm{f,i,o,rot,c}\}}$ are weight matrices, $\mathbf{b}_{\{\mathrm{f,i,o,rot,c}\}}$ are biases ($\mathbf{W}$s and $\mathbf{b}$s learned during training), $\mathbf{h_{t-1}}$ is the previous cell output, $\mathbf{h_t}$ is the output the cell produces for the current timestep, similarly $\mathbf{c_{t-1}}$ and $\mathbf{c_t}$ are the cell states for the previous and current timestep, $\circ$ is element-wise multiplication and $[\cdot, \cdot]$ is concatenation. $\mathbf{U}$ as defined in Equation 2, parametrised by $\mathbf{u_t}$. Figure 1 shows a RotLSTM unit in detail.

Assuming cell state size n, input size m, the RotLSTM has $n(n+m)/2$ extra parameters, a 12.5% increase (ignoring biases). Our expectation is that we can decrease n without harming performance and the rotations will enforce a better representation for the cell state.

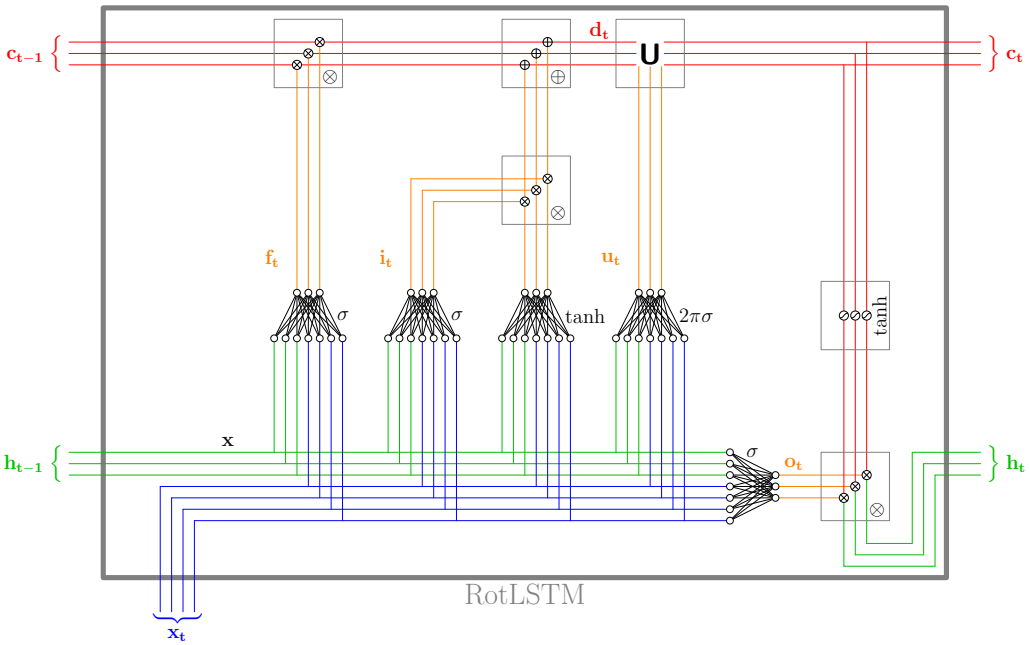


Figure 1: RotLSTM diagram. $\mathbf{x}$ is the concatenation of $\mathbf{h_{t-1}}$ and $\mathbf{x_t}$ in the diagram (green and blue lines). Note that this differs from a regular LSTM by the introduction of the network producing angles $\mathbf{u_t}$ and the rotation module marked $\mathbf{U}$. In the diagram input size is 4 and cell state size is 3.

4 Experiments and results

To empirically evaluate the performance of adding the rotation gate to LSTMs we use the toy NLP dataset bAbI with 1000 samples per task. The bAbI dataset is composed of 20 different tasks of various difficulties, starting from easy questions based on a single supporting fact (for example: *John is in the kitchen. Where is John? A: Kitchen*) and going to more difficult tasks of reasoning about size (example: *The football fits in the suitcase. The box is smaller than the football. Will the box fit in the suitcase? A: yes*) and path finding (example: *The bathroom is south of the office. The bathroom is north of the hallway. How do you go from the hallway to the office? A: north, north*). A summary of

Table 1: bAbI dataset tasks.

#	Description	#	Description
1	Factoid QA with one supporting fact	**11**	Basic coreference
2	Factoid QA with two supporting facts	**12**	Conjunction
3	Factoid QA with three supporting facts	**13**	Compound coreference
4	Two argument relations: subject vs. object	**14**	Time manipulation
5	Three argument relations	**15**	Basic deduction
6	Yes/No questions	**16**	Basic induction
7	Counting	**17**	Positional reasoning
8	Lists/Sets	**18**	Reasoning about size
9	Simple Negation	**19**	Path finding
10	Indefinite Knowledge	**20**	Reasoning about agent's motivation

all tasks is available in Table 1. We are interested in evaluating the behaviour and performance of rotations on RNN units rather than beating state of the art.

We compare a model based on RotLSTM with the same model based on the traditional LSTM. All models are trained with the same hyperparameters and we do not perform any hyperparameter tuning apart from using the sensible defaults provided in the Keras library and example code (Chollet et al., 2015).

For the first experiment we train a LSTM and RotLSTM based model 10 times using a fixed cell state size of 50. In the second experiment we train the same models but vary the cell state size from 6 to 50 to assess whether the rotations help our models achieve good performance with smaller state sizes. We only choose even numbers for the cell state size to make all units go through rotations.

4.1 The model architecture

The model architecture, illustrated in Figure 2, is based on the Keras example implementation[1]. This model architecture, empirically, shows better performance than the LSTM baseline published in Weston et al. (2015). The input question and sentences are passed thorugh a word embedding layer (not shared, embeddings are different for questions and sentences). The question is fed into an RNN which produces a representation of the question. This representation is concatenated to every word vector from the story, which is then used as input to the second RNN. Intuitively, this helps the second RNN (Query) to *focus* on the important words to answer the question. The output of the second RNN is passed to a fully connected layer with a softmax activation of the size of the dictionary. The answer is the word with the highest activation.

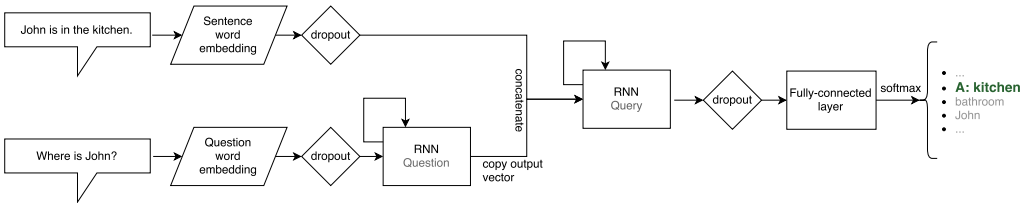


Figure 2: Model architecture for the bAbI dataset. The RNN is either LSTM or RotLSTM.

The categorical cross-entropy loss function was used for training. All dropout layers are dropping 30% of the nodes. The train-validation dataset split used was 95%-5%. The optimizer used was Adam with learning rate 0.001, no decay, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. The training set was randomly shuffled before every epoch. All models were trained for 40 epochs. After every epoch the model performance was evaluated on the validation and training sets, and every 10 epochs on the test set. We set the random seeds to the same number for reproducibility and ran

[1] Available at `https://goo.gl/9wfzr5`.

Table 2: Performance comparison on the bAbI dataset. Values are % average accuracy on test set ±standard deviation taken from training each model 10 times. For each of the trained models the test accuracy is taken for the epoch with the best validation accuracy (picked from epoch numbers 1, 11, 21, 31, 40 since we only evaluated on the test set for those).

Task:	1	2	3	4	5	6	7	8	9	10
LSTM	49.8 ±3.3	26.5 ±3.8	21.1 ±1.0	63.7 ±9.4	33.6 ±6.2	49.2 ±0.8	62.7 ±15.5	**68.8 ±8.3**	**63.9 ±0.2**	45.2 ±2.0
RotLSTM	**52.3 ±1.1**	**27.1 ±1.3**	**22.4 ±1.3**	**65.8 ±3.6**	**55.7 ±5.5**	**50.1 ±1.9**	**76.7 ±3.0**	66.1 ±6.2	61.5 ±1.9	**48.1 ±1.1**
Task:	11	12	13	14	15	16	17	18	19	20
LSTM	73.6 ±2.2	74.3 ±1.7	94.4 ±0.2	**20.7 ±3.3**	21.4 ±0.5	**48.0 ±2.2**	48.0 ±0.0	70.3 ±20.8	8.5 ±0.6	87.7 ±4.2
RotLSTM	**73.9 ±1.2**	**76.5 ±1.1**	**94.4 ±0.0**	19.9 ±2.0	**28.8 ±8.8**	46.7 ±2.0	**54.2 ±3.1**	**90.5 ±0.9**	**8.9 ±1.1**	**89.9 ±2.4**

the experiments 10 times with 10 different random seeds. The source code is available at `https://github.com/vladvelici/swaplstm`.

4.2 Results

In this subsection we compare the the performance of models based on the LSTM and RotLSTM units on the bAbI dataset.

Applying rotations on the unit memory of the LSTM cell gives a slight improvement in performance overall, and significant improvements on specific tasks. Results are shown in Table 2. The most significant improvements are faster convergence, as shown in Figure 3, and requiring smaller state sizes, illustrated in Figure 4.

On tasks 1 (basic factoid), 11 (basic coreference), 12 (conjunction) and 13 (compound coreference) the RotLSTM model reaches top performance a couple of epochs before the LSTM model consistently. The RotLSTM model also needs a smaller cell state size, reaching top performance at state size 10 to 20 where the LSTM needs 20 to 30. The top performance is, however, similar for both models, with RotLSTM improving the accuracy with up to 2.5%.

The effect is observed on task 18 (reasoning about size) at a greater magnitude where the RotLSTM reaches top performance before epoch 20, after which it plateaus, while the LSTM model takes 40 epochs to fit the data. The training is more stable for RotLSTM and the final accuracy is improved by 20%. The RotLSTM reaches top performance using cell state 10 and the LSTM needs size 40. Similar performance increase for the RotLSTM (22.1%) is observed in task 5 (three argument relations), reaching top performance around epoch 25 and using a cell state of 50. Task 7 (counting) shows a similar behaviour with an accuracy increase of 14% for RotLSTM.

Tasks 4 (two argument relations) and 20 (agent motivation) show quicker learning (better performance in the early epochs) for the RotLSTM model but both models reach their top performance after the same amount of traning. On task 20 the RotLSTM performance reaches top accuracy using state size 10 while the LSTM incremetally improves until using state size 40 to 50.

Signs of overfitting for the RotLSTM model can be observed more prominently than for the LSTM model on tasks 15 (basic deduction) and 17 (positional reasoning).

Our models, both LSTM and RotLSTM, perform poorly on tasks 2 and 3 (factoid questions with 2 and 3 supporting facts, respectively) and 14 (time manipulation). These problem classes are solved very well using models that look over the input data many times and use an attention mechanism that allows the model to focus on the relevant input sentences to answer a question (Sukhbaatar et al., 2015; Kumar et al., 2016). Our models only look at the input data once and we do not filter out irrelevant information.

5 RotGRU

A popular and successful Recurrent Neural Network (RNN) cell is the Gated Recurrent Unit (GRU) unit (Cho et al., 2014). Unlike the LSTM, the GRU unit only has a cell state that can be interpreted as output when needed. Intuitively we want the transformation matrix to enable the unit to learn more compact, richer representations. The rotation is inserted at the *remember* ($\mathbf{r}$) gate to avoid applying a

Table 3: Performance comparison on the bAbI dataset. Values are % average accuracy on test set ±standard deviation taken from training each model 10 times. For each of the trained models the test accuracy is taken for the epoch with the best validation accuracy (picked from epoch numbers 1, 11, 21, 31, 40 since we only evaluated on the test set for those).

Task:	1	2	3	4	5	6	7	8	9	10
GRU	**51.3 ±0.9**	34.6 ±3.8	**21.5 ±1.4**	69.8 ±1.9	67.1 ±5.7	49.9 ±1.0	**79.2 ±0.5**	**76.3 ±1.2**	62.6 ±0.7	48.2 ±1.1
RotGRU	50.2 ±1.3	**35.7 ±1.9**	21.3 ±1.1	**70.8 ±1.2**	**73.4 ±2.6**	**50.1 ±1.1**	78.4 ±2.6	75.9 ±2.0	**62.8 ±0.4**	**48.4 ±1.0**
Task:	11	12	13	14	15	16	17	18	19	20
GRU	77.7 ±3.3	**76.8 ±0.8**	**94.4 ±0.0**	**31.4 ±5.6**	**27.8 ±6.7**	**48.0 ±1.3**	**48.9 ±1.7**	**91.1 ±0.5**	**8.9 ±0.8**	92.3 ±1.9
RotGRU	**79.2 ±2.6**	76.7 ±0.7	94.3 ±0.1	29.9 ±4.7	23.3 ±4.1	47.1 ±3.7	48.6 ±1.6	90.9 ±0.8	8.9 ±1.0	**93.6 ±2.9**

transformation directly on the outputs. The RotGRU equations are the following:

$$\mathbf{x} = [\mathbf{h_{t-1}}, \mathbf{x_t}], \tag{11}$$
$$\mathbf{z_t} = \sigma(\mathbf{W}_z\mathbf{x} + \mathbf{b}_z), \tag{12}$$
$$\mathbf{d_t} = \mathbf{h_{t-1}} \circ \sigma(\mathbf{W}_r\mathbf{x} + \mathbf{b}_r), \tag{13}$$
$$\mathbf{U} = 2\pi\sigma(\mathbf{W}_{\text{rot}}\mathbf{x} + \mathbf{b}_{\text{rot}}), \tag{14}$$
$$\mathbf{r_t} = \mathbf{U}\mathbf{d_t} \tag{15}$$
$$\hat{\mathbf{h}}_\mathbf{t} = \tanh(\mathbf{W}_h[\mathbf{r_t}, \mathbf{x_t}] + \mathbf{b}_h), \tag{16}$$
$$\mathbf{h_t} = (1 - \mathbf{z_t}) \circ \mathbf{h_{t-1}} + \mathbf{z_t} \circ \hat{\mathbf{h}}_\mathbf{t}. \tag{17}$$

We show the results of training a RotGRU and a GRU model on the bAbI tasks in Table 3 and Figure 5. Notice that models based on the GRU cell perform better than the LSTM and RotLSTM cells. From this experiment we also observe that RotGRU performs the same or slightly worse than the initial GRU cell. The RotGRU also requires more computation and introduces more parameters, as a result it takes longer (wall clock) time to train and run for inference.

6 Discussion

A limitation of the models in our experiments is only applying pairwise 2D rotations. Representations of past input can be larger groups of the cell state vector, thus 2D rotations might not fully exploit the benefits of transformations. Rotating groups of elements and multi-dimensional rotations was not investigated, but could potentially also force the models to learn a more structured representation of the world, similar to how forcing a model to learn specific representations of scenes, as presented in Higgins et al. (2017), yields semantic representations of the scene.

In this work we presented preliminary tests for adding rotations to simple models but we only used a toy dataset. The bAbI dataset has certain advantages such as being small thus easy to train many models on a single machine, not having noise as it is generated from a simulation, and having a wide range of tasks of various difficulties. However it is a toy dataset that has a very limited vocabulary and lacks the complexity of real world datasets (noise, inconsistencies, larger vocabularies, more complex language constructs, and so on).

A brief exploration of the angles produced by $\mathbf{u}$ and the weight matrix $\mathbf{W}_{\text{rot}}$ show that $\mathbf{u}$ does not saturate, thus rotations are in fact applied to our cell states and do not converge to 0 (or 360 degress). A more in-depth qualitative analysis of the rotation gate might give a better insight into what the rotations are actually achieving. Peeking into the activations of our rotation gates could help understand the behaviour of rotations and to what extent they help (or do not) better represent long-term memory.

A very successful and popular mutation of the LSTM is the GRU unit (Cho et al., 2014). The GRU only has an output as opposed to both a cell state and an output and uses fewer gates. We showed that adding similar rotations to GRU cells does not show the same performance improvements, and that the baseline GRU model outperforms both LSTM and RotLSTM models in most cases.

7 Conclusion

We have introduced a novel gating mechanism for RNN units that enables applying a parametrised transformation matrix to the cell state. We picked pairwise 2D rotations as the transformation and shown how this can be added to the popular LSTM units to create what we call RotLSTM.

We trained a simple model using RotLSTM units and compared them with the same model based on LSTM units. We show that for the LSTM-based architectures adding rotations has a positive impact on most bAbI tasks, making the training require fewer epochs to achieve similar or higher accuracy. On some tasks the RotLSTM model can use a lower dimensional cell state vector and maintain its performance.

Significant accuracy improvements of approximatively 20% for the RotLSTM model over the LSTM model are visible on bAbI tasks 5 (three argument relations) and 18 (reasoning about size).

The same improvements could not be replicated into the GRU cell when adding our rotation gate, and models based on the GRU cell outperform both the LSTM and RotLSTM on most tasks.

References

Sercan O Arik, Mike Chrzanowski, Adam Coates, Gregory Diamos, Andrew Gibiansky, Yongguo Kang, Xian Li, John Miller, Jonathan Raiman, Shubho Sengupta, et al. Deep voice: Real-time neural text-to-speech. *arXiv preprint arXiv:1702.07825*, 2017.

Martín Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. *CoRR*, abs/1511.06464, 2015. URL http://arxiv.org/abs/1511.06464.

Kyunghyun Cho, Bart Van Merriënboer, Çaglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

François Chollet et al. Keras. https://github.com/fchollet/keras, 2015.

Rumen Dangovski, Li Jing, and Marin Soljacic. Rotational unit of memory. *CoRR*, abs/1710.09537, 2017. URL http://arxiv.org/abs/1710.09537.

Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *Acoustics, speech and signal processing (icassp), 2013 ieee international conference on*, pp. 6645–6649. IEEE, 2013.

Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. Lstm: A search space odyssey. *IEEE transactions on neural networks and learning systems*, 2017.

Awni Y. Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, and Andrew Y. Ng. Deep speech: Scaling up end-to-end speech recognition. *CoRR*, abs/1412.5567, 2014. URL http://arxiv.org/abs/1412.5567.

Mikael Henaff, Arthur Szlam, and Yann LeCun. Orthogonal rnns and long-memory tasks. *CoRR*, abs/1602.06662, 2016. URL http://arxiv.org/abs/1602.06662.

Irina Higgins, Nicolas Sonnerat, Loic Matthey, Arka Pal, Christopher P Burgess, Matthew Botvinick, Demis Hassabis, and Alexander Lerchner. Scan: Learning abstract hierarchical compositional visual concepts. *arXiv preprint arXiv:1707.03389*, 2017.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

Li Jing, Çaglar Gülçehre, John Peurifoy, Yichen Shen, Max Tegmark, Marin Soljacic, and Yoshua Bengio. Gated orthogonal recurrent units: On learning to forget. *CoRR*, abs/1706.02761, 2017a. URL http://arxiv.org/abs/1706.02761.

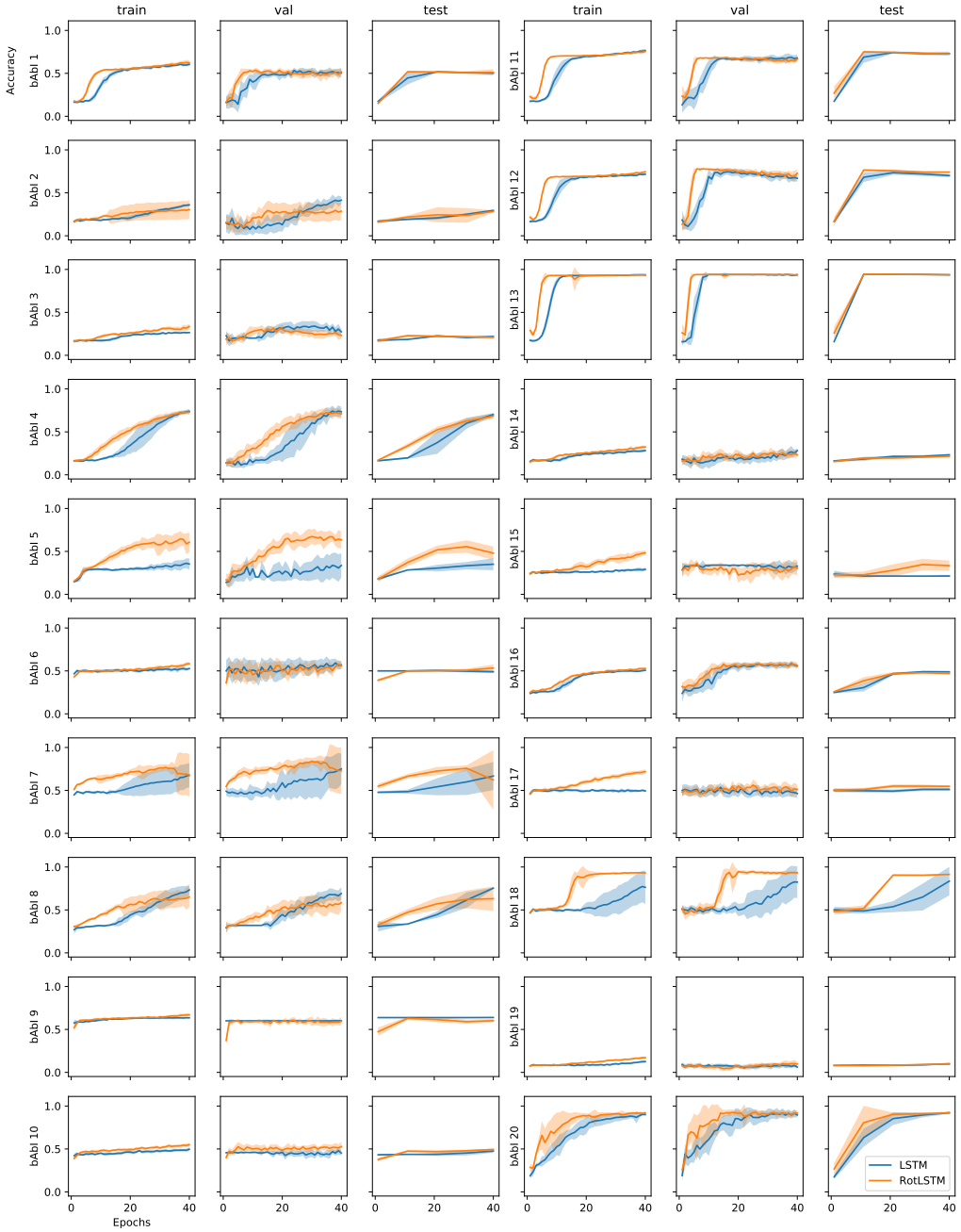


Figure 3: Accuracy comparison on training, validation (val) and test sets over 40 epochs for LSTM and RotLSTM models. The models were trained 10 times and shown is the average accuracy and in faded colour is the standard deviation. Test set accuracy was computed every 10 epochs.

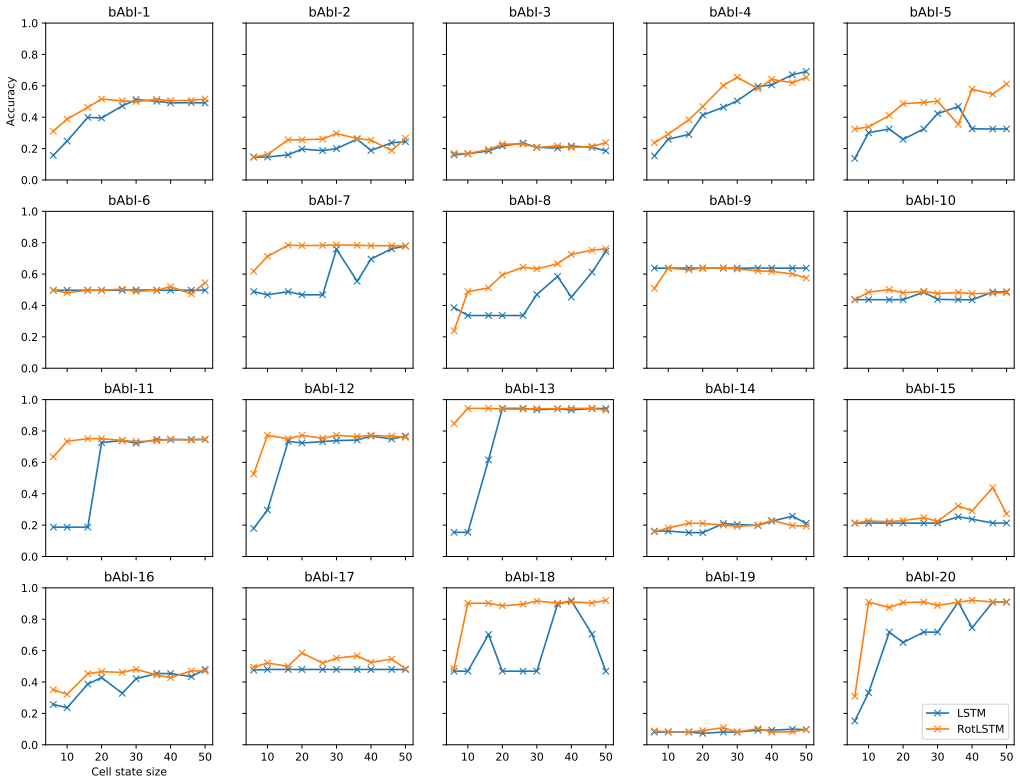


Figure 4: Accuracy on the test set for the LSTM and RotLSTM while varying the cell state size from 6 to 50. The shown numbers are for the epochs with best validation set accuracy.

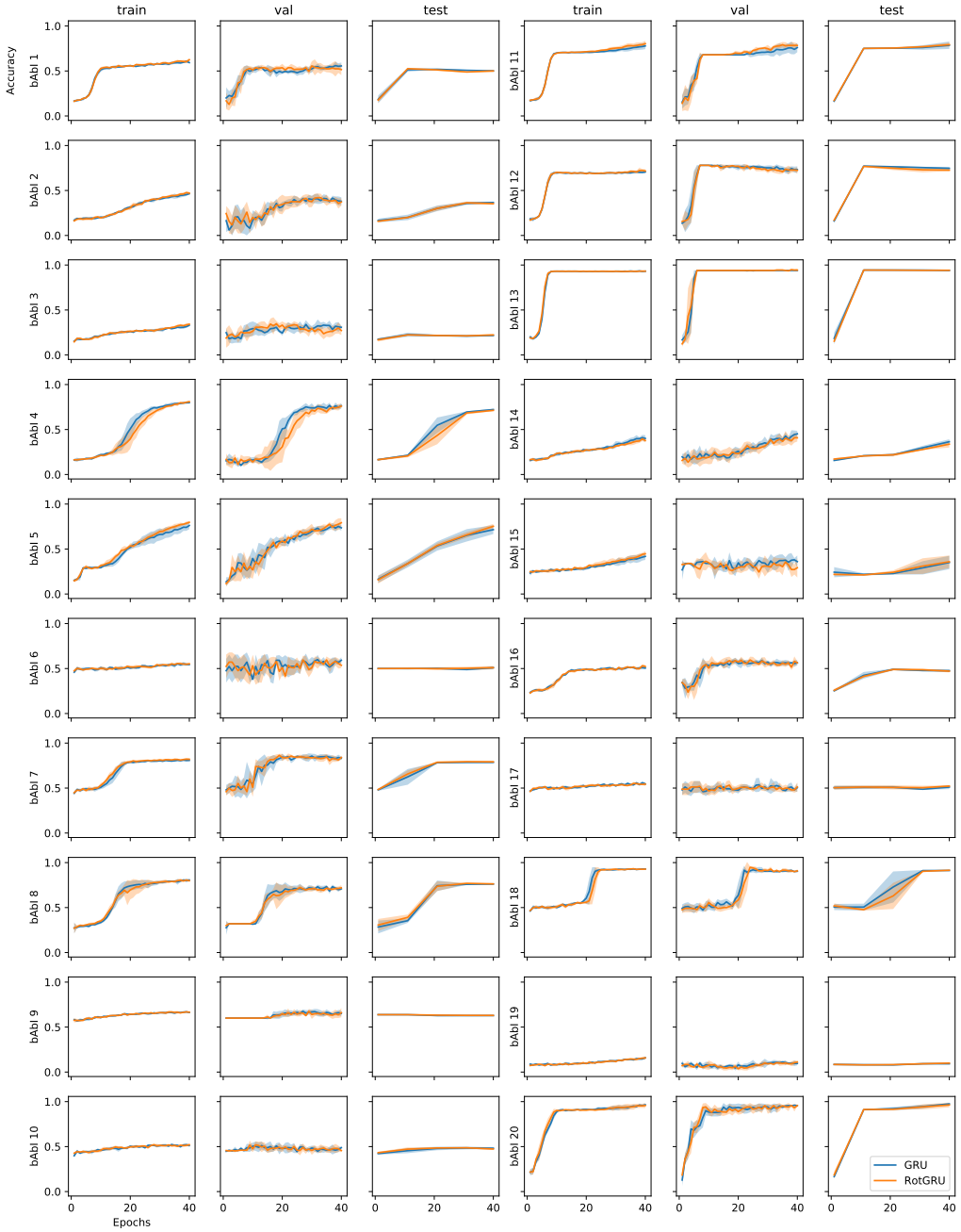


Figure 5: Accuracy comparison on training, validation (val) and test sets over 40 epochs for GRU and RotGRU models. The models were trained 5 times and shown is the average accuracy and in faded colour is the standard deviation. Test set accuracy was computed every 10 epochs.

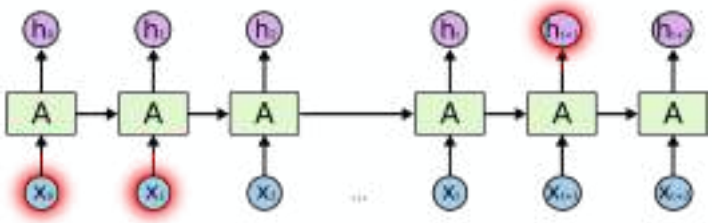
Tunable Efficient Unitary Neural Networks (EUNN) and their application to RNNs

Li Jing*, Yichen Shen*, Tena Dubček, John Peurifoy, Scott Skirlo,
Yann LeCun, Max Tegmark, Marin Soljačić

* equal contribution

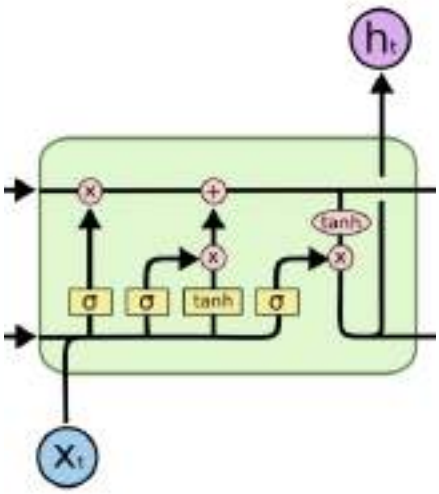
Massachusetts Institute of Technology

Gradient Vanishing/Explosion Problem



- During backpropagation through time, hidden to hidden Jacobin matrix is multiplied multiple times.
- Gradient vanishing/explosion makes RNN hard to train

Conventional Solution: LSTM



- Practically, gradient clipping is required
- slow to learn long term dependency

New Solution: Unitary/Orthogonal RNN

Unitary/Orthogonal matrices keep the norm of vectors: $\|\boldsymbol{U}\boldsymbol{x}\| = \|\boldsymbol{x}\|$

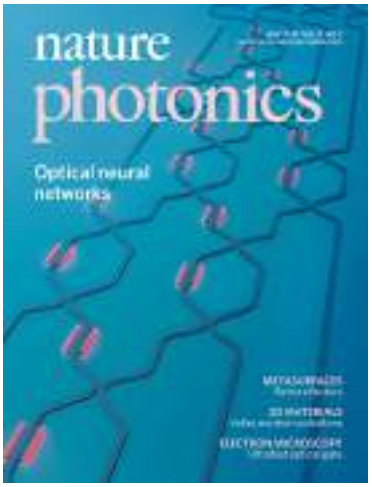
By enforcing hidden to hidden transition matrix to be unitary/ orthogonal, no matter how many time steps are propagated, the norm of the gradient will stay the same

$$\left\| \prod_{k=t}^{T-1} \frac{\partial \mathbf{h}^{(k+1)}}{\partial \mathbf{h}^{(k)}} \right\| \sim 1$$

Related Works

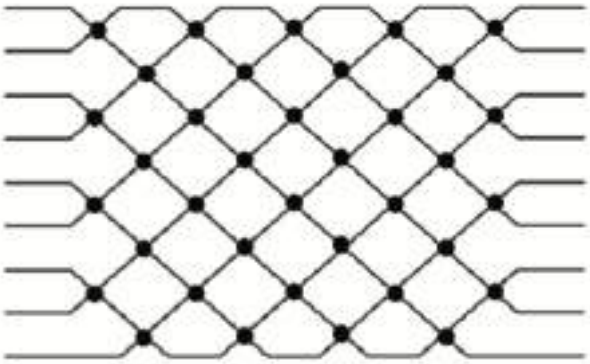
- Restricted-capacity Unitary Matrix Parametrization (Arjovsky, ICML 2016)
- Full-capacity Unitary Matrix by projection (Wisdom, NIPS 2016)
- Orthogonal Matrix Parametrization by reflection (Mhammedi, ICML 2017)
- Orthogonal Matrix by regularization (Vorontsov, ICML 2017)

Efficient Unitary Matrix Parametrization



(Y. Shen, et al, Nature Photonics 2017)

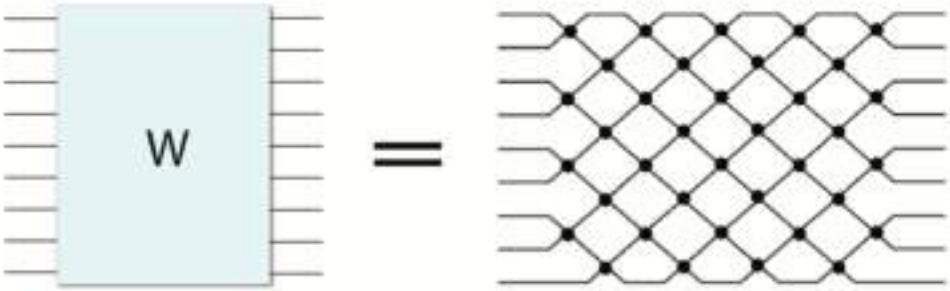
Physical system inspired:



Efficient Unitary Matrix Parametrization

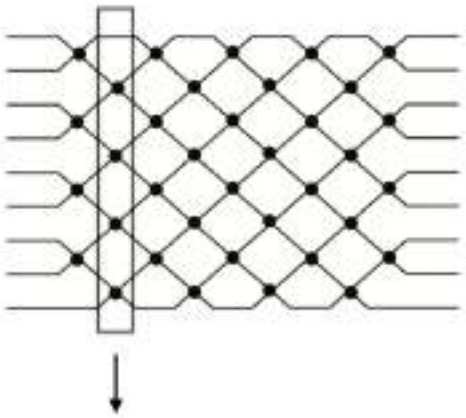
$$= \begin{pmatrix} e^{i\phi}\cos\theta & -e^{i\phi}\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

2-by-2 element



- SU(2) element: rotation matrix + relevant phase
- strict, complete, unique
- Full unitary space

Efficient Unitary Matrix Implementation



sparse block diagonal matrix

element-wise functions

Algorithm 1 Efficient implementation for $\mathbf{F}$ with parameter θ_i and ϕ_i.

Input: input $\mathbf{x}$, size N; parameters θ and ϕ, size $N/2$; constant permuatation index list $\mathbf{ind_1}$ and $\mathbf{ind_2}$.
Output: output $\mathbf{y}$, size N.

$\mathbf{v_1} \leftarrow$ concatenate($\cos\theta$, $\cos\theta * \exp(i\phi)$)
$\mathbf{v_2} \leftarrow$ concatenate($\sin\theta$, $-\sin\theta * \exp(i\phi)$)
$\mathbf{v_1} \leftarrow$ permute($\mathbf{v_1}, \mathbf{ind_1}$)
$\mathbf{v_2} \leftarrow$ permute($\mathbf{v_2}, \mathbf{ind_1}$)
$\mathbf{y} \leftarrow \mathbf{v_1} * \mathbf{x} + \mathbf{v_2} *$ permute($\mathbf{x}, \mathbf{ind_2}$)

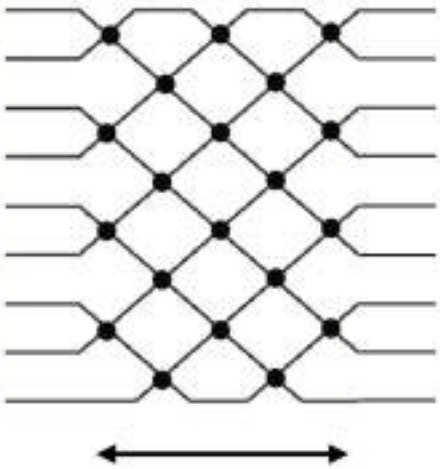
- Parallel, Sparse
- O(1) per parameter
- No need to customize gradient

Tensorflow: https://github.com/jingli9111/EUNN-tensorflow

PyTorch: https://github.com/jingli9111/URNN-PyTorc

Theano: https://github.com/iguanaus/EUNN-theano

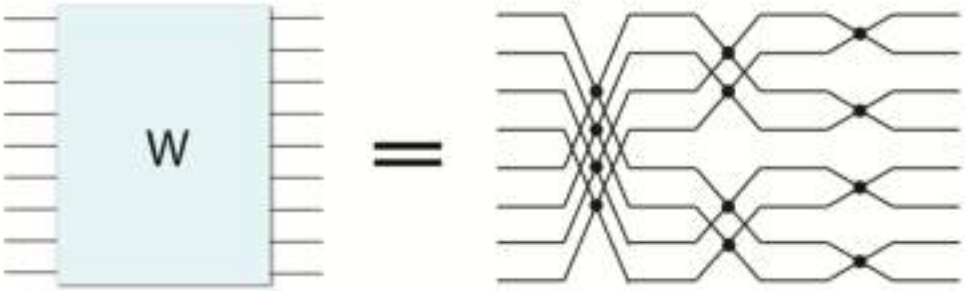
Tunable Parametrization



- reduce capacity allows larger hidden state
- reduce capacity increases training speed

FFT-style Parametrization

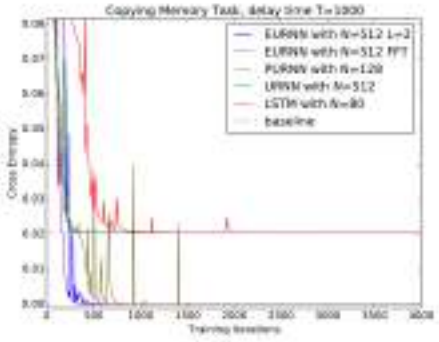
$$\begin{pmatrix} e^{i\phi}\cos\theta & -e^{i\phi}\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$



- nlog(n) parameters
- minimum number of parameters for symmetric unitary space
- almost same performance as Tunable style with large capacity

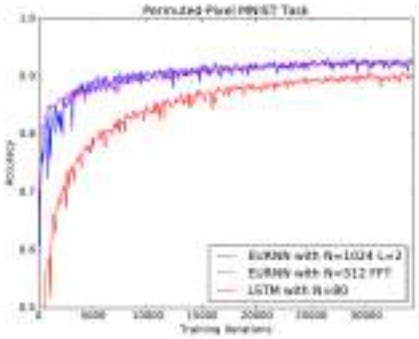
Experiment results

- Copying Memory



EURNN outperforms all other models for long delay time case.

- Pixel Permuted MNIST



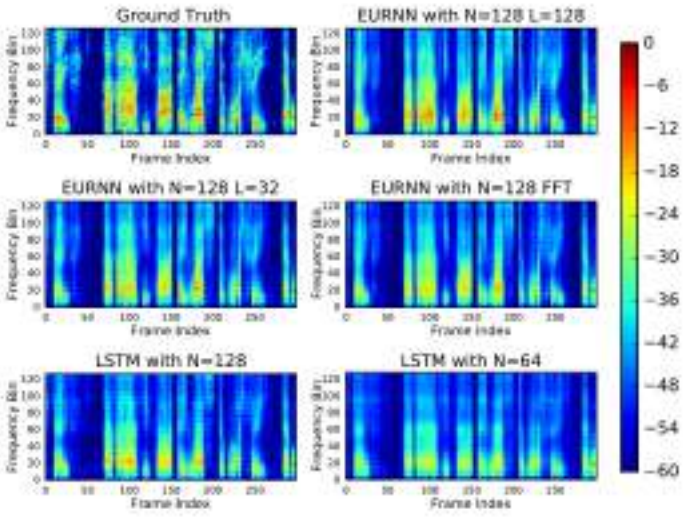
EURNN outperforms LSTM in both final performance and training speed.

EURNN tunable-style achieves highest accuracy with least number of parameters.

Model	hidden size (capacity)	number of parameters	validation accuracy	test accuracy
LSTM	80	16k	0.908	0.902
URNN	512	16k	**0.942**	0.933
PURNN	116	16k	0.922	0.921
EURNN (tunable style)	1024 (2)	13.3k	0.940	**0.937**
EURNN (FFT style)	512 (FFT)	9.0k	0.928	0.925

Experiment results

- Speech spectrum prediction



TIMIT spectrum sampled in 8 GHz. RNN model is required to predict next frame based on previous frames.

EURNN outperforms LSTM with less number of parameters

Model	hidden size (capacity)	number of parameters	MSE (validation)	MSE (test)
LSTM	64	33k	71.4	66.0
LSTM	128	98k	55.3	54.5
EURNN (tunable style)	128 (2)	33k	63.3	63.3
EURNN (tunable style)	128 (32)	35k	52.3	52.7
EURNN (tunable style)	128 (128)	41k	**51.8**	**51.9**
EURNN (FFT style)	128 (FFT)	34k	52.3	52.4

Conclusion

Efficient Unitary NN (EUNN)

- **Efficient**: O(1) operation per parameter
- **Strict**: strictly unitary parametrization
- **Tunable**: from small subspace to full unitary space, trade off capacity to hidden size
- **Easy implementation**: element-wise functions, no need to implement backpropagation
- **FFT approximation**: provides further speed-up

Tunable Efficient Unitary Neural Networks (EUNN) and their application to RNNs

Li Jing [*1] **Yichen Shen** [*1] **Tena Dubcek** [1] **John Peurifoy** [1] **Scott Skirlo** [1] **Yann LeCun** [2] **Max Tegmark** [1] **Marin Soljačić** [1]

Abstract

Using unitary (instead of general) matrices in artificial neural networks (ANNs) is a promising way to solve the gradient explosion/vanishing problem, as well as to enable ANNs to learn long-term correlations in the data. This approach appears particularly promising for Recurrent Neural Networks (RNNs). In this work, we present a new architecture for implementing an Efficient Unitary Neural Network (EUNNs); its main advantages can be summarized as follows. Firstly, the representation capacity of the unitary space in an EUNN is fully tunable, ranging from a subspace of SU(N) to the entire unitary space. Secondly, the computational complexity for training an EUNN is merely $\mathcal{O}(1)$ per parameter. Finally, we test the performance of EUNNs on the standard copying task, the pixel-permuted MNIST digit recognition benchmark as well as the Speech Prediction Test (TIMIT). We find that our architecture significantly outperforms both other state-of-the-art unitary RNNs and the LSTM architecture, in terms of the final performance and/or the wall-clock training speed. EUNNs are thus promising alternatives to RNNs and LSTMs for a wide variety of applications.

1. Introduction

Deep Neural Networks (LeCun et al., 2015) have been successful on numerous difficult machine learning tasks, including image recognition(Krizhevsky et al., 2012; Donahue et al., 2015), speech recognition(Hinton et al., 2012) and natural language processing(Collobert et al., 2011; Bahdanau et al., 2014; Sutskever et al., 2014). However, deep neural networks can suffer from vanishing and exploding gradient problems(Hochreiter, 1991; Bengio et al., 1994), which are known to be caused by matrix eigenvalues far from unity being raised to large powers. Because the severity of these problems grows with the the depth of a neural network, they are particularly grave for Recurrent Neural Networks (RNNs), whose recurrence can be equivalent to thousands or millions of equivalent hidden layers.

Several solutions have been proposed to solve these problems for RNNs. Long Short Term Memory (LSTM) networks (Hochreiter & Schmidhuber, 1997), which help RNNs contain information inside hidden layers with gates, remains one of the the most popular RNN implementations. Other recently proposed methods such as GRUs(Cho et al., 2014) and Bidirectional RNNs (Berglund et al., 2015) also perform well in numerous applications. However, none of these approaches has fundamentally solved the vanishing and exploding gradient problems, and gradient clipping is often required to keep gradients in a reasonable range.

A recently proposed solution strategy is using orthogonal hidden weight matrices or their complex generalization (unitary matrices) (Saxe et al., 2013; Le et al., 2015; Arjovsky et al., 2015; Henaff et al., 2016), because all their eigenvalues will then have absolute values of unity, and can safely be raised to large powers. This has been shown to help both when weight matrices are initialized to be unitary (Saxe et al., 2013; Le et al., 2015) and when they are kept unitary during training, either by restricting them to a more tractable matrix subspace (Arjovsky et al., 2015) or by alternating gradient-descent steps with projections onto the unitary subspace (Wisdom et al., 2016).

In this paper, we will first present an Efficient Unitary Neural Network (EUNN) architecture that parametrizes the entire space of unitary matrices in a complete and computationally efficient way, thereby eliminating the need for time-consuming unitary subspace-projections. Our architecture has a wide range of capacity-tunability to represent subspace unitary models by fixing some of our parameters; the above-mentioned unitary subspace models correspond to special cases of our architecture. We also implemented an EUNN with an earlier introduced FFT-like architecture which efficiently approximates the unitary space with min-

[*]Equal contribution [1]Massachusetts Institute of Technology [2]New York University, Facebook AI Research. Correspondence to: Li Jing <ljing@mit.edu>, Yichen Shen <ycshen@mit.edu>.

imum number of required parameters(Mathieu & LeCun, 2014b).

We then benchmark EUNN's performance on both simulated and real tasks: the standard copying task, the pixel-permuted MNIST task, and speech prediction with the TIMIT dataset (Garofolo et al., 1993). We show that our EUNN algorithm with an $\mathcal{O}(N)$ hidden layer size can compute up to the entire $N \times N$ gradient matrix using $\mathcal{O}(1)$ computational steps and memory access per parameter. This is superior to the $\mathcal{O}(N)$ computational complexity of the existing training method for a full-space unitary network (Wisdom et al., 2016) and $\mathcal{O}(\log N)$ more efficient than the subspace Unitary RNN(Arjovsky et al., 2015).

2. Background

2.1. Basic Recurrent Neural Networks

A recurrent neural network takes an input sequence and uses the current hidden state to generate a new hidden state during each step, memorizing past information in the hidden layer. We first review the basic RNN architecture.

Consider an RNN updated at regular time intervals $t = 1, 2, ...$ whose input is the sequence of vectors $\mathbf{x}^{(t)}$ whose hidden layer $\mathbf{h}^{(t)}$ is updated according to the following rule:

$$\mathbf{h}^{(t)} = \sigma(\mathbf{U}\mathbf{x}^{(t)} + \boldsymbol{W}\mathbf{h}^{(t-1)}), \tag{1}$$

where σ is the nonlinear activation function. The output is generated by

$$\mathbf{y}^{(t)} = \mathbf{W}\mathbf{h}^{(t)} + \mathbf{b}, \tag{2}$$

where $\mathbf{b}$ is the bias vector for the hidden-to-output layer. For $t = 0$, the hidden layer $\mathbf{h^{(0)}}$ can be initialized to some special vector or set as a trainable variable. For convenience of notation, we define $\mathbf{z}^{(t)} = \mathbf{U}\mathbf{x}^{(t)} + \mathbf{W}\mathbf{h}^{(t-1)}$ so that $\mathbf{h}^{(t)} = \sigma(\mathbf{z}^{(t)})$.

2.2. The Vanishing and Exploding Gradient Problems

When training the neural network to minimize a cost function C that depends on a parameter vector $\mathbf{a}$, the gradient descent method updates this vector to $\mathbf{a} - \lambda\frac{\partial C}{\partial \mathbf{a}}$, where λ is a fixed learning rate and $\frac{\partial C}{\partial \mathbf{a}} \equiv \nabla C$. For an RNN, the vanishing or exploding gradient problem is most significant during back propagation from hidden to hidden layers, so we will only focus on the gradient for hidden layers. Training the input-to-hidden and hidden-to-output matrices is relatively trivial once the hidden-to-hidden matrix has been successfully optimized.

In order to evaluate $\frac{\partial C}{\partial W_{ij}}$, one first computes the derivative $\frac{\partial C}{\partial h^{(t)}}$ using the chain rule:

$$\frac{\partial C}{\partial \mathbf{h}^{(t)}} = \frac{\partial C}{\partial \mathbf{h}^{(T)}}\frac{\partial \mathbf{h}^{(T)}}{\partial \mathbf{h}^{(t)}} \tag{3}$$

$$= \frac{\partial C}{\partial \mathbf{h}^{(T)}}\prod_{k=t}^{T-1}\frac{\partial \mathbf{h}^{(k+1)}}{\partial \mathbf{h}^{(k)}} \tag{4}$$

$$= \frac{\partial C}{\partial \mathbf{h}^{(T)}}\prod_{k=t}^{T-1}\mathbf{D}^{(k)}\mathbf{W}, \tag{5}$$

where $\mathbf{D}^{(k)} = \mathrm{diag}\{\sigma'(\mathbf{U}\mathbf{x}^{(k)} + \mathbf{W}\mathbf{h}^{(k-1)})\}$ is the Jacobian matrix of the pointwise nonlinearity. For large times T, the term $\prod \mathbf{W}$ plays a significant role. As long as the eigenvalues of $\mathbf{D}^{(k)}$ are of order unity, then if $\mathbf{W}$ has eigenvalues $\lambda_i \gg 1$, they will cause gradient explosion $|\frac{\partial C}{\partial \mathbf{h}^{(T)}}| \to \infty$, while if $\mathbf{W}$ has eigenvalues $\lambda_i \ll 1$, they can cause gradient vanishing, $|\frac{\partial C}{\partial \mathbf{h}^{(T)}}| \to 0$. Either situation prevents the RNN from working efficiently.

3. Unitary RNNs

3.1. Partial Space Unitary RNNs

In a breakthrough paper, Arjovsky, Shah & Bengio (Arjovsky et al., 2015) showed that unitary RNNs can overcome the exploding and vanishing gradient problems and perform well on long term memory tasks if the hidden-to-hidden matrix in parametrized in the following unitary form:

$$\mathbf{W} = \mathbf{D}_3\mathbf{T}_2\mathcal{F}^{-1}\mathbf{D}_2\mathbf{\Pi}\mathbf{T}_1\mathcal{F}\mathbf{D}_1. \tag{6}$$

Here $\mathbf{D}_{1,2,3}$ are diagonal matrices with each element $e^{i\omega_j}, j = 1, 2, \cdots, n$. $\mathbf{T}_{1,2}$ are reflection matrices, and $\mathbf{T} = I - 2\frac{\widehat{\mathbf{v}}\widehat{\mathbf{v}}^\dagger}{||\widehat{\mathbf{v}}||^2}$, where $\widehat{\mathbf{v}}$ is a vector with each of its entries as a parameter to be trained. $\mathbf{\Pi}$ is a fixed permutation matrix. $\mathcal{F}$ and $\mathcal{F}^{-1}$ are Fourier and inverse Fourier transform matrices respectively. Since each factor matrix here is unitary, the product $\mathbf{W}$ is also a unitary matrix.

This model uses $\mathcal{O}(N)$ parameters, which spans merely a part of the whole $\mathcal{O}(N^2)$-dimensional space of unitary $N \times N$ matrices to enable computational efficiency. Several subsequent papers have tried to expand the space to $\mathcal{O}(N^2)$ in order to achieve better performance, as summarized below.

3.2. Full Space Unitary RNNs

In order to maximize the power of Unitary RNNs, it is preferable to have the option to optimize the weight matrix $\mathbf{W}$ over the full space of unitary matrices rather than a subspace as above. A straightforward method for implementing this is by simply updating $\mathbf{W}$ with standard back-propagation and then projecting the resulting matrix (which will typically no longer be unitary) back onto to the space

of unitary matrices. Defining $\mathbf{G}_{ij} \equiv \frac{\partial C}{\partial W_{ij}}$ as the gradient with respect to $\mathbf{W}$, this can be implemented by the procedure defined by (Wisdom et al., 2016):

$$\mathbf{A}^{(t)} \equiv \mathbf{G}^{(t)\dagger}\mathbf{W}^{(t)} - \mathbf{W}^{(t)\dagger}\mathbf{G}^{(k)}, \quad (7)$$

$$\mathbf{W}^{(t+1)} \equiv \left(\mathbf{I} + \frac{\lambda}{2}\mathbf{A}^{(t)}\right)^{-1}\left(\mathbf{I} - \frac{\lambda}{2}\mathbf{A}^{(t)}\right)\mathbf{W}^{(t)}. \quad (8)$$

This method shows that full space unitary networks are superior on many RNN tasks (Wisdom et al., 2016). A key limitation is that the back-propation in this method cannot avoid N-dimensional matrix multiplication, incurring $\mathcal{O}(N^3)$ computational cost.

4. Efficient Unitary Neural Network (EUNN) Architectures

In the following, we first describe a general parametrization method able to represent arbitrary unitary matrices with up to N^2 degrees of freedom. We then present an efficient algorithm for this parametrization scheme, requiring only $\mathcal{O}(1)$ computational and memory access steps to obtain the gradient for each parameter. Finally, we show that our scheme performs significantly better than the above mentioned methods on a few well-known benchmarks.

4.1. Unitary Matrix Parametrization

Any $N \times N$ unitary matrix $\mathbf{W}_N$ can be represented as a product of rotation matrices $\{\mathbf{R}_{ij}\}$ and a diagonal matrix $\mathbf{D}$, such that $\mathbf{W}_N = \mathbf{D}\prod_{i=2}^{N}\prod_{j=1}^{i-1}\mathbf{R}_{ij}$, where $\mathbf{R}_{ij}$ is defined as the N-dimensional identity matrix with the elements R_{ii}, R_{ij}, R_{ji} and R_{jj} replaced as follows (Reck et al., 1994; Clements et al., 2016):

$$\begin{pmatrix} R_{ii} & R_{ij} \\ R_{ji} & R_{jj} \end{pmatrix} = \begin{pmatrix} e^{i\phi_{ij}}\cos\theta_{ij} & -e^{i\phi_{ij}}\sin\theta_{ij} \\ \sin\theta_{ij} & \cos\theta_{ij} \end{pmatrix}. \quad (9)$$

where θ_{ij} and ϕ_{ij} are unique parameters corresponding to $\mathbf{R_{ij}}$. Each of these matrices performs a $U(2)$ unitary transformation on a two-dimensional subspace of the N-dimensional Hilbert space, leaving an $(N-2)$-dimensional subspace unchanged. In other words, a series of $U(2)$ rotations can be used to successively make all off-diagonal elements of the given $N \times N$ unitary matrix zero. This generalizes the familiar factorization of a 3D rotation matrix into 2D rotations parametrized by the three Euler angles. To provide intuition for how this works, let us briefly describe a simple way of doing this that is similar to Gaussian elimination by finishing one column at a time. There are infinitely many alternative decomposition schemes as well; Fig. 1 shows two that are particularly convenient to implement in software (and even in neuromorphic hardware (Shen et al., 2016)). The unitary matrix $\mathbf{W}_N$ is multiplied from the right by a succession of unitary matrices $\mathbf{R}_{Nj}$ for $j = N-1, \cdots, 1$. Once all elements of the last row except the one on the diagonal are zero, this row will not be affected by later transformations. Since all transformations are unitary, the last column will then also contain only zeros except on the diagonal:

$$\mathbf{W}_N\mathbf{R}_{N,N-1}\mathbf{R}_{N,N-2}\cdot\cdot\mathbf{R}_{N,1} = \begin{pmatrix} \mathbf{W}_{N-1} & 0 \\ 0 & e^{iw_N} \end{pmatrix} \quad (10)$$

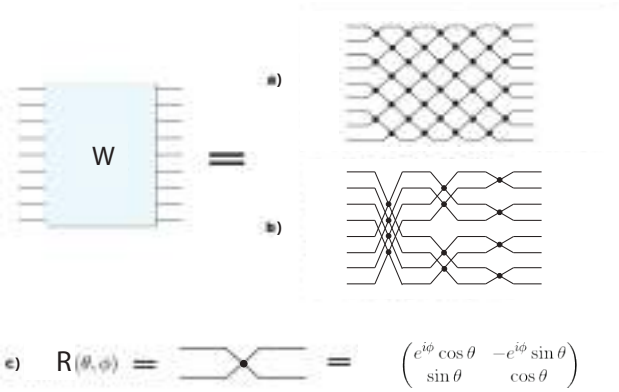


Figure 1. **Unitary matrix decomposition:** An arbitrary unitary matrix $\mathbf{W}$ can be decomposed (a) with the square decomposition method of Clements *et al.* (Clements et al., 2016) discussed in section 4.2; or approximated (b) by the Fast Fourier Transformation(FFT) style decomposition method (Mathieu & LeCun, 2014b) discussed in section 4.3. Each junction in the a) and b) graphs above represent the U(2) matrix as shown in c).

The effective dimensionality of the the matrix $\mathbf{W}$ is thus reduced to $N-1$. The same procedure can then be repeated $N-1$ times until the effective dimension of $\mathbf{W}$ is reduced to 1, leaving us with a diagonal matrix:[1]

$$\mathbf{W}_N\mathbf{R}_{N,N-1}\mathbf{R}_{N,N-2}\cdots\mathbf{R}_{i,j}\mathbf{R}_{i,j-1}\cdots\mathbf{R}_{3,1}\mathbf{R}_{2,1} = \mathbf{D}, \quad (11)$$

where $\mathbf{D}$ is a diagonal matrix whose diagonal elements are e^{iw_j}, from which we can write the direct representation of $\mathbf{W}_N$ as

$$\begin{aligned} \mathbf{W}_N &= \mathbf{D}\mathbf{R}_{2,1}^{-1}\mathbf{R}_{3,1}^{-1}\ldots\mathbf{R}_{N,N-2}^{-1}\mathbf{R}_{N,N-1}^{-1} \\ &= \mathbf{D}\mathbf{R}'_{2,1}\mathbf{R}'_{3,1}\ldots\mathbf{R}'_{N,N-2}\mathbf{R}'_{N,N-1}. \end{aligned} \quad (12)$$

where

$$\mathbf{R}'_{ij} = \mathbf{R}(-\theta_{ij}, -\phi_{ij}) = \mathbf{R}(\theta_{ij}, \phi_{ij})^{-1} = \mathbf{R}_{ij}^{-1} \quad (13)$$

[1]Note that Gaussian Elimination would make merely the upper triangle of a matrix vanish, requiring a subsequent series of rotations (complete Gauss-Jordan Elimination) to zero the lower triangle. We need no such subsequent series because since $\mathbf{W}$ is unitary: it is easy to show that if a unitary matrix is triangular, it must be diagonal.

This parametrization thus involves $N(N-1)/2$ different θ_{ij}-values, $N(N-1)/2$ different ϕ_{ij}-values and N different w_i-values, combining to N^2 parameters in total and spans the entire unitary space. Note we can always fix a portion of our parameters, to span only a subset of unitary space – indeed, our benchmark test below will show that for certain tasks, full unitary space parametrization is not necessary. [2]

4.2. Tunable space implementation

The representation in Eq. 12 can be made more compact by reordering and grouping specific rotational matrices, as was shown in the optical community (Reck et al., 1994; Clements et al., 2016) in the context of universal multiport interferometers. For example (Clements et al., 2016), a unitary matrix can be decomposed as

$$\begin{aligned}\mathbf{W}_N &= \mathbf{D}\left(\mathbf{R}^{(1)}_{1,2}\mathbf{R}^{(1)}_{3,4}\dots\mathbf{R}^{(1)}_{N/2-1,N/2}\right)\\ &\quad\times\left(\mathbf{R}^{(2)}_{2,3}\mathbf{R}^{(2)}_{4,5}\dots\mathbf{R}^{(2)}_{N/2-2,N/2-1}\right)\\ &\quad\times\dots\\ &= \mathbf{D}\mathbf{F}^{(1)}_A\mathbf{F}^{(2)}_B\dots\mathbf{F}^{(L)}_B,\end{aligned}\tag{14}$$

where every

$$\mathbf{F}^{(l)}_A = \mathbf{R}^{(l)}_{1,2}\mathbf{R}^{(l)}_{3,4}\dots\mathbf{R}^{(l)}_{N/2-1,N/2}$$

is a block diagonal matrix, with N angle parameters in total, and

$$\mathbf{F}^{(l)}_B = \mathbf{R}^{(l)}_{2,3}\mathbf{R}^{(l)}_{4,5}\dots\mathbf{R}^{(l)}_{N/2-2,N/2-1}$$

with $N-1$ parameters, as is schematically shown in Fig. 1a. By choosing different values for L , $\mathbf{W}_N$ will span a different subspace of the unitary space. Specifically,when $L = N$, $\mathbf{W}_N$ will span the entire unitary space.

Following this physics-inspired scheme, we decompose our unitary hidden-to-hidden layer matrix $\mathbf{W}$ as

$$\mathbf{W} = \mathbf{D}\mathbf{F}^{(1)}_A\mathbf{F}^{(2)}_B\mathbf{F}^{(3)}_A\mathbf{F}^{(4)}_B\cdots\mathbf{F}^{(L)}_B.\tag{15}$$

4.3. FFT-style approximation

Inspired by (Mathieu & LeCun, 2014a), an alternative way to organize the rotation matrices is implementing an FFT-style architecture. Instead of using adjacent rotation matrices, each $\mathbf{F}$ here performs a certain distance pairwise rotations as shown in Fig. 1b:

$$\mathbf{W} = \mathbf{D}\mathbf{F}_1\mathbf{F}_2\mathbf{F}_3\mathbf{F}_4\cdots\mathbf{F}_{\log(N)}.\tag{16}$$

The rotation matrices in $\mathbf{F}_i$ are performed between pairs of coordinates

$$(2pk+j, p(2k+1)+j)\tag{17}$$

where $p = \frac{N}{2^i}$, $k \in \{0,...,2^{i-1}\}$ and $j \in \{1,...,p\}$. This requires only $\log(N)$ matrices, so there are a total of $N\log(N)/2$ rotational pairs. This is also the minimal number of rotations that can have all input coordinates interacting with each other, providing an approximation of arbitrary unitary matrices.

4.4. Efficient implementation of rotation matrices

To implement this decomposition efficiently in an RNN, we apply vector element-wise multiplications and permutations: we evaluate the product $\mathbf{F}\mathbf{x}$ as

$$\mathbf{F}\mathbf{x} = \mathbf{v_1} * \mathbf{x} + \mathbf{v_2} * \mathrm{permute}(\mathbf{x})\tag{18}$$

where $*$ represents element-wise multiplication, $\mathbf{F}$ refers to general rotational matrices such as $\mathbf{F}_{A/B}$ in Eq. 14 and $\mathbf{F}_i$ in Eq. 16. For the case of the tunable-space implementation, if we want to implement $\mathbf{F}^{(l)}_A$ in Eq. 14, we define $\mathbf{v}$ and the permutation as follows:

$$\begin{aligned}\mathbf{v_1} &= (e^{i\phi_1^{(l)}}\cos\theta_1^{(l)}, \cos\theta_1^{(l)}, e^{i\phi_2^{(l)}}\cos\theta_2^{(l)}, \cos\theta_2^{(l)}, \cdots)\\ \mathbf{v_2} &= (-e^{i\phi_1^{(l)}}\sin\theta_1^{(l)}, \sin\theta_1^{(l)}, -e^{i\phi_2^{(l)}}\sin\theta_2, \sin\theta_2^{(l)}, \cdots)\\ \mathrm{permute}(\mathbf{x}) &= (x_2, x_1, x_4, x_3, x_6, x_5, \cdots).\end{aligned}$$

For the FFT-style approach, if we want to implement $\mathbf{F}_1$ in Eq 16, we define $\mathbf{v}$ and the permutation as follows:

$$\begin{aligned}\mathbf{v_1} &= (e^{i\phi_1^{(l)}}\cos\theta_1^{(l)}, e^{i\phi_2^{(l)}}\cos\theta_2^{(l)}, \cdots, \cos\theta_1^{(l)}, \cos\theta_2^{(l)}, \cdots)\\ \mathbf{v_2} &= (-e^{i\phi_1^{(l)}}\sin\theta_1^{(l)}, -e^{i\phi_2^{(l)}}\sin\theta_2, \cdots, \sin\theta_1^{(l)}, \sin\theta_2^{(l)}, \cdots)\\ \mathrm{permute}(\mathbf{x}) &= (x_{\frac{n}{2}+1}, x_{\frac{n}{2}+2}\cdots x_n, x_1, x_2\cdots).\end{aligned}$$

In general, the pseudocode for implementing operation $\mathbf{F}$ is as follows:

Algorithm 1 Efficient implementation for $\mathbf{F}$ with parameter θ_i and ϕ_i.

Input: input $\mathbf{x}$, size N; parameters θ and ϕ, size $N/2$; constant permuatation index list $\mathbf{ind_1}$ and $\mathbf{ind_2}$.
Output: output $\mathbf{y}$, size N.
$\mathbf{v_1} \leftarrow$ concatenate($\cos\theta$, $\cos\theta$ * $\exp(i\phi)$)
$\mathbf{v_2} \leftarrow$ concatenate($\sin\theta$, - $\sin\theta$ * $\exp(i\phi)$)
$\mathbf{v_1} \leftarrow \mathrm{permute}(\mathbf{v_1}, \mathbf{ind_1})$
$\mathbf{v_2} \leftarrow \mathrm{permute}(\mathbf{v_2}, \mathbf{ind_1})$
$\mathbf{y} \leftarrow \mathbf{v_1} * \mathbf{x} + \mathbf{v_2} * \mathrm{permute}(\mathbf{x}, \mathbf{ind_2})$

Note that $\mathbf{ind_1}$ and $\mathbf{ind_2}$ are different for different $\mathbf{F}$.

From a computational complexity viewpoint, since the operations $*$ and permute take $\mathcal{O}(N)$ computational steps, evaluating $\mathbf{F}\mathbf{x}$ only requires $\mathcal{O}(N)$ steps. The product $\mathbf{D}\mathbf{x}$ is trivial, consisting of an element-wise vector multiplication. Therefore, the product $\mathbf{W}\mathbf{x}$ with the total unitary

[2] Our preliminary experimental tests even suggest that a full-capacity unitary RNN is even undesirable for some tasks.

matrix $\mathbf{W}$ can be computed in only $\mathcal{O}(NL)$ steps, and only requires $\mathcal{O}(NL)$ memory access (for full-space implementation $L = N$, for FFT-style approximation gives $L = \log N$). A detailed comparison on computational complexity of the existing unitary RNN architectures is given in Table 1.

4.5. Nonlinearity

We use the same nonlinearity as (Arjovsky et al., 2015):

$$(\mathrm{modReLU}(\mathbf{z}, \mathbf{b}))_i = \frac{z_i}{|z_i|} * \mathrm{ReLU}(|z_i| + b_i) \quad (19)$$

where the bias vector $\mathbf{b}$ is a shared trainable parameter, and $|z_i|$ is the norm of the complex number z_i.

For real number input, $\mathrm{modReLU}$ can be simplified to:

$$(\mathrm{modReLU}(\mathbf{z}, \mathbf{b}))_i = \mathrm{sign}(z_i) * \mathrm{ReLU}(|z_i| + b_i) \quad (20)$$

where $|z_i|$ is the absolute value of the real number z_i.

We empirically find that this nonlinearity function performs the best. We believe that this function possibly also serves as a forgetting filter that removes the noise using the bias threshold.

5. Experimental tests of our method

In this section, we compare the performance of our Efficient Unitary Recurrent Neural Network (EURNN) with

1. an LSTM RNN (Hochreiter & Schmidhuber, 1997),
2. a Partial Space URNN (Arjovsky et al., 2015), and
3. a Projective full-space URNN (Wisdom et al., 2016).

All models are implemented in both Tensorflow and Theano, available from `https://github.com/jingli9111/EUNN-tensorflow` and `https://github.com/iguanaus/EUNN-theano`.

5.1. Copying Memory Task

We compare these networks by applying them all to the well defined *Copying Memory Task* (Hochreiter & Schmidhuber, 1997; Arjovsky et al., 2015; Henaff et al., 2016). The copying task is a synthetic task that is commonly used to test the network's ability to remember information seen T time steps earlier.

Specifically, the task is defined as follows (Hochreiter & Schmidhuber, 1997; Arjovsky et al., 2015; Henaff et al., 2016). An alphabet consists of symbols $\{a_i\}$, the first n of which represent data, and the remaining two representing "blank" and "start recall", respectively; as illustrated by the following example where $T = 20$ and $M = 5$:

```
Input:  BACCA--------------------:----
Output: ------------------------BACCA
```

In the above example, $n = 3$ and $\{a_i\} = \{A, B, C, -, :\}$. The input consists of M random data symbols ($M = 5$ above) followed by $T - 1$ blanks, the "start recall" symbol and M more blanks. The desired output consists of $M + T$ blanks followed by the data sequence. The cost function C is defined as the cross entropy of the input and output sequences, which vanishes for perfect performance.

We use $n = 8$ and input length $M = 10$. The symbol for each input is represented by an n-dimensional one-hot vector. We trained all five RNNs for $T = 1000$ with the same batch size 128 using RMSProp optimization with a learning rate of 0.001. The decay rate is set to 0.5 for EURNN, and 0.9 for all other models respectively. (Fig. 2). This results show that the EURNN architectures introduced in both Sec.4.2 (EURNN with N=512, selecting L=2) and Sec.4.3 (FFT-style EURNN with N=512) outperform the LSTM model (which suffers from long term memory problems and only performs well on the copy task for small time delays T) and all other unitary RNN models, both in-terms of learnability and in-terms of convergence rate. Note that the only other unitary RNN model that is able to beat the baseline for $T = 1000$ (Wisdom et al., 2016) is significantly slower than our method.

Moreover, we find that by either choosing smaller L or by using the FFT-style method (so that $\mathbf{W}$ spans a smaller unitary subspace), the EURNN converges toward optimal performance significantly more efficiently (and also faster in wall clock time) than the partial (Arjovsky et al., 2015) and projective (Wisdom et al., 2016) unitary methods. The EURNN also performed more robustly. This means that a full-capacity unitary matrix is not necessary for this particular task.

5.2. Pixel-Permuted MNIST Task

The MNIST handwriting recognition problem is one of the classic benchmarks for quantifying the learning ability of neural networks. MNIST images are formed by a 28×28 grayscale image with a target label between 0 and 9.

To test different RNN models, we feed all pixels of the MNIST images into the RNN models in 28×28 time steps, where one pixel at a time is fed in as a floating-point number. A fixed random permutation is applied to the order of input pixels. The output is the probability distribution quantifying the digit prediction. We used RMSProp with a learning rate of 0.0001 and a decay rate of 0.9, and set the batch size to 128.

As shown in Fig. 3, EURNN significantly outperforms LSTM with the same number of parameters. It learns faster, in fewer iteration steps, and converges to a higher classifi-

Table 1. Performance comparison of four Recurrent Neural Network algorithms: URNN (Arjovsky et al., 2015), PURNN (Wisdom et al., 2016), and EURNN (our algorithm). T denotes the RNN length and N denotes the hidden state size. For the tunable-style EURNN, L is an integer between 1 and N parametrizing the unitary matrix capacity.

Model	Time complexity of one online gradient step	number of parameters in the hidden matrix	Transition matrix search space
URNN	$\mathcal{O}(TN\log N)$	$\mathcal{O}(N)$	subspace of $\mathbf{U}(N)$
PURNN	$\mathcal{O}(TN^2+N^3)$	$\mathcal{O}(N^2)$	full space of $\mathbf{U}(N)$
EURNN (tunable style)	$\mathcal{O}(TNL)$	$\mathcal{O}(NL)$	tunable space of $\mathbf{U}(N)$
EURNN (FFT style)	$\mathcal{O}(TN\log N)$	$\mathcal{O}(N\log N)$	subspace of $\mathbf{U}(N)$

Table 2. MNIST Task result. EURNN corresponds to our algorithm, PURNN corresponds to algorithm presented in (Wisdom et al., 2016), URNN corresponds to the algorithm presented in (Arjovsky et al., 2015).

Model	hidden size (capacity)	number of parameters	validation accuracy	test accuracy
LSTM	80	16k	0.908	0.902
URNN	512	16k	**0.942**	0.933
PURNN	116	16k	0.922	0.921
EURNN (tunable style)	1024 (2)	13.3k	0.940	**0.937**
EURNN (FFT style)	512 (FFT)	9.0k	0.928	0.925

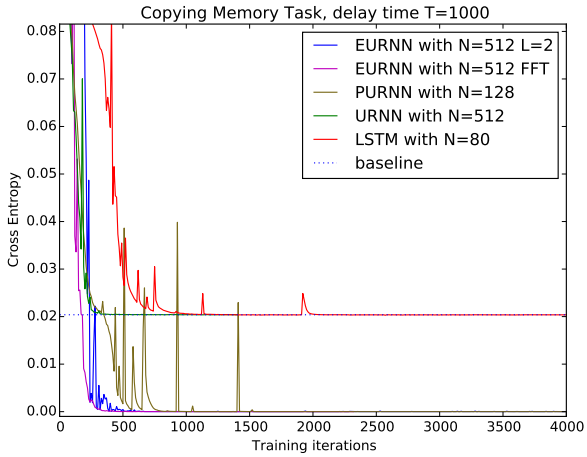


Figure 2. Copying Task for $T = 1000$. EURNN corresponds to our algorithm, projective URNN corresponds to algorithm presented in (Wisdom et al., 2016), URNN corresponds to the algorithm presented in (Arjovsky et al., 2015). A useful baseline performance is that of the memoryless strategy, which outputs $M+T$ blanks followed by M random data symbols and produces a cross entropy $C = (M\log n)/(T+2*M)$. [Note that each iteration for PURNN takes about 32 times longer than for EURNN models, for this particular simulation, so the speed advantage is much greater than apparent in this plot.]

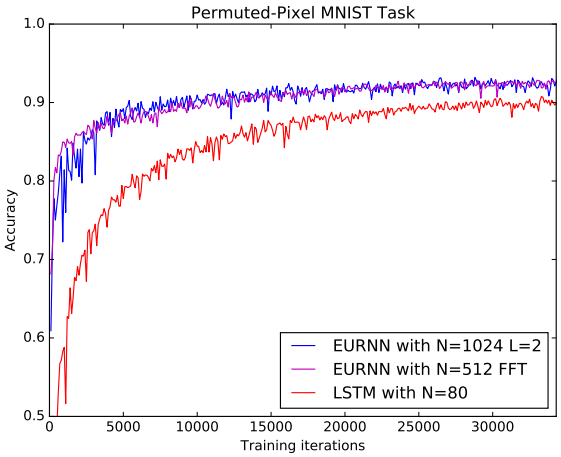


Figure 3. Pixel-permuted MNIST performance on the validation dataset.

cation accuracy. In addition, the EURNN reaches a similar accuracy with fewer parameters. In Table. 2, we compare the performance of different RNN models on this task.

5.3. Speech Prediction on TIMIT dataset

We also apply our EURNN to real-world speech prediction task and compare its performance to LSTM. The main task we consider is predicting the log-magnitude of future frames of a short-time Fourier transform (STFT) (Wisdom et al., 2016; Sejdi et al., 2009). We use the TIMIT dataset (Garofolo et al., 1993) sampled at 8 kHz. The audio **.wav** file is initially diced into different time frames (all frames have the same duration referring to the Hann analysis window below). The audio amplitude in each frame is then

Table 3. Speech Prediction Task result. EURNN corresponds to our algorithm, projective URNN corresponds to algorithm presented in (Wisdom et al., 2016), URNN corresponds to the algorithm presented in (Arjovsky et al., 2015).

Model	hidden size (capacity)	number of parameters	MSE (validation)	MSE (test)
LSTM	64	33k	71.4	66.0
LSTM	128	98k	55.3	54.5
EURNN (tunable style)	128 (2)	33k	63.3	63.3
EURNN (tunable style)	128 (32)	35k	52.3	52.7
EURNN (tunable style)	128 (128)	41k	**51.8**	**51.9**
EURNN (FFT style)	128 (FFT)	34k	52.3	52.4

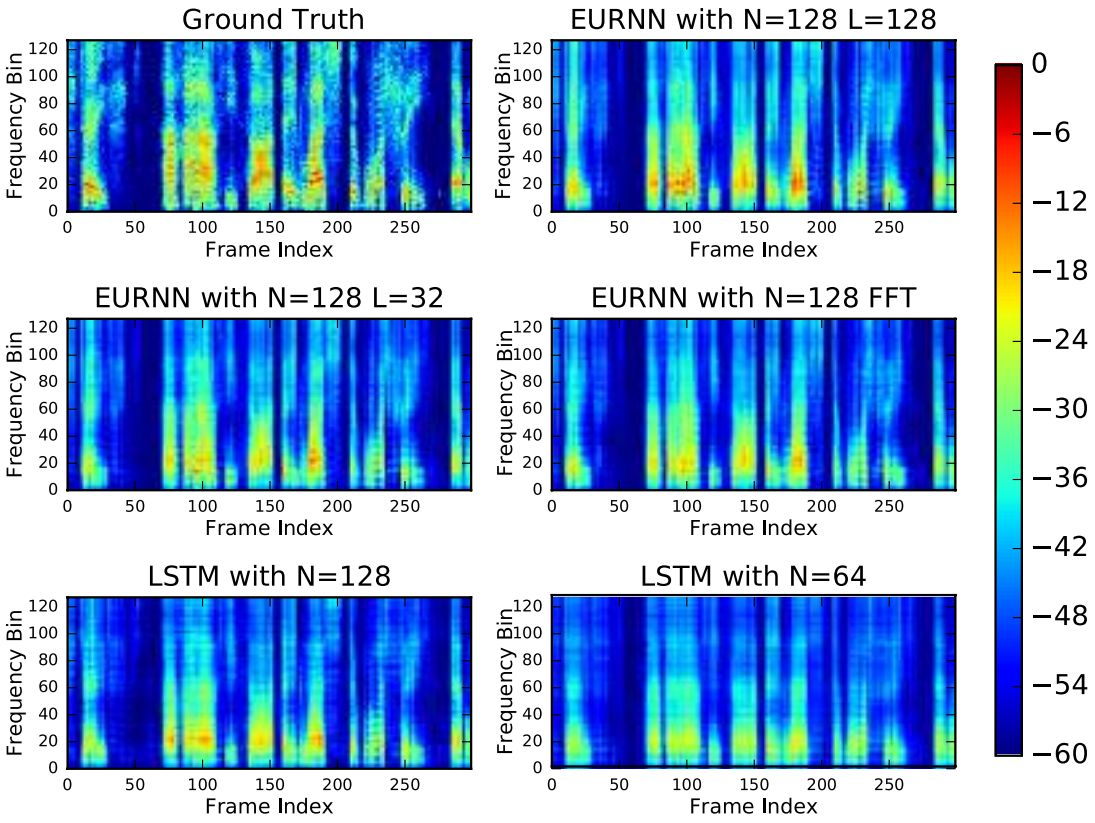


Figure 4. Example spectrograms of ground truth and RNN prediction results from evaluation sets.

Fourier transformed into the frequency domain. The log-magnitude of the Fourier amplitude is normalized and used as the data for training/testing each model. In our STFT operation we uses a Hann analysis window of 256 samples (32 milliseconds) and a window hop of 128 samples (16 milliseconds). The frame prediction task is as follows: given all the log-magnitudes of STFT frames up to time t, predict the log-magnitude of the STFT frame at time $t+1$ that has the minimum mean square error (MSE). We use a training set with 2400 utterances, a validation set of 600 utterances and an evaluation set of 1000 utterances. The training, validation, and evaluation sets have distinct speakers. We trained all RNNs for with the same batch size 32 using RMSProp optimization with a learning rate of 0.001, a momentum of 0.9 and a decay rate of 0.1.

The results are given in Table. 3, in terms of the mean-squared error (MSE) loss function. Figure. 4 shows prediction examples from the three types of networks, illustrat-

ing how EURNNs generally perform better than LSTMs. Furthermore, in this particular task, full-capacity EURNNs outperform small capacity EURNNs and FFT-style EURNNs.

6. Conclusion

We have presented a method for implementing an Efficient Unitary Neural Network (EUNN) whose computational cost is merely $\mathcal{O}(1)$ per parameter, which is $\mathcal{O}(\log N)$ more efficient than the other methods discussed above. It significantly outperforms existing RNN architectures on the standard Copying Task, and the pixel-permuted MNIST Task using a comparable parameter count, hence demonstrating the highest recorded ability to memorize sequential information over long time periods.

It also performs well on real tasks such as speech prediction, outperforming an LSTM on TIMIT data speech prediction.

We want to emphasize the generality and tunability of our method. The ordering of the rotation matrices we presented in Fig. 1 are merely two of many possibilities; we used it simply as a concrete example. Other ordering options that can result in spanning the full unitary matrix space can be used for our algorithm as well, with identical speed and memory performance. This tunability of the span of the unitary space and, correspondingly, the total number of parameters makes it possible to use different capacities for different tasks, thus opening the way to an optimal performance of the EUNN. For example, as we have shown, a small subspace of the full unitary space is preferable for the copying task, whereas the MNIST task and TIMIT task are better performed by EUNN covering a considerably larger unitary space. Finally, we note that our method remains applicable even if the unitary matrix is decomposed into a different product of matrices (Eq. 12).

This powerful and robust unitary RNN architecture also might be promising for natural language processing because of its ability to efficiently handle tasks with long-term correlation and very high dimensionality.

Acknowledgment

We thank Hugo Larochelle and Yoshua Bengio for helpful discussions and comments.

This work was partially supported by the Army Research Office through the Institute for Soldier Nanotechnologies under contract W911NF-13-D0001, the National Science Foundation under Grant No. CCF-1640012 and the Rothberg Family Fund for Cognitive Science.

References

Arjovsky, Martin, Shah, Amar, and Bengio, Yoshua. Unitary evolution recurrent neural networks. *arXiv preprint arXiv:1511.06464*, 2015.

Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Bengio, Yoshua, Simard, Patrice, and Frasconi, Paolo. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2): 157–166, 1994.

Berglund, Mathias, Raiko, Tapani, Honkala, Mikko, Kärkkäinen, Leo, Vetek, Akos, and Karhunen, Juha T. Bidirectional recurrent neural networks as generative models. In *Advances in Neural Information Processing Systems*, pp. 856–864, 2015.

Cho, Kyunghyun, Van Merriënboer, Bart, Bahdanau, Dzmitry, and Bengio, Yoshua. On the properties of neural machine translation: Encoder-decoder approaches. *arXiv preprint arXiv:1409.1259*, 2014.

Clements, William R., Humphreys, Peter C., Metcalf, Benjamin J., Kolthammer, W. Steven, and Walmsley, Ian A. An optimal design for universal multiport interferometers, 2016. arXiv:1603.08788.

Collobert, Ronan, Weston, Jason, Bottou, Léon, Karlen, Michael, Kavukcuoglu, Koray, and Kuksa, Pavel. Natural language processing (almost) from scratch. *Journal of Machine Learning Research*, 12(Aug):2493–2537, 2011.

Donahue, Jeffrey, Anne Hendricks, Lisa, Guadarrama, Sergio, Rohrbach, Marcus, Venugopalan, Subhashini, Saenko, Kate, and Darrell, Trevor. Long-term recurrent convolutional networks for visual recognition and description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2625–2634, 2015.

Garofolo, John S, Lamel, Lori F, Fisher, William M, Fiscus, Jonathon G, and Pallett, David S. Darpa timit acoustic-phonetic continous speech corpus cd-rom. nist speech disc 1-1.1. *NASA STI/Recon technical report n*, 93, 1993.

Henaff, Mikael, Szlam, Arthur, and LeCun, Yann. Orthogonal rnns and long-memory tasks. *arXiv preprint arXiv:1602.06662*, 2016.

Hinton, Geoffrey, Deng, Li, Yu, Dong, Dahl, George E, Mohamed, Abdel-rahman, Jaitly, Navdeep, Senior, Andrew, Vanhoucke, Vincent, Nguyen, Patrick, Sainath,